

Ю. В. Кагарлицкий

Разработка документации пользователя программного продукта

{методика и стиль изложения}

Ю. В. Кагарлицкий

Разработка документации пользователя программного продукта

{методика и стиль изложения}



PHILOSOFT
TECHNICAL COMMUNICATIONS

Москва, 2012

Кагарлицкий Ю. В.

Разработка документации пользователя программного продукта. Методика и стиль изложения. — 2-е изд., испр. и доп. — М.: ООО «Философт Сервисы», 2012. — 232 с., 8 табл.

Задача этой книги — помочь разработчикам технической документации создавать информативные и удобные руководства для пользователей программных продуктов. Подробно рассматриваются различные этапы создания документации: изучение программного продукта и выявление особенностей его применения; определение состава комплекта документации; отбор сведений, включаемых в документацию в целом и в каждый документ в отдельности; составление структуры отдельного документа и ее последующая корректировка; работа над текстом документации и соблюдение базовых требований к этому тексту.

Для технических писателей, программистов, инженеров служб технической поддержки.

Издание исправленное и дополненное. Первое издание вышло в 2008 г.

Оглавление

Введение.....	6
Глава 1. Постановка задачи.....	14
1.1. Действующие лица	14
1.2. Предмет документирования.....	18
1.2.1. Понятие программного продукта.....	18
1.2.2. Рабочая среда пользователя и ее составляющие	19
1.2.3. Пользователь: функции и пользовательские задачи.....	23
1.2.4. Пользователи: один или много	26
1.2.5. Понятие пользовательской перспективы.....	28
1.3. Документация пользователя и ее характеристики	31
1.3.1. Понятие документации пользователя.....	31
1.3.2. Предмет документирования	34
1.3.3. Способы чтения документации пользователем	37
1.3.4. Комплект документации	38
Глава 2. Требования к содержанию документации	55
2.1. Постановка проблемы	55
2.2. Объективная корректность	56
2.3. Соответствие документа поставленным перед ним задачам ...	60
2.3.1. Требования, определяемые задачами документа.....	60
2.3.2. Актуальность.....	60
2.3.3. Полнота	62
2.3.4. Детальность изложения.....	69
2.4. Доступность.....	77

Глава 3. Структура документации пользователя	80
3.1. О структуре документации пользователя	80
3.1.1. Структурированность — сквозное свойство документации	80
3.1.2. Содержательная и формальная структура	81
3.1.3. Первопроходец или экскурсовод?	82
3.1.4. Вводные главы: торжество формальной структуры	83
3.1.5. Основная часть документа: способы изложения материала	84
3.2. Деление документа на основные структурные элементы	89
3.2.1. Основные структурные элементы: тематический принцип	89
3.2.2. Глубина и сечение формальной структуры документа	91
3.2.3. Парадокс формальной структуры и пути его разрешения	97
3.3. Сплошной текст и его структура	103
3.3.1. Структура сплошного текста	103
3.3.2. Конструктивные структурные элементы	104
3.3.3. Маркировочные структурные элементы	111
3.3.4. Абзацы	114
3.4. Типы информации и их взаимное расположение	115
3.4.1. Общие принципы	115
3.4.2. Структурная информация	117
3.4.3. Процедурно-функциональная информация	119
3.4.4. Справочная информация	125
3.4.5. Компоновка различных типов информации	126
3.5. Проблема логического повтора	127
3.5.1. Повторяемость как свойство технического текста	127
3.5.2. Фигуры описания	128
3.5.3. Логические повторы и перекрестные ссылки	134
3.5.4. Логические повторы в структуре документа	136
3.6. Перекрестные ссылки и случаи их использования	139
Глава 4. Слова и формулировки	142
4.1. Словарный запас технического писателя	142
4.1.1. Терминология	142
4.1.2. Вспомогательная терминология	159
4.1.3. Слова-«артикли»	177
4.2. Формулировки	185
4.2.1. Требования к формулировкам	185
4.2.2. Строгость	185

4.2.3. Лаконичность.....	187
4.2.4. Завершенность.....	188
4.2.5. Однозначность.....	190

Глава 5. Некоторые проблемы изложения материала..... 207

5.1. Громоздкие и ветвящиеся процедуры.....	207
5.2. Порядок изложения сведений: сложные случаи.....	213
5.2.1. Проблема порядка изложения сведений.....	213
5.2.2. Курица или яйцо: взаимосвязанные понятия	214
5.2.3. Канонический порядок изложения сведений.....	214
5.2.4. Унификация порядка изложения сведений	218
5.3. Реальное и виртуальное.....	219
5.3.1. Реальное и виртуальное в документации пользователя	219
5.3.2. Объекты реальности, их образы и идентификаторы.....	221
5.3.3. Люди и организации, учетные записи, имена	223
5.4. Образно или строго?	225
5.4.1. Язык образов и его место в строгом изложении	225
5.4.2. Антропоморфизм и техногенность.....	225
5.4.3. Наглядные средства подачи информации	227

Введение

Книга, которую читатель держит перед собой, посвящена вопросам стиля и методики изложения, применяемым в пользовательской документации, и адресована тем, кто занимается подготовкой этой документации, — тем, кто называет себя еще недавно диковинным, а теперь все более привычным словосочетанием: технический писатель. Да, круг тех, кто посвятил себя этой сравнительно молодой и весьма востребованной профессии, неуклонно расширяется, чему свидетельством широкая популярность первого издания этой книги, вышедшего в свет пару лет назад, огромный читательский спрос на него. Скромный тираж разошелся за считанные месяцы, и книга если и не стала библиографической редкостью, то во всяком случае пропала с магазинных полок. Мы чрезвычайно рады и польщены, что профессиональное сообщество проявляет столь явный интерес к нашему детищу, хотя причины этого видим не в последнюю очередь в крайней скудости русскоязычной литературы, посвященной документированию программных продуктов и техническим коммуникациям в целом. Разумеется, это в известной мере ставит нас в выгодное положение, но и накладывает серьезную ответственность за каждое сказанное или написанное слово по теме.

Почтенная традиция велит в начале долгого пути дать краткий его обзор, сделать некоторые предварительные замечания, чтобы читатель примерно представлял себе с самого начала, о чем ему будут рассказывать, а о чем — нет.

Эта книга написана в 2008 году, и по сравнению с 1-м изданием в нее внесено мало изменений. Не потому, что автору нечего добавить и уточнить, — напротив, повседневная профессиональная практика почти каждый день заставляет размышлять над новыми вопросами и приносит неожиданные открытия. Однако книга — в том виде, в котором ее увидели первые читатели несколько лет назад, — кажется состоявшейся. Не хочется резать по живому и вставлять в нее новые темы; им можно посвятить новые публикации, доклады, новые книги, наконец. Поэтому автор ограничился легким редактированием, исправлением того, что сегодня, на свежую голову, кажется несообразностью, а также заменой некоторых примеров. Более структурированной и удобной для восприятия стала верстка книги. Содержание же глав и разделов осталось практически неизменным.

Книга включает, помимо введения, пять глав, каждая из которых посвящена какой-либо важной теме. Первая глава посвящена проблемам, которые технический писатель решает при постановке задачи документирования: с кем ему придется контактировать, кто будет снабжать его информацией, а кто — принимать работу; какой именно продукт, в каком объеме и какой версии станет предметом документирования; каковы условия целевого применения документируемого продукта и какой комплект документации необходим пользователю для успешного целевого применения продукта пользователем.

Вторая глава, небольшая, но, как кажется, очень важная, касается требований, предъявляемых к содержанию документации. Технический писатель обречен постоянно спрашивать себя, достаточно ли полно и детально он описал программный продукт, не зияют ли в документации белые пятна, не перегружена ли она непонятной или излишней информацией. О том, как отвечать на эти вопросы, рассказывается во второй главе.

В третьей главе подробно рассмотрены проблемы структурирования изложения. Именно проблемы, потому что сама по себе необходимость структурирования изложения очевидна для любого технического писателя. Однако как оптимизировать структуру документа, какие применять средства структурирования изложения, как учесть интересы пользователей, практикующих

разные формы работы с документацией, — эти вопросы ставят порой в тупик даже опытного профессионала. Хочется думать, что многие коллеги найдут здесь если не ответы на вопросы, то, во всяком случае, соображения по поводу того, в каком направлении эти ответы следует искать.

Четвертая глава посвящена словам и формулировкам, которые, по мнению автора, следует или, напротив, не следует употреблять в документации пользователя.

Наконец, в пятой, заключительной главе рассматриваются некоторые специальные проблемы изложения материала, часто становящиеся камнем преткновения для технических писателей и требующие особого подхода для их решения.

Вот, собственно говоря, о чем рассказывается в этой книге. Теперь следует сказать несколько слов о том, чего не следует ждать от этой книги, — дабы не дезориентировать читателя, впервые берущего в руки нашу книгу, и не обмануть его ожидания.

Книга, которую читатель держит перед собой, посвящена технической документации, а точнее, документации пользователя программного продукта. Однако не стоит рассматривать ее как всеобъемлющее руководство по документированию программ и систем. За пределами книги целиком или большей частью остались многие ключевые вопросы разработки документации — организация процесса разработки, выбор инструментария, оформление документов, подготовка иллюстративного материала и т. д. Этот вполне осознанный шаг продиктован отнюдь не пренебрежением к этим важнейшим темам, а, скорее наоборот, чрезвычайно серьезным к ним отношением. Каждая из них чрезвычайно обширна, попытка подробно разобрать их все в рамках одного пособия привела бы к непомерному разрастанию объема последнего, что, в свою очередь, растянуло бы сроки работы над книгой и, скорее всего, сделало бы ее тяжеловесной и трудной для изучения. К тому же автор, откровенно говоря, не чувствует себя экспертом по всем упомянутым выше темам. По этим и некоторым другим соображениям в книге рассматривается только круг вопросов, так или иначе связанных с методикой и стилем изложения, используемых при написании пользовательской документации. За консультациями же по прочим темам приходит-

ся отослать читателя к другим книгам, статьям, интернет-публикациям, учебным курсам и т. п.

Книга предназначена для широкого круга читателей — в том смысле, что не требует от читателя никаких специальных знаний и навыков. Однако хочется сразу предупредить: книга адресована в первую очередь тем, кому уже приходилось заниматься подготовкой пользовательской документации и кто столкнулся с проблемами и затруднениями при выборе способа и порядка изложения, при разработке структуры документов и определении требований к их стилю. Иными словами, читателю, не имеющему собственного опыта в этой области, какие-то суждения и соображения могут показаться не до конца понятными, не слишком актуальными, попросту не вызвать интереса; в то же время он не найдет здесь систематического введения в профессию, начальных рекомендаций, без которых трудно начать самостоятельную работу над документацией. Это не означает, что книга не может быть ему полезной, — напротив, хочется верить, что она будет ему подспорьем в грядущей профессиональной практике. Но следует иметь в виду, что она не является и не может являться учебником для начинающих. В ней рассказывается о проблемах изложения, с которыми сталкивается технический писатель, и о том, как такие проблемы, с точки зрения автора, стоит решать. Оценить же подобные рекомендации проще всего тому, кто сам сталкивался с этими проблемами на практике.

Зачем же читателю, уже имеющему опыт подготовки пользовательской документации, обращаться к какой-то специальной литературе? Неужели он не справится самостоятельно с теми задачами, которые возникают перед ним в его профессиональной деятельности? Что полезного найдет он в нашей книге?

Дело в том, что начать работать над пользовательской документацией не так уж сложно. Конечно, начинающему техническому писателю не помешали бы заготовки и шаблоны, но даже если ничего такого под рукой нет, можно просто раздобыть образцы работы старших коллег и ориентироваться на них буквально во всем — от структуры документа до стиля изложения. Но затем выясняется, что этим профессиональные премудрости в нашем деле не ограничиваются. В каком порядке расположить необхо-

димые пользователю сведения, как облегчить ему восприятие и при этом не пожертвовать строгостью и точностью информации, как логично и последовательно рассказать о взаимосвязанных явлениях и процессах — эти и подобные вопросы все чаще возникают при написании документации. Ответ на них каждый раз приходится давать применительно к конкретной ситуации, и это не всегда легко. По мере обогащения своего профессионального опыта технический писатель, с одной стороны, до автоматизма отрабатывает начальные навыки, с другой — время от времени сталкивается с проблемами структурирования и изложения материала. Вот тут ему потребуется помощь, которую, хочется верить, он хотя бы частично отыщет в этой книге.

Именно поэтому читатель не найдет здесь однозначных рецептов — как поступать в том или ином случае. Многолетняя практика общения с коллегами убедила автора в том, что технические писатели — доброжелательная, но независимо мыслящая аудитория; они обычно не спешат применять те или иные конкретные рекомендации, предпочитают опробовать разные пути решения проблемы и выбрать тот путь, который лучше всего подходит к конкретным условиям и к постановке конкретной задачи. Едва ли был смысл навязывать этим людям готовые схемы и шаблоны. Поэтому автор предпочел пойти по другому пути. Рассказывая о тех или иных проблемах и предлагая в каждом случае пути их решения, он стремился показать, как конкретные советы и рекомендации вписываются в общую логику работы над пользовательской документацией. Ему бы казалось высшей оценкой его труда не безоглядное принятие читателем предлагаемых им конкретных предписаний, а усвоение этой самой общей логики, общего подхода к документированию. Решать же конкретные задачи каждому из наших уважаемых коллег все равно придется самому. И если кто-то, опираясь на предложенный в этой книге подход, найдет, применительно к конкретным условиям, другое, более удачное решение. — автор сочтет, что его задача выполнена.

Этот проблемный взгляд на документирование призван скомпенсировать избыток норм, правил, предписаний, стандартов, присутствующих в жизни технического писателя. Следует на-

помнить и себе, и коллегам, и заказчикам пользовательской документации о том, что последняя призвана решать вполне определенную — коммуникативную, как сказал бы лингвист, — задачу: быстро и эффективно, с минимальными затратами для пользователя, сообщить ему, как достигать определенных практических целей с помощью программного продукта. Что нормы и предписания, правила и стандарты в конечном итоге существуют для того, чтобы обеспечивать оптимальное решение этой коммуникативной задачи. Поэтому на вопрос: как правильно писать то-то и то-то, как правильно расположить материал, какие документы следует подготовить в таком-то случае — в этой книге чаще всего звучит ответ: так, чтобы это позволило наилучшим образом решить поставленную задачу.

Разумеется, это отнюдь не означает, что надо отказаться от технической четкости и строгости изложения и писать в произвольном стиле, поскольку нам-де показалось, что так «будет понятнее». Напротив, принятые в технической литературе в целом и в документации в частности правила и нормы хранят бесценный опыт наших предшественников и помогают, а не мешают создавать более совершенную документацию; пренебрегать этим опытом было бы неразумно и неблагодарно. Точно так же неразумно было бы отбрасывать и те наработки, которые хранят многочисленные стандарты в области документации и в смежных областях — национальные, международные, отраслевые; в книге нередко явные или неявные отсылки к этим стандартам. Хочется, однако, предупредить тех читателей, для которых документация, написанная «не по ГОСТу», не документация: при всем уважении к стандартам, документация здесь рассматривается как совокупность документов, предназначенных для решения определенной задачи, и в книге рассматриваются пути наиболее эффективного решения этой задачи. Использование тех или иных стандартов в целом, отдельно или совместно; применение отдельных их положений; интерес к тем или иным концепциям, представленным в стандартах, — все подчинено поиску этих путей.

Выбирая такой угол зрения, автор прекрасно отдавал себе отчет в том, что для многих коллег определенный стандарт не

просто методическое пособие, а закон. **обязательный** для соблюдения, рамочное условие работы над документацией. Возможно, у них некоторые положения книги вызовут известное отторжение. Тем не менее, хочется призвать их не торопиться с выводами. Думается, что и они найдут в книге кое-что полезное для себя, а какие-то положения стандарта станут для них более естественными, органичными, привязанными к коммуникативной задаче — и менее навязанными, надуманными, привнесенными извне. В любом случае хочется подчеркнуть: автор выбрал такой угол зрения не случайно, не по недомыслию, а вполне сознательно: в этом выборе отразился взгляд — его и его коллег из компании «Философт» — на документирование в целом. Взгляд, выработанный и, не побоимся пафоса, выстраданный нами за почти полтора десятилетия работы в области документирования программных продуктов.

Разумеется, это не означает, что мы не приемлем никакой критики. Напротив, хотелось бы стимулировать читателя к критическому осмыслению как конкретных рецептов, так и общих подходов, представленных в книге. Однако автор настаивал бы и на своем праве видеть те или иные проблемы документирования так, как подсказывают ему разум и опыт. Ведь в конечном итоге именно ради возможности усвоить опыт коллег и стоит читать такие книги, как эта. Именно опыт столкновения с проблемами и их решения в концентрированном виде представлен в книге, которую читатель держит в руках. Автор осознает, что это не так уж много; он, однако, тешит себя надеждой, что и не мало.

Благодарности

Автор выражает сердечную благодарность всем, кто советом, замечанием или хотя бы просто добрым словом содействовал выходу этой книги в свет в том виде, в котором она существует ныне.

Автор не может не произнести слов горячей признательности:

- руководителю компании «Философт» М. Ю. Острогорскому, без деятельного участия которого написание книги и тем более ее выход в свет едва ли были бы возможны;
- М. А. Селисской, многолетнему партнеру по проведению тренингов для технических писателей;
- коллективу компании в целом.

Автор также хочет поблагодарить всех, кто проявил столь горячий интерес к 1-му изданию книги и тем самым убедил нас в том, что мы на верном пути и новое издание книги необходимо. Хочется верить, что оно также найдет своих читателей и расширит круг тех, кто разделяет наш подход к документированию программных продуктов и наше понимание профессионального долга технического писателя.

Постановка задачи

1.1. Действующие лица

Едва ли найдется специалист, который позволит себе поставить под сомнение ведущую роль документации в успешном внедрении программного продукта, в его эффективном освоении пользователем, а также в последующей успешной эксплуатации. И если до сих пор созданию качественной документации пользователя уделяется, как мы считаем, недостаточно внимания, так это потому, что процесс создания документации пользователя в значительной степени подчинен другим процессам, в первую очередь процессу создания самого программного средства и процессу его упорядоченной поставки: продажи, внедрения, установки по заказу и т. п. Грубо говоря, подготовка документации пользователя программного продукта в значительной степени зависит от тех, кто создает программный продукт, и от тех, кто его продает. И те, и другие не без оснований рассматривают свою деятельность как имеющую первостепенное значение. Сколько бы ни отстаивать мысль о важности документации пользователя для успешного применения программного продукта, этим нельзя отменить простого соображения: если отсутствует программа в рабочем состоянии, никакая документация не исправит дела, а если программа не найдет своего пользователя, то документацию некому будет читать.

Вот почему, хотя эта книга адресована тем, кто непосредственно создает документацию пользователя — техническим писателям,

необходимо помнить, что рабочий процесс создания документации постоянно требует участия других лиц. Эти лица имеют свои интересы, свои амбиции, да и просто свое собственное мнение относительно того, какой должна быть документация пользователя. Сделать их своими союзниками, а не оппонентами, эффективно использовать их знания и опыт и в то же время деликатно, но твердо отстаивать свою независимость от них в тех случаях, когда вопрос принадлежит к сфере профессиональной компетенции технического писателя, — задача не менее трудная, чем написать технически грамотный, пригодный для использования текст.

Вся деятельность технического писателя по подготовке документации пользователя разворачивается на трех, если можно так выразиться, рубежах:

- формирование заказа на документацию пользователя;
- работа над документацией пользователя;
- сдача готовой документации пользователя.

Мы говорим — рубежах, а не стадиях или этапах, потому что в действительности работа технического писателя разворачивается в нескольких направлениях. Выполнив часть работы, он может вернуться к уточнению заказа или, напротив, представить часть работы на сдачу раньше, чем будет готова вся документация. В процессе сдачи документации может возникнуть необходимость вернуться к работе над ней или даже к корректировке исходных требований. В этом нет ничего необычного: документация пользователя — многоплановый объект, и его жизненный цикл может быть достаточно сложным. Однако три рубежа подготовки документации пользователя существуют независимо от этих сложностей.

Формирование заказа на документацию пользователя

Основные действующие лица: технический писатель, заказчик.

Заказчиком называется лицо, выдающее техническому писателю задание на подготовку документации пользователя.

Заказчиком совсем не обязательно является какой-то посторонний человек, заключающий с техническим писателем договор

на подготовку документации пользователя к тому или иному программному средству. Отношения между заказчиком и техническим писателем могут быть самыми разными: заказчик может быть руководителем группы программистов, работающих над программным средством; начальником отдела продаж; руководителем отдела документирования и т. д.

Первый рубеж весьма важен, его не следует недооценивать. На этом рубеже формулируются и уточняются следующие рамки планируемой работы:

- требования к документации пользователя;
- порядок изучения техническим писателем программного продукта;
- календарный план работы и порядок реализации тех или иных ее этапов;
- консультанты (лица, ответственные за предоставление технического писателю необходимых сведений);
- ответственный приемщик, или кратко, приемщик (лицо, ответственное за прием готовой документации пользователя).

Работа над документацией пользователя

Основные действующие лица: технический писатель, консультанты.

Консультантом называется лицо, ответственное за предоставление техническому писателю необходимых сведений.

Консультанты чаще всего назначаются заказчиком и в дальнейшем отвечают на вопросы технического писателя.

Как правило, консультантами являются руководители проектов, программисты-разработчики, системные аналитики, постановщики задачи, или реже, менеджеры, занятые продажей программного продукта.

Удобнее всего, когда консультант один. Однако чем сложнее программный продукт, тем менее вероятно, что удастся обойтись одним консультантом. Впрочем, и чрезмерно увеличивать количество консультантов не следует, поскольку это снижает чувство ответственности каждого из них за достоверность и полноту предоставляемых сведений.

На этом рубеже технический писатель занят непосредственно подготовкой текста документации. Он в оговоренном заранее порядке получает доступ к программному продукту и знакомится с его возможностями. Дополнительную информацию он получает от консультантов. На основе полученной информации он подготавливает документы.

Фрагменты подготавливаемой документации технический писатель может обсуждать с консультантами. Обсуждение при этом не должно касаться состава комплекта документации, структуры документа, стиля, оформления — но только корректности излагаемых сведений. Следует помнить, что консультант дает лишь содержательные пояснения, но может быть не в курсе конкретной задачи, за выполнение которой технический писатель отвечает перед заказчиком.

Сдача готовой документации пользователя

***Основные действующие лица:** технический писатель, приемщик, рецензенты, корректор, литературный редактор, нормоконтролер.*

Ответственным приемщиком, или кратко, *приемщиком* называются лицо, ответственное за проверку готовой документации и вынесение решения о приемки документации. Приемщик совпадает с заказчиком или назначается им и в дальнейшем является его полномочным представителем.

Рецензентом называется лицо, знакомящееся с документацией и представляющее свои замечания, исправления и дополнения. Рецензенты назначаются приемщиком в том случае, если он не справляется с рецензированием представленной ему работы из-за ее объема, специфики или иных причин.

Крайне нежелательно, чтобы приемка работы осуществлялась несколькими лицами сразу, поскольку это неизбежно делает технического писателя заложником скрытых и открытых разногласий между приемщиками. Оптимальным является вариант, когда приемщик вместе с техническим писателем обсуждает поступившие от рецензентов замечания, устанавливает иерархию значи-

мости этих замечаний, указывает, какие пожелания рецензентов следует исполнить, а какими пренебречь и т. д.

Иногда в требованиях к документации серьезную роль играют *формальные требования*: языковая правильность, стилистическая правильность, соответствие стандартам. В этом случае назначаются *корректор, литредактор, нормоконтролер*. Эти лица проверяют документ с точки зрения тех или иных формальных требований.

1.2. Предмет документирования

1.2.1. Понятие программного продукта

Здесь и далее мы всюду пользуемся понятием программного продукта, поэтому необходимо, во-первых, дать определение этого понятия, а во-вторых, пояснить, от каких понятий мы здесь стремимся дистанцироваться.

Под *программным продуктом* мы понимаем программу или набор программ, поставляемых как единое целое, вместе с документацией и иногда вместе с данными, для решения тех или иных задач¹.

В состав программного продукта входит одна или несколько программ. Тем не менее, если программный продукт имеет единую процедуру запуска и единый пользовательский интерфейс для работы со всеми его функциями, мы для краткости называем его *программой*. Например: «программа складского учета товаров». Если же программный продукт состоит из двух или более компонентов, каждый из которых запускается особо и имеет обособленный пользовательский интерфейс, такой программный продукт мы называем *программным комплексом*².

¹ Ср. определение программного продукта в ГОСТ Р ИСО/МЭК 12207-99: «Набор машинных программ, процедур и, возможно, связанных с ними документации и данных».

² Существуют также программные продукты, традиционно содержащие в своем наименовании слово «система»: система управления базами данных; информационная система; операционная система.

Перед техническим писателем может стоять задача документирования не только программного продукта, но и технических решений более комплексного характера: программируемых или управляемых устройств, аппаратно-программных комплексов, автоматизированных систем — иными словами, технических решений, которые вовлекают, помимо тех или иных программ, аппаратные средства, технологические процессы, схемы распределения должностных обязанностей на предприятии или в организации и т. п. В настоящем руководстве мы ограничиваемся рассмотрением документации пользователя на программные продукты, хотя многие рекомендации приложимы и к документированию комплексных решений.

1.2.2. Рабочая среда пользователя и ее составляющие

Документация пользователя поставляется пользователю программного продукта и относится к этому программному продукту. В соответствии с этим, мы говорим о пользовании программой, ее функциональными возможностями. Следует, однако, принимать в расчет, что пользователь практически никогда не работает непосредственно с программой, а действует в определенной рабочей среде.

Рабочую среду пользователя образуют:

- рабочий компьютер пользователя или другое рабочее устройство: специализированный терминал, смартфон, электронная книга, навигатор и т. д.;
- операционная система;
- сторонние программы;
- хранящиеся на доступных устройствах данные;
- периферийные устройства, подключенные к рабочему устройству, аппаратура.

Если рабочий компьютер подключен к локальной или глобальной сети, что в последние годы стало скорее правилом, чем исключением, в рабочую среду также входят:

- компьютеры, периферийные устройства и аппаратура, доступные по сети;

- сторонние программы, функционирующие на других компьютерах;
- данные, хранящиеся на других компьютерах в сети или распределенно.

Ко всем составляющим рабочей среды могут предъявляться определенные требования: к техническим характеристикам компьютера и периферийных устройств, к операционной системе, к наличию на компьютере дополнительного программного обеспечения, к наличию доступных данных и т. д. Требования могут быть не только строгими, но и рекомендательными, иметь не только позитивный, но негативный, ограничительный характер. Например, некоторые программы могут оказаться необходимыми для нормальной эксплуатации документируемого продукта, некоторые необязательными, но весьма полезными для полноценного его использования, некоторые, — напротив, конкурировать с ним за ресурсы или за данные, даже конфликтовать с ним, блокировать его работу. В связи с этим наличие одних программ на рабочем компьютере будет обязательным, других желательным, третьих нежелательным, четвертых недопустимым.

Желательно как можно раньше выяснить все эти требования и ограничения не только потому что они в обязательном порядке должны быть приведены в документации, но и потому что при документировании необходимо представлять себе диапазон, в котором могут варьироваться характеристики составляющих рабочей среды. Дело в том, что технический писатель, как и пользователь, работает с программным продуктом, установленным в конкретной рабочей среде, — тогда как в документации должны быть отражены особенности применения программного продукта во всех допустимых вариантах рабочей среды. Например, работа в разных операционных системах может быть построена несколько по-разному, окна и элементы пользовательского интерфейса могут иметь разный вид, иногда при смене графического режима возникают те или иные проблемы в работе программы.

Итак, следует отделять программный продукт как таковой (программу или набор программ, поставляемый пользователю; см. п. 1.2.1) от конкретной установки программного продукта. При документировании необходимо проводить границу между

особенностями, свойствами, характеристиками программного продукта и конкретной его установки. Важно, чтобы пользователь также ощущал эту границу.

В случае если программный продукт передается пользователю в виде установочного комплекта, эта граница дана в ощущении: вот установочный комплект (продукт как таковой), а вот его конкретная установка (у нее есть конкретные параметры, определяемые рабочей средой; установок может быть несколько, каждая со своими параметрами и т. д.).

В случае же если пользователь работает с программой, уже установленной на его компьютере или в сети, — он всегда имеет дело с конкретной установкой программного продукта, тогда как в документации пользователя описан программный продукт как таковой.

Для технического писателя это означает, что ему надлежит учесть в документации пользователя и то, и другое, разъяснить пользователю, какие черты программного продукта в нем наличествуют в неизменном виде, какие — зависят от рабочей среды и меняются от установки к установке.

Вообще, надо помнить, что программный продукт не всегда дан пользователю в четких границах. Он может иметь визуально выраженный пользовательский интерфейс (например, главное окно с названием продукта в заголовке), а может быть доступен в виде дополнительного пункта меню в другой программе. Работа пользователя с программным продуктом может строиться по-разному — в виде последовательности сеансов, во время каждого из которых пользователь в диалоговом режиме задействует определенные функциональные возможности; в виде периодических обращений к определенным сервисам во время работы с другими программами (программы конвертации файловых форматов, электронные словари); в виде постоянно работающих без вмешательства пользователя сервисов (антивирусы, программы проверки орфографии). Для того чтобы начать работать с программным продуктом, пользователь предпринимает определенные действия, характер которых также может быть различным: иногда это запуск экземпляра (instance) программы, иногда — запуск экземпляра клиентской части программного комплекса

и подключение к серверу, иногда — вызов диалогового окна уже работающей программы; иногда работа с программным продуктам начинается автоматически, сразу после запуска компьютера и загрузки операционной системы.

Технический писатель, занимаясь подготовкой документации, должен — в первую очередь в уме для самого себя, а потом и в тексте для пользователя — четко сформулировать:

- как и в каких формах документируемый продукт присутствует в рабочей среде;
- как организована во времени работа с программным продуктом;
- как начать и завершить работу с программным продуктом.

В этом случае пользователь сможет сразу понять, где и как он получает доступ к программному продукту, как ему приступить к его применению — информация не менее, а иногда и более важная, чем конкретные пошаговые описания процедур. Следует, однако, помнить, что, хотя пользователь обращается при решении конкретных практических задач к определенному интерфейсу, который он опознает как интерфейс программного продукта, — в формировании ответа могут принимать участие, помимо программного продукта, другие компоненты рабочей среды. Программный продукт может выполнять определенное действие сам, а может вызывать функцию операционной системы или сторонней программы. По большому счету, пользователю не так уж важно, какой компонент именно ответствен за какое действие, ему достаточно понимания, что он ведет диалог с системой в целом. В систему входит экземпляр программного продукта, с которым он работает в настоящий момент, но также и другие экземпляры того же продукта (те, например, с которыми работают другие пользователи), а также все, что составляет рабочую среду (см. выше). Необходимость понимать, какие компоненты выполняют то или иное действие, решают ту или иную задачу, возникает у пользователя тогда, когда от этого зависит решение определенной проблемы: возникает конфликт из-за ресурсов или данных, определенная операция требует настройки не только самого программного продукта, но и сторонних программ, в результате нарушения канала данных выдается иной ответ на запрос и т. п. На наиболее типичные ситу-

ации такого рода пользователю надлежит указать в документации. Поэтому техническому писателю следует внимательно изучить архитектуру системы и характер взаимодействия между ее компонентами, понять, какое место в системе занимает программный продукт. Надо помнить, что хотя документация подготавливается для программного продукта, в нее зачастую включается описание компонентов, каналов передачи данных, процессов и процедур, внешних по отношению к программному продукту как таковому.

Итак, для того чтобы четко определить границы предмета документирования, мало знать, какой именно программный продукт (в какой версии и комплектации) предстоит документировать. Для технического писателя и для его читателя — пользователя чрезвычайно важны следующие сведения:

- требуемый состав рабочей среды пользователя, требования к ее компонентам, накладываемые на нее ограничения;
- возможные особенности работы конкретной установки программы (в зависимости от конкретной рабочей среды);
- присутствие и визуальное представление программного продукта в рабочей среде, организация работы с ним во времени, действия по началу и завершению работы с ним;
- архитектура системы в целом и характер взаимодействия между ее компонентами.

1.2.3. Пользователь: функции и пользовательские задачи

Пользователем программного продукта, или для краткости *пользователем*, мы называем любое лицо, которое взаимодействует с программным продуктом, используя функциональные возможности последнего для решения тех или иных практических задач.

Совокупность средств, которая обеспечивает передачу информации между пользователем — физическим лицом — и программным продуктом, называется *пользовательским интерфейсом* программного продукта³.

³ Ср. определение пользовательского интерфейса (user interface), данное авторитетной зарубежной организацией: „interface that enables information to be passed between a human user and hardware or software components of a computer

Под функциональной возможностью, или коротко *функцией* программного продукта, мы понимаем возможность обработки данных или настройки программного продукта, предоставляемую через интерфейс и реализуемую как последовательность действий пользователя и/или операций, выполняемых программным продуктом.

Пример. Текстовый редактор оснащен функцией копирования текстового фрагмента в буфер обмена.

Пример. Программа учета рабочего времени оснащена функцией формирования отчетов.

Функции чаще всего образуют не ряд независимых, реализованных отдельно друг от друга возможностей, а иерархическую систему: более элементарные функции используются в ходе реализации более сложных, комплексных.

Пример. В текстовом редакторе комплексная функция настройки шрифта использует функции выбора гарнитуры шрифта, выбора размера шрифта и выбора шрифтового выделения.

Под *пользовательской задачей* мы понимаем задачу, возникающую в практической деятельности пользователя и решаемую за счет использования функций программного продукта.

Пример. В программе работы с электронной почтой предусмотрены пользовательские задачи создания нового сообщения (с последующей отправкой или без таковой), пакетной отправки нескольких сообщений, получения сообщений (с последующим просмотром или без такового).

system“ («интерфейс, который делает возможной передачу информации между пользователем-человеком и аппаратными либо программными компонентами компьютерной системы») — IEEE Std 610.12-1990.

В дальнейшем мы для краткости будем вместо термина «пользовательский интерфейс» употреблять менее громоздкий, хотя и менее точный термин «интерфейс», поскольку ни о каких других интерфейсах, кроме пользовательского, в этой книге не говорится.

Функция и пользовательская задача представляют собой два взаимосвязанных понятия: функция обретает смысл в контексте решения тех или иных пользовательских задач, а пользовательская задача может быть решена только с использованием тех или иных функций. В то же время эти два понятия соответствуют двум противоположным взглядам на программный продукт. Условно можно сформулировать разницу между этими взглядами так: функции — это то, что программный продукт предлагает пользователю, тогда как пользовательские задачи — это то, чего пользователь требует от программного продукта.

Следует отметить, что все пользовательские задачи решаются с помощью функций программного продукта, но не все функции непосредственно задействованы в решении пользовательских задач. Дело в том, что функции программного продукта нередко предназначены для настройки программного продукта, его обслуживания или оптимизации его использования.

Пример. Программа работы с электронной почтой оснащена функцией настройки кодировки отправляемых сообщений. Эта функция не участвует напрямую в решении какой-либо пользовательской задачи, но подготавливает программный продукт к наиболее удачному решению целого ряда пользовательских задач.

Пример. Система управления базами данных оснащена функцией оптимизации базы данных, позволяющей уменьшить пространство, занимаемое базой данных на диске. Эта функция также не участвует в решении пользовательских задач, но служит для оптимизации работы программного продукта.

Для дальнейшего изложения важно отметить, что иногда пользователю при освоении программного продукта приходится отталкиваться от функций программного продукта, а иногда — от пользовательских задач. Более конкретные соображения по этому поводу приведены в п. 3.1.5.

1.2.4. Пользователи: один или много

Мы различаем *однопользовательские* и *многопользовательские* программные продукты. *Однопользовательским* называется программный продукт, с которым одновременно работает только один пользователь. Это не означает, что к такому продукту не могут получить доступ для работы двое, трое и более пользователей. Однако, во-первых, в каждый момент времени с ним работает только один пользователь, а во-вторых, в самом продукте нет разделения прав доступа пользователей к функциональным возможностям и данным: «для него» пользователь всегда один.

Многопользовательским называется программный продукт, в котором предполагается разграничение работы с ним разных пользователей. Например, корпоративная почтовая программа, установленная в организации и поддерживающая работу с ней нескольких пользователей, является многопользовательским программным продуктом.

В многопользовательских программных продуктах пользователи могут иметь различные *пользовательские роли*. Под *пользовательской ролью* мы понимаем класс пользователей, которые решают сходные задачи и используют при этом одни и те же функции. Пользовательские роли подразделяются на две категории — *целевые* и *служебные*. Первые связаны с целевым применением программного продукта, вторые — с обслуживанием программ или данных, входящих в состав программного продукта. К служебным ролям относятся роли инженера по установке, администратора и т. п. Служебные роли обычно стоят особняком, предполагают использование особых функций, связанных с обслуживанием программного продукта; документация для носителей служебных ролей тоже подготавливается особая. Далее мы как бы выносим служебные роли за скобки — там, где говорится о наличии в программном продукте одной или нескольких пользовательских ролей, по умолчанию имеются в виду целевые роли.

В приведенном нами примере корпоративной почтовой программы все пользователи являются носителями одной пользовательской роли. Такие программные продукты называются *одно-*

ролевыми. Однако во многих программных продуктах предусмотрено несколько (целевых) пользовательских ролей.

Пример. В программе электронного документооборота и делопроизводства могут быть предусмотрены роли *Исполнитель* (готовит документ, направляет его руководителю и/или делопроизводителю), *Руководитель* (подписывает или утверждает документ либо ставит на нем отметку о согласовании), *Делопроизводитель* (регистрирует документ, направляет его руководителю).

Такие программные продукты называются *многоролевыми*.

Многоролевые программные продукты обычно предназначены для внедрения в организациях и на предприятиях. При внедрении многоролевого программного продукта он адаптируется к условиям конкретной организации (предприятия) и эксплуатируется как программное обеспечение или часть программного обеспечения автоматизированной системы.

Типы программных продуктов с точки зрения пользовательских ролей перечислены в таблице 1.

Таблица 1. Типы программных продуктов с точки зрения пользовательских ролей

Тип	Описание	Пример
Однопользовательские	Один пользователь	Текстовый редактор
Однорольевые	Одна пользовательская роль, к которой относятся все пользователи, работающие с программным продуктом	Корпоративная почтовая программа
Многоролевые	Несколько пользовательских ролей, носитель каждой роли выполняет свою часть работы с данными и вносит свой вклад в общую работу	Программа электронного документооборота и делопроизводства

1.2.5. Понятие пользовательской перспективы

Каждому пользователю предоставляется набор возможностей, ограничений, рамочных условий взаимодействия с программным продуктом — *пользовательская перспектива*. В одном случае эта перспектива связана с бухгалтерским учетом и выполнением основных операций с бухгалтерскими документами, в другом — с ведением мониторинга технологического процесса, в третьем — с участием в цикле документооборота. Именно детальное описание пользовательской перспективы и составляет основное содержание документации пользователя.

Однако в разных случаях — для разных программных продуктов, для разных пользовательских ролей многопользовательского программного продукта — пользовательские перспективы устроены по-разному. В одних случаях пользователь должен четко следовать предлагаемым ему инструкциям, в других — он учится самостоятельно применять определенные навыки, в третьих — реагировать на определенные ситуации. В одних случаях он работает в диалоге с программой в собственном смысле слова, то есть вводит одни данные и получает на выходе другие; в других случаях он создает рабочий объект и выполняет различные операции с этим объектом; в третьих — выполняет операции с объектом, созданным другим пользователем. Все это позволяет говорить о разных *типах* пользовательских перспектив.

Мы выделяем несколько наиболее распространенных типов пользовательских перспектив (таблица 2).

В многопользовательских многоролевых программных продуктах каждой роли соответствует своя пользовательская перспектива, причем типы пользовательских перспектив зачастую не совпадают.

Для дальнейшего изложения важно отметить, что при освоении пользователем программного продукта состав необходимых ему сведений существенно зависит от типа пользовательской перспективы: в одних случаях (тип *Среда*) пользователю необходим полный свод сведений об элементарных функциях-инструментах и методические рекомендации по решению наиболее

типичных задач; в других (тип *Инструмент*) — полный перечень пользовательских задач и указания по решению каждой из них.

Таблица 2. Типы пользовательских перспектив

Тип	Описание	Примеры
<i>Среда</i>	Пользователю предоставляется возможность с помощью используемых в различном порядке элементарных функций решать неограниченное количество пользовательских задач. Как правило, пользователь создает рабочий объект, представляющий собой виртуальную модель реального объекта (документа, рисунка, макета книги и т. д.), а затем ставит перед собой ту или иную задачу, имеющую практический смысл, и решает ее путем использования ряда элементарных функций. В выборе того или иного действия пользователь ограничен предоставляемым ему набором функций (впрочем, достаточно обширным), в выборе порядка действий — не ограничен, по существу, ничем	Текстовый процессор, издательская система, графический редактор
<i>Инструмент</i>	Пользователь получает возможность решать пользовательские задачи из ограниченного набора, предусмотренного создателями программного продукта. Для решения каждой из задач выполняется определенная, наперед заданная последовательность действий. По отношению к программному продукту пользователь свободен в выборе задачи и в определении ее исходных параметров, но общий порядок его действий предопределен	Почтовая программа, программа бухгалтерского учета

Тип	Описание	Примеры
<i>Поток</i>	Пользователь обслуживает определенный поток данных, решая пользовательские задачи, предусмотренные его служебными обязанностями. По отношению к программному продукту пользователь в известной мере свободен в выборе задачи, однако эта свобода уже, чем для типа <i>Инструмент</i>: круг задач определяется конкретными, внешними по отношению к пользователю обстоятельствами, исходные параметры задачи определяются предшествующей судьбой обрабатываемых данных и т. д. Общий порядок действий пользователя при решении той или иной задачи предопределен (как и для типа <i>Инструмент</i>)	Программа электронного делопроизводства и документооборота, программа складского учета товаров (обычно реализуется в многопользовательских многопользовательских программах)
<i>Реальное время</i>	Пользователь обслуживает процесс или ряд внешних процессов, протекающих в реальном времени. Программный продукт используется для своевременного контроля, управления или реагирования на внешний процесс. Действия пользователя подчинены внешним требованиям и в значительной мере предопределены, поскольку: круг пользовательских задач — реакций на типичные нештатные ситуации предопределен; пользовательская задача ставится извне и требует оперативного решения; исходные параметры задачи определяются ситуацией. Свобода пользователя ограничивается только выбором одной из возможных реакций, причем выбором, осуществляемым в весьма узких временных рамках	Программы мониторинга технологического процесса, мониторинга состояния охранной или противопожарной системы

1.3. Документация пользователя и ее характеристики

1.3.1. Понятие документации пользователя

Любым требованиям предшествует общее представление о том, что такое вообще документация пользователя. В чем специфика этого предмета? Невозможно приступить ни к разработке, ни даже к формулированию основных требований, не уяснив себе специфику того, над чем будет вестись работа.

Любой объект можно определить двумя способами: по тому, чем он является, и по тому, чем он не является. Что является документацией пользователя? Авторитетный стандарт дает на это следующий ответ: «Полный комплект документов, поставляемых в печатном или другом виде, который обеспечивает применение продукта, а также является неотъемлемой частью продукта»⁴. Это определение нуждается в уточнениях и комментариях.

Во-первых, обратим внимание на термин *продукт*, который указывает на то, что пользователь имеет дело не с абстрактным программным средством, а с товаром, поставляемым в том или ином виде. Программным средством может быть любая программа, разработанная профессиональным программистом, инженером, ученым или кем-то еще для решения прикладных задач. Создатель такой программы может пользоваться ею сам или предоставлять ее в пользование коллегам, партнерам и вообще любым другим людям. Однако если существует упорядоченная процедура поставки программного средства лицам и/или организациям: продажа установочных комплектов, установка в офисе или дома силами специалистов, разработка на заказ с последующей установкой и наладкой — программное средство считается *программным продуктом* (кратко: *продуктом*) и в обязательном порядке снабжается необходимой документацией, в частности *документацией пользователя*.

Во-вторых, внесем некоторые уточнения в термин «применение продукта». Вообще говоря, любое использование программного продукта в тех или иных целях является его применением. Например, встроенный графический редактор какого-нибудь

⁴ ГОСТ Р ИСО/МЭК 12119-2000, п. 2.4.

прикладного программного средства в принципе не возбраняется использовать также и для создания и редактирования совершенно посторонних графических образов. Однако каждый программный продукт имеет определенное назначение. Применение программного продукта в соответствии с предусмотренным разработчиками назначением называется *целевым применением* продукта.

В-третьих, добавим в определение уточняющее слово: «...целевое применение продукта пользователем...». Это уточнение представляется нам очень существенным: программный продукт может применяться не только пользователем, но и, к примеру, разработчиком автоматизированной системы, который включает программный продукт в разрабатываемую систему в качестве компонента. В этом случае также необходима документация, но не пользовательская, а *эксплуатационная документация* иного характера — описания программных интерфейсов, возможностей интеграции с другими программными и аппаратными средствами, возможностей модификации программного кода и т. д. О понятии эксплуатационной документации см. ниже.

Итак, *документацией пользователя* программного продукта называется полный комплект документов, поставляемых в печатном или другом виде, который обеспечивает целевое применение продукта пользователем, а также является неотъемлемой частью этого продукта.

В приведенном определении одно слово остается не до конца ясным: «...обеспечивает...». Очевидно, речь идет об обеспечении пользователя информацией: ему сообщают сведения, необходимые для целевого применения программного продукта. При этом сведения сообщаются в такой форме, чтобы можно было усвоить их оперативно, за короткий срок.

Что не является документацией пользователя?

Во-первых, ею не являются различные средства обучения. Масштабные программные средства, особенно те, что обслуживают сложные, специализированные предметные области, нуждаются в систематическом, поэтапном освоении пользователем. Для этого разрабатываются учебники, учебные пособия, интерактивные курсы, обучающие слайд-шоу. Они включают в себя методически эффективное изложение материала, сопровождаемое

подробным разбором примеров, упражнениями, контрольными задачами, итоговыми тестами и т. п. Вся эта продукция может быть весьма полезна, а иногда и необходима пользователю, но она не включается нами в понятие документации пользователя и не рассматривается как часть последней. Документация пользователя предназначена для оперативного получения сведений, необходимых для целевого применения программного продукта, а не для постепенного усвоения навыков работы с программным продуктом в течение более или менее протяженного периода.

Во-вторых, документацией пользователя не являются руководства и справочники общего характера, посвященные не конкретным программным продуктам, а определенным разновидностям программных продуктов (например, системам управления базами данных, геоинформационным системам и т. п.), а также научные труды, посвященные определенным разновидностям информационных технологий. Документация пользователя всегда посвящена конкретному программному средству, хотя и может включать в минимальном объеме общие сведения о программных средствах данного типа.

В-третьих, документацией пользователя не являются служебные инструкции, хотя бы и содержащие положения, касающиеся работы того или иного сотрудника или должностного лица с программным средством. Документация пользователя не предписывает пользователю никаких действий, а содержит лишь сведения о том, как с помощью программного средства решить ту или иную практическую задачу.

В-четвертых, документацией пользователя не являются другие разновидности технической документации: технические задания, пояснительные записки к проектам и другие документы, создаваемые по ходу разработки программного средства. Более того, документацией пользователя не является эксплуатационная документация, адресованная персоналу, обслуживающему программное средство и обеспечивающему его эффективное целевое применение: системным программистам, администраторам, инженерам по установке. Документация пользователя всегда адресована пользователю и рассматривает программное средство только в перспективе пользовательской активности.

Документация пользователя призвана существенно облегчить, а в некоторых случаях — сделать возможной работу пользователя. Поэтому очень часто говорят о том, что технический писатель, создавая документацию, должен ориентироваться на пользователя. Это верно, но лишь в определенной степени.

Документация пользователя должна быть адаптирована к особенностям восприятия пользователя. Она, по возможности, должна оперировать знакомыми пользователю понятиями, учитывать его ограниченную способность к усвоению новых сведений и к осмыслению сложных взаимосвязей между объектами, предоставлять ему широкие возможности поиска информации и т. д. Однако документация пользователя в неменьшей степени призвана адаптировать умения и навыки самого пользователя к программному продукту. Обращаясь к документации, пользователь привыкает к точному, недвусмысленному техническому языку, приучается воспринимать информацию в виде однотипных блоков, настраивается на определенный порядок действий с программой.

Таким образом, цель документации пользователя — предоставить пользователю максимальную широту возможностей в решении конкретных задач и в то же время унифицировать его деятельность на уровне подходов и приемов. Можно сказать, что документация пользователя ориентирована не просто на пользователя, но на гипотетического идеального пользователя, к которому она подтягивает пользователя реального. Эту задачу мы всегда будем иметь в виду, обсуждая пути создания качественной документации пользователя.

1.3.2. Предмет документирования

В этой книге, как уже говорилось, мы сосредотачиваем наше внимание на документировании программных продуктов и по большей части выносим за скобки документирование аппаратно-программных комплексов, автоматизированных систем и т. п. Таким образом, постановка задачи документирования в первую очередь предполагает четкую идентификацию программного продукта.

Четко идентифицировать программный продукт означает не только сообщить техническому писателю его полное название, но и указать способ получения доступа к программному продукту в объеме, необходимом для документирования. В одних случаях техническому писателю предоставляется установочный комплект программного продукта, в других — полный или ограниченный доступ к пользовательскому интерфейсу на фиксированном рабочем месте, в третьих — демонстрационная версия программного продукта. Так или иначе способ получения доступа к программному продукту в обязательном порядке должен быть оговорен заранее и согласован с заказчиком. Иначе может оказаться, что технический писатель обращается к одному источнику, а заказчик, рецензент и приемщик считают аутентичным другой.

Однако, даже если программный продукт идентифицирован достаточно четко, иногда возникает необходимость уточнить его границы. В противном случае при сдаче документации приемщику могут возникнуть недоразумения и даже серьезные проблемы.

Во-первых, следует провести границу между программным продуктом и смежными программными средствами, в особенности — того же производителя. Заказчик подчас склонен не обращать внимание на границы между программным комплексом и отдельным его компонентом; между двумя компонентами программного комплекса; между программным продуктом и служебной программой («утилитой»), которая вызывается для выполнения какой-либо специфической функции; между программным продуктом и берущей на себя часть функций операционной системой. Мы рекомендуем сразу установить, какие смежные программные средства будут документироваться наряду с программным продуктом, в каких случаях будет предполагаться по умолчанию, что пользователь владеет этими программными средствами на должном уровне, в каких — необходимо сослаться на документацию к ним.

Во-вторых, следует провести границу между версиями программного продукта. При получении заказа на документацию пользователя необходимо зафиксировать версию и в дальнейшем внимательно отслеживать любые версионные изменения. Проблема состоит в том, что во время работы над документацией

в программный продукт (если он все еще находится в развитии, что скорее правило, чем исключение) вносятся изменения. Такое «ползучее» развитие продукта часто приводит к разногласиям при сдаче работы техническим писателем — он подготовил документацию к старой версии, а надо было учесть вновь внесенные изменения. Мы рекомендуем четко оговорить в самом начале работы версию программного продукта, которую предстоит документировать. Если по ходу документирования версия меняется, следует своевременно обратить внимание заказчика на новые обстоятельства и тем самым избежать недоразумений при сдаче документации приемщику или вынужденном переносе сроков окончания работы.

В-третьих, следует провести границу между различными вариантами поставки программного продукта, в особенности между базовым вариантом и вариантом, устанавливаемым в определенной организации и адаптируемым к ее условиям. Технический писатель должен точно знать с самого начала, к какому варианту или вариантам поставки он готовит документацию⁵.

Необходимо еще в начале работы, на первом рубеже подготовки документации, обсудить с заказчиком «границы» программного продукта и в дальнейшем по возможности придерживаться этих границ. Иногда, впрочем, границы все-таки нарушаются и уже в ходе работы над документацией выявляется необходимость документировать не только сам программный продукт, но и отдельные функции смежных программных средств, или отразить в документации версионные изменения программного продукта, или учесть различия между разными вариантами поставки программного продукта. В этом случае следует опять-таки обратить внимание заказчика на необходимость скорректировать

⁵ Иногда перед техническим писателем (чаще перед группой из одного или нескольких технических писателей) ставится более сложная задача: подготовить единый, всеобъемлющий комплект документации, на основе которого можно будет впоследствии оперативно подготовить комплект документации для того или иного варианта поставки. Подобные задачи могут успешно решаться на основе технологий единого источника (single source), однако рассмотрение этих технологий выходит за рамки настоящей книги.

объем и сроки выполнения работы при сдаче работы приемщику или вынужденном переносе сроков окончания работы.

1.3.3. Способы чтения документации пользователем

Мы выделяем два основных способа чтения документации пользователем:

- систематическое чтение;
- поиск конкретных сведений.

Первый способ предполагает, что пользователь знакомится со сведениями о применении программного продукта в том порядке, в котором они представлены в документации. Он может просто знакомиться с этими сведениями либо усваивать их, чередуя чтение документации и пробное использование тех или иных функций программного продукта. Однако важно, что его исходным намерением является не немедленное решение стоящих перед ним пользовательских задач, а оперативное освоение программного продукта в целом.

Второй способ предполагает, что пользователь периодически нуждается в получении тех или иных конкретных сведений в процессе применения программного продукта и обращается к документации за этими сведениями. Обращение к документации с этой целью означает, что пользователь либо уже освоил программный продукт, либо счел, что в систематическом освоении этого продукта нет необходимости; он нуждается лишь в том, чтобы время от времени уточнять некоторые конкретные вопросы.

При подготовке документации пользователя следует учитывать интересы пользователей, прибегающих как к первому, так и ко второму способу. Для этого используются следующие подходы:

- структура и логика изложения выбираются таким образом, чтобы сделать наиболее удобным и эффективным систематическое чтение и усвоение сведений, тогда как аппарат документации: оглавления, указатели, перекрестные ссылки — облегчает поиск конкретных сведений;
- бумажная документация предназначена преимущественно для систематического чтения и усвоения сведений, тог-

да как контекстная справка, поставляемая вместе с программным продуктом, позволяет, не отрываясь от работы, оперативно найти конкретные сведения, необходимые для успешного применения тех или иных функций и решения тех или иных пользовательских задач;

- одни документы создаются для систематического чтения и усвоения сведений, тогда как другие — для поиска конкретных сведений в тех случаях, когда это необходимо. Например, руководство пользователя предназначается скорее для систематического чтения, справочник пользователя — для поиска.

Перечисленные подходы обычно используются совместно, что позволяет обеспечить пользователю, в зависимости от его текущих предпочтений, как эффективное систематическое усвоение сведений, так и оперативный поиск ответа на тот или иной частный вопрос.

1.3.4. Комплект документации

1.3.4.1. Формирование комплекта: два подхода

В п. 1.3.1 нами было дано определение документации пользователя программного средства, в центре которого находилось словосочетание «полный комплект документов». Читатель, хотя бы раз сталкивавшийся с задачей самостоятельного создания документации пользователя, обязательно задаст уточняющий вопрос: из каких же документов состоит этот «полный комплект»?

К ответу на этот вопрос можно подойти двояко. Можно полагать, что документация пользователя складывается из отдельных документов. При этом исходят или из положений какого-либо стандарта, устанавливающего состав комплекта, или просто из устоявшихся представлений, — например, что в комплект входят руководство пользователя (в печатном виде) и контекстная справка (в одном из принятых форматов). Иными словами, первичны отдельные документы, а комплект документации является по отношению к ним производным.

Можно, напротив, полагать, что первична документация пользователя как единое целое — текстовый массив, обеспечивающий пользователя сведениями, а разбиение этого массива на отдельные документы определяется соображениями удобства чтения и восприятия документации.

Каждый из этих подходов имеет свои достоинства и недостатки. Первый подход удобнее тем, что пользователю предоставляется набор привычных документов, с аналогами которых он уже работал ранее, осваивая другие программные продукты⁶. Удобнее этот подход и для заказчика; зачастую именно заказчик указывает, какие документы он хотел бы видеть в составе документации пользователя. Однако программные продукты могут быть очень разными и по назначению, и по условиям применения, и по составу пользователей. Устанавливать заранее «типичный» состав комплекта — значит лишать себя определенной гибкости, обрекать себя на постоянные сомнения в необходимости того или иного документа и в то же время нуждаться в дополнительных документах, постоянно колебаться по поводу того, как распределить сведения между документами в комплекте.

Второй подход гораздо более гибок. У нас не возникает колебаний по поводу необходимости того или иного документа или по поводу правильности распределения сведений между документами, поскольку деление документации на документы осуществляется не на основе каких-то внешних шаблонов, а исходя из удобства пользования документацией. Однако если мы полностью откажемся от каких-либо канонов и традиций в области документирования, у нас может получиться комплект документации, непривычный для пользователя по составу и подаче материала.

Предлагаемый нами подход является компромиссом между первым и вторым подходом. Мы считаем, что, с одной стороны, объективно существует массив сведений, которые необходимо передать пользователю в удобной для восприятия форме и с этой целью разбить на отдельные документы, соответствующие разным пользователям, разным способам чтения, разным темам и т. п.; с

⁶ В настоящей книге мы не рассматриваем вопросы применения стандартов на документацию пользователя, и в частности, стандартов, устанавливающих более или менее фиксированный состав комплекта документации пользователя.

другой стороны, существуют традиционные жанры документов: руководство пользователя, справочник пользователя, — которые соответствуют привычной практике пользования документацией. Поэтому проектирование комплекта документации представляет собой работу одновременно в двух направлениях — отталкиваясь от единого целого и от более или менее знакомой пользователю «сетки» документов.

Разбиение документации пользователя на отдельные документы предполагает, что эти документы различаются как содержанием, так и структурой и стилем изложения сведений. При разбиении документации пользователя на отдельные документы соблюдаются следующие общие принципы:

- Принцип профилирования.

В составе комплекта документации пользователя выделяются документы, различающиеся по адресату, тематике и/или основному способу чтения документа (систематическое чтение или поиск конкретных сведений).

Примеры. Руководство пользователя и руководство администратора предполагают разных адресатов; руководство к разным компонентам программного комплекса предполагают разную тематику; руководство пользователя и справочник пользователя предполагают разный способ чтения.

- Принцип пользовательской перспективы.

Документ, адресованный носителю той или иной пользовательской роли, строится в соответствии с пользовательской перспективой адресата.

Примеры. Руководство бухгалтера содержит сведения о выполнении операций бухгалтерского учета с помощью программного продукта; руководство диспетчера — сведения об основных критических ситуациях и способах реакции на них со стороны пользователя программы мониторинга; руководство инженера-проектировщика — сведения об элементарных функциях-инструментах системы автоматизированного проектирования и методах решения прикладных задач.

- Принцип консерватизма.

Документы, входящие в состав документации пользователя, по возможности делаются номинально соответствующими общепринятой классификации.

Примеры. Руководство пользователя, справочник администратора, технологическая инструкция.

При необходимости документы общепринятых видов и жанров модифицируются. Лишь в особых случаях вводятся документы, не имеющие аналога в других комплектах документации.

1.3.4.2. Профилирование документов

Под профилированием документов понимается подготовка серии документов, относящихся к одному и тому же предмету, но содержательно различающихся по какому-то одному признаку. Например, документы относятся к одному и тому же программному комплексу, но каждый посвящен какому-то одному компоненту (профилированы по тематике). Или например, в одном документе собраны сведения, предназначенные для одной группы пользователей, в другом — для другой, в третьем — для третьей (профилированы по адресату). Профилирование облегчает пользователю ознакомление с необходимыми ему в данный момент сведениями, сокращает его усилия по поиску и усвоению информации.

Документы профилируются, главным образом, по следующим признакам:

- адресат;
- основной способ чтения;
- тематика.

Наиболее важным с практической точки зрения является профилирование по адресату, поскольку особенно важно разделить сведения между разными адресатами; профилирование по основному способу чтения и по тематике применяется для того, чтобы сделать чтение документации более удобным.

Профилирование по адресату предполагает учет, главным образом, следующих его характеристик:

- квалификация;
- потребности;
- полномочия.

Иными словами, документ, адресованный определенной группе пользователей, в идеале должен содержать те, и только те сведения, которые:

- доступны для усвоения пользователями данной группы;
- необходимы этим пользователям для успешного целевого применения программного продукта;
- соответствуют полномочиям этих пользователей.

Например, пользователю могли бы быть полезны определенные сведения относительно структуры базы данных системы, но квалификации, которой, как мы полагаем, он в типичном случае обладает, для понимания этих сведений недостаточно. Или, например, пользователь вполне в состоянии усвоить сведения об использовании тех или иных функций программы и даже иметь намерение ими воспользоваться, но, в силу распределения полномочий среди пользователей программного продукта, не имеет права пользоваться ими⁷.

При документировании однопользовательских программных продуктов все документы адресуются одному и тому же пользователю. Пользователь является более или менее полномочным распорядителем программного продукта (по крайней мере, в части его целевого применения), а также данных, обрабатываемых с помощью этого продукта. Он ставит перед собой определенные задачи, выполняет действия, необходимые для их решения, создает и редактирует объекты, формирует с помощью программного продукта выходные документы и т. д. Вся эта деятельность пользователя направлена на достижение им его собственных практических целей — личных, служебных, производственных.

⁷ Конечно, если пользователь не имеет права воспользоваться той или иной функцией, скорее всего, у него это и не получится. Но если мы ненароком снабдим его сведениями о применении этой функции, ему придется самому, порой методом проб и ошибок, выяснять, предоставлены ли ему полномочия пользоваться этой функцией или нет, а это не слишком удобно.

В однопользовательских программных продуктах профилирование по адресату применяется только для одной цели — опубликовать отдельно сведения, предназначенные для пользователей с различной квалификацией. Часто выделяются особые документы, адресованные начинающим пользователям; так, в комплект документации может включаться так называемый «быстрый старт» — компактный документ, позволяющий оперативно освоить основные приемы работы с программным продуктом и приступить к эксплуатации если не всех функций программного продукта, то хотя бы ограниченного их объема.

Между тем, основным адресатом документации остается пользователь, стремящийся освоить целевое применение программного продукта в полном объеме (*пользователь полного профиля*). Для этого он должен иметь исчерпывающее представление о работе с программным продуктом, обо всех его функциях и всех видах работы с данными. Исходя из общей концепции двух основных способов чтения документации, пользователю целесообразно предоставить два документа. Первый из них предназначен для систематического чтения — в нем последовательно и подробно описаны этапы установки, настройки и эксплуатации программного продукта. Второй предназначен для поиска конкретных сведений — в нем описание всех функций программного продукта представлено в удобной для поиска форме. Первый документ хорошо известен нам как *руководство пользователя*, второй — как *справочник пользователя*⁸. Таким образом, документы, входящие в комплект документации к однопользовательскому программному продукту, профилированы по основному способу чтения.

Для многопользовательских однорольевых программных продуктов приобретает актуальность разграничение носителей целевой роли и носителей одной или нескольких служебных ролей (администратора, инженера по установке и т. п.). Таким образом, профилирование по адресату учитывает не только квалификацию последнего, но и его потребности и полномочия. Исходя из

⁸ Справочник пользователя подготавливается далеко не всегда. Зачастую его роль играет контекстная справка (online help) по программному продукту.

этого, в комплект включаются, к примеру, следующие документы: руководство пользователя; справочник пользователя (для пользователя полного профиля); «быстрый старт» (для начинающего пользователя); руководство администратора; справочник администратора (для администратора); руководство по установке (для инженера по установке).

Следует обратить внимание на очень важное сходство между однопользовательскими и многопользовательскими однорольвыми программными продуктами: центральной фигурой является пользователь. Во втором случае, как и в первом, пользователь применяет программный продукт для достижения своих собственных целей — хотя пользователей может быть несколько, они решают свои задачи независимо друг от друга. Как и в первом случае, пользователь по своему усмотрению выполняет те или иные действия, создает и редактирует объекты, формирует выходные документы — хотя здесь для каждого отдельного пользователя возможны ограничения на доступ к определенным данным и функциональным возможностям. Как и в первом случае, каждый пользователь фактически работает с программным продуктом в одиночку, хотя на самом деле пользователей может быть много. Носители же служебных ролей лишь обеспечивают полноценную и эффективную работу всех пользователей с программным продуктом.

В некоторых случаях на профилирование по адресату и по основному способу чтения накладывается профилирование по тематике.

Так, крупные программные продукты часто состоят из отдельных компонентов, обособленных по функциональности и по пользовательскому интерфейсу. В этих случаях для каждого компонента создается отдельное руководство пользователя.

В некоторых случаях программные продукты оснащаются функциями или комплексами функций, существенным образом обособленными от остальных функций программного продукта, имеющими специфический интерфейс и узкую, специализированную область применения.

Пример. Программный продукт имеет встроенный язык программирования, позволяющий создавать сложные сценарии работы. Эта возможность необходима далеко не всем пользователям, характер действий по созданию сценария принципиально отличается от эксплуатации основной части функций программного продукта. Поэтому обычно нет смысла помещать описание встроенного языка программирования в общее руководство пользователя; целесообразнее тематически выделить это описание в отдельный документ.

Многопользовательские многоролевые программные продукты сталкивают нас с совершенно новой ситуацией. Обычно они предназначены для обеспечения рабочих процессов на предприятиях и в организациях. Достаточно сложно рассматривать многоролевые программные продукты вне конкретных условий их внедрения. Чаще всего ставится задача документирования не программного продукта, а автоматизированной системы, включающей в себя средства автоматизации определенных рабочих процессов на конкретном предприятии или в организации.

Автоматизированная система включает в себя техническое и программное обеспечение, установленное на этом предприятии (в этой организации), оснащенные всем необходимым автоматизированные рабочие места для персонала, а также организационные решения, лежащие в основе применения средств автоматизации в повседневной деятельности персонала. Многоролевые программные продукты зачастую используются как платформы для развертывания автоматизированных систем.

Пример. Программный комплекс электронного документооборота и делопроизводства, будучи внедрен в конкретной организации, становится основой автоматизированной системы электронного документооборота и делопроизводства. Эта система включает в себя программный комплекс, настроенный в соответствии с правилами документооборота и делопроизводства в организации, а также рабочие места, оснащенные компьютерами с установленными клиентскими компонентами программного комплекса.

Коль скоро речь идет об автоматизированной системе, следует различать документацию на программный продукт (программную документацию) и документацию на автоматизированную систему (системную документацию).

В настоящей книге мы не рассматриваем проблемы документирования автоматизированных систем, поэтому речь пойдет лишь о программной документации. Однако следует учитывать весьма вероятную перспективу последующего внедрения программного продукта на предприятиях и/или в организациях и использования его в качестве основы для автоматизированных систем.

Многоролевой программный продукт предполагает, что в процессах ввода-вывода и обработки данных участвует сразу несколько пользователей, причем каждый из них задействует только часть функций, имеющую отношение к его пользовательской роли.

Пример. В программе электронного документооборота предусмотрены пользовательские роли *Исполнитель*, *Руководитель* и *Делопроизводитель*. В ходе документооборота документы переходят от пользователя к пользователю, а те задействуют, в зависимости от своей роли, те или иные функции. Так, при работе с исходящим документом функции распределяются между ролями следующим образом:

Роль	Функции
Исполнитель	• Создание и редактирование документа • Отправка документа на согласование руководителю • Отправка документа на подпись руководителю
Руководитель	• Согласование документа • Подписание документа • Отправка документа делопроизводителю на регистрацию
Делопроизводитель	• Регистрация документа • Отправка документа в другую организацию

Однако руководитель не пользуется функцией регистрации документа, а делопроизводитель — функциями создания и редактирования документа⁹.

Из приведенного примера видно, что ни один из носителей пользовательской роли, вообще говоря, не работает с данными и объектами на всем протяжении их жизненного цикла. В этом и состоит коренное отличие многоролевых программных продуктов от однопользовательских и одноролевых: невозможно последовательное описание действий пользователя на всех этапах обработки данных, от начала до завершения, поскольку носитель той или иной роли отвечает лишь за свой участок работы. Следовательно, ему необходимы сведения только о тех функциях, которые задействуются на этом участке. О процессе же в целом пользователю необходимы лишь общие сведения: основные виды данных, объекты обработки, этапы, участие носителей других пользовательских ролей.

Таким образом, руководство пользователя для многоролевого продукта в своей содержательной части должно состоять из общего описания процессов ввода-вывода и обработки данных и описания всех функций программного продукта, причем каждая функция описывается детально и независимо от остальных функций. Пользователь по мере надобностей обращается к описанию той или иной функции.

Из этого руководства можно почерпнуть детальные сведения о том, как воспользоваться той или иной функцией, однако сведений о том, в какой ситуации следует воспользоваться той или иной функцией, там нет. Последовательность действий носителя каждой пользовательской роли является принадлежностью не программного продукта, но его конкретной установки — автоматизированной системы, созданной на конкретном предприятии или в организации. В разных автоматизированных системах на базе одного и того же программного продукта носители одной и той же роли могут действовать по-разному.

⁹ Чтобы не загромождать изложение, состав пользовательских ролей и соответствующих им функций в приведенной в качестве примера программе электронного документооборота и делопроизводства существенно упрощен по сравнению с реальными образцами подобного программного обеспечения.

Пример. В описанной выше программе электронного документооборота предполагается, что руководитель отправляет подписанный им документ на регистрацию делопроизводителю. Однако в конкретной автоматизированной системе на базе этой программы руководитель подразделения отправляет документ на регистрацию одному делопроизводителю, руководитель организации или его заместитель — другому. В общих чертах эти действия совпадают, однако в деталях отличаются друг от друга.

Дело в том, что роли в автоматизированной системе могут не совпадать с пользовательскими ролями, предусмотренными в программном продукте, а возникать на пересечениях пользовательских ролей и конкретных должностных позиций.

Пример. Пользовательской роли *Руководитель* в автоматизированной системе будут соответствовать роли *Начальник отдела*, *Заместитель директора* и *Директор*. Всем этим ролям в программе соответствует общий интерфейс и общий набор доступных функций, однако типичные ситуации и выполняемые в этих ситуациях операции будут несколько различаться.

При документировании автоматизированной системы для каждой роли создается особый документ, называемый *технологической инструкцией*. В технологической инструкции описаны операции, которые носитель данной роли должен выполнять в той или иной ситуации, возникающей в его повседневной деятельности. При этом описание операций дается в общем виде, пользовательский интерфейс и конкретные особенности программного обеспечения не рассматриваются.

Пример. В технологической инструкции для роли *Исполнитель* указывается:

Получив поручение подготовить исходящий документ, исполнитель выполняет следующие операции:

1. Создает новый исходящий документ.
2. Вводит/редактирует его текст.
3. По завершении редактирования направляет документ на согласование руководителю подразделения.

Технологическая инструкция компактна, и пользователь — носитель той или иной роли в автоматизированной системе — получает возможность оперативно выяснить, какие именно функции программного обеспечения он должен задействовать в текущей ситуации и каким образом. Сами функции здесь не описываются, для этого существует программная документация. Технологическая же инструкция является частью не программной, но системной документации, и следовательно, находится за рамками настоящей книги. Она составляется уже при внедрении программного продукта на предприятии или в организации и находится в тесной связи не только с продуктом, но и должностными обязанностями сотрудников¹⁰.

Таким образом, профилирование по адресату для многоролевых программных продуктов должно в первую очередь учитывать полномочия носителей разных ролей и их потребности, однако это профилирование осуществляется в явном виде на уровне системной, а не программной документации. Руководство пользователя к такому программному продукту состоит из независимых описаний каждой из функций продукта. Пользователю удобнее всего иметь это руководство как в бумажном, так и в электронном варианте, чтобы проще было найти нужную функцию и ознакомиться со сведениями по ее применению. Хотя руководство пользователя, как правило, представляет собой единый документ, оно распадается на независимые части, профилированные по тематическому принципу.

Следует заметить, что в эксплуатации многоролевого программного продукта участвуют носители не только целевых, но и служебных ролей. Для них подготавливаются такие же документы, как и в случае одноролевого программного продукта: руководство администратора, руководство по установке и т. п.

Здесь рассмотрены лишь основные принципы профилирования документов по адресату, по способу чтения и по тематике.

¹⁰ Для облегчения последующего внедрения программного продукта на стадии подготовки программной документации часто составляется ряд типовых технологических инструкций (для каждой пользовательской роли — своя). В этом случае технологические инструкции составляются на основе разработанных поставщиками типовых вариантов внедрения продукта.

В каждом отдельном случае необходимо тщательное изучение состава пользователей, их деления на роли и их запросов. Только по результатам этого изучения можно сделать вывод о необходимости профилирования документов по том или иному признаку.

1.3.4.3. Пользовательская перспектива

В п. 1.3.4.2 мы подробно остановились на профилировании документов по различным признакам, в первую очередь — по адресату. Однако комплект документов, предоставляемый пользователю (группе пользователей, носителям определенной пользовательской роли), и в особенности содержание и структура отдельных документов связаны не только с профилированием документа по адресату, способу чтения и тематике, но и с характером взаимодействия пользователя и программного продукта, то есть с тем, что мы называем пользовательской перспективой (см. п. 1.2.5).

Пользовательская перспектива типа *Инструмент* предполагает решение с помощью программного продукта ограниченного числа пользовательских задач. Поэтому руководство пользователя будет содержать формулировки этих задач, с последующим описанием решения каждой задачи средствами программного продукта. Таким образом, руководство пользователя представляет собой документ, структура которого в основном выстроена в соответствии с перечнем пользовательских задач¹¹. Если исходить из того, что руководство пользователя выстроено так, чтобы его читали подряд, а справочник пользователя — так, чтобы в нем искали конкретные сведения, то естественно, чтобы справочник был структурирован иным способом, нежели руководство. Поэтому будет целесообразным структуру справочника выстроить, напротив, в соответствии с перечнем функций про-

¹¹ В большинстве случаев работа с программным продуктом не исчерпывается решением пользовательских задач и включает в себя также установку, настройку, обслуживание программного продукта и данных и, возможно, некоторые другие действия. Структура этих разделов выстраивается несколько по-иному (о проектировании структуры см. главу 4). Однако основная масса сведений о целевом применении программы-инструмента в руководстве пользователя структурируется по отдельным пользовательским задачам.

граммного продукта. Читая руководство пользователя систематически, подряд, пользователь отталкивается от тех или иных задач, возникающих у него в практической деятельности, и от каждой задачи переходит к конкретным способам ее решения в программном продукте. Испытывая необходимость получить исчерпывающее представление о назначении и реализации той или иной функции, пользователь обращается к справочнику пользователя и разыскивает там сведения об этой функции¹².

Пример. Почтовая программа снабжается руководством пользователя и справочником пользователя. Руководство пользователя содержит сведения о том, как создать и послать письмо адресату или группе адресатов; проверить почту и ознакомиться с полученными письмами; ответить на полученное письмо; переслать письмо другому адресату и т. д. Справочник пользователя содержит описание всех функций, используемых в программном продукте. При этом дается ссылка на конкретные пользовательские задачи; скажем, функция *Отправить* (отправка письма) используется в пользовательских задачах *Создать и отправить письмо* и *Ответить на полученное письмо*.

Пользовательская перспектива типа *Среда* сталкивает нас с принципиально иной ситуацией: количество пользовательских задач, решаемых в программе-среде, очень велико, потенциально неограниченно. Это означает, что не получается дать пользователю документ, в котором будут перечислены все задачи и последовательно изложен порядок их решения. Вместо этого ему должны быть предоставлены сведения о том, как с помощью последовательного применения ряда элементарных функций решать любые задачи.

¹² Такое распределение способов изложения материала между руководством пользователя и справочником пользователя имеет еще и то основание, что эти документы, по самой своей природе, имеют разные критерии полноты сведений: руководство пользователя должно, по умолчанию, как можно более полно освещать вопросы применения программного продукта для решения тех или иных задач, справочник же пользователя должен содержать как можно более полное описание самого программного продукта, его функций.

Мы предлагаем для этих целей трехступенчатую реализацию — пользователю должны быть предоставлены три документа или один документ, распадающийся на три раздела:

- а) Введение в работу с программой. Содержит сведения по следующим темам:
 - установка, настройка, обслуживание (если они входят в компетенцию пользователя);
 - начало работы с программным продуктом, открытие очередного сеанса работы;
 - среда и основные рабочие объекты;
 - наиболее простые задачи и пути их решения.
- б) Справочник пользователя. Содержит описание всех функций программного продукта, в том числе всех элементарных функций, с помощью которых реализуется работа в среде.
- с) Методическое руководство. Содержит сведения по следующим темам:
 - приемы работы в среде;
 - решение наиболее типичных прикладных задач.
 - пути модификации приведенных в руководстве решений.

Пользователь в первую очередь знакомится с *введением в работу с программой*. В этом документе содержится описание общего подхода к работе и дается описание решения наиболее простых пользовательских задач.

Пример. При работе с текстовым редактором к этим задачам будет отнесено создание и сохранение нового документа, набор текста, простейшие задачи форматирования. Далее пользователь расширяет свое представление о возможностях программы сразу в двух направлениях. Если ему необходимо ознакомиться с тем, какие еще возможности предоставляются в среде, он обращается к *справочнику пользователя* и узнает оттуда, что существуют функции создания таблиц, вставки рисунков и т. п.; если ему необходимо узнать, как создаются наиболее распространенные виды документов, и путем модификации или комбинирования предложенных решений создать решение своей, достаточно сложной, задачи, он обращается к *методическому руководству*.

Пользовательская перспектива типа *Поток* обычно реализуется в многопользовательских многоролевых программных продуктах. Как уже отмечалось в п. 1.3.4.2, в этих случаях невозможно рассматривать задачу документирования программного продукта в отрыве от задачи последующего внедрения этого продукта на предприятии или в организации и превращения его в основу для автоматизированной системы. Если пользователь включается в потоковую обработку данных, пользовательские задачи ставятся перед ним извне, ходом рабочего процесса, тогда как функции программного продукта он применяет по своему усмотрению. Таким образом, постановка и порядок решения задачи, с одной стороны, и средства для ее решения, с другой, описываются раздельно: постановка задачи и порядок ее решения — в системной документации, средства же — в программной. Поэтому руководство пользователя включает в себя изолированные описания отдельных функций, из которых складывается деятельность носителей разных пользовательских ролей. Конкретные задачи и порядок их решения возникают только в контексте процесса и описываются в технологических инструкциях (см. п. 1.3.4.2).

Пользовательская перспектива типа *Реальное время* предполагает деление всех действий пользователя на два вида: инициативные действия пользователя, предпринимаемые с той или иной целью, и реакции на те или иные ситуации, фиксируемые программным продуктом в реальном времени. Инициативные действия пользователя организованы так же, как и в пользовательской перспективе типа *Инструмент* или, реже, *Среда*. Реакции на те или иные ситуации описываются в специальном справочнике (конкретное название документа зависит от природы обслуживаемого процесса и характера вмешательства пользователя). В этом документе сведения структурированы с соответствии с перечнем сигналов, которые пользователь получает через интерфейс программного продукта или иные каналы и по которым он судит о возникновении той или иной ситуации. В удобной, обычно табличной, форме пользователю сообщаются следующие сведения по каждому из сигналов:

- Сигнал.
Сигналами могут быть стандартные сообщения о событиях, появляющиеся в отдельном окне или специально отведенной области окна; изменения пользовательского интерфейса программного продукта; изменения характеристик аппаратных средств и т. д.
- Интерпретация.
Кратко разъясняется смысл сигнала, его соответствие той или иной ситуации.
- Ответные действия пользователя.
Описываются так же, как и действия пользователя при решении пользовательской задачи: в зависимости от намерений пользователя ему предлагается та или иная последовательность действий.

Сведения размещены в справочнике таким образом, чтобы пользователь мог оперативно найти сведения об интересующем его сигнале, интерпретировать его и предпринять необходимые действия.

Таким образом, конкретный состав, содержание и структура документов, предоставляемых тому или иному пользователю, существенным образом зависят от типа пользовательской перспективы. Анализ пользовательской перспективы (или перспектив — для многоролевых программных продуктов) необходимо провести на самом начальном этапе планирования работы над документацией; только в этом случае можно хотя бы в первом приближении представить себе контуры будущей документации пользователя и от этого эскизного представления перейти к проектированию структуры отдельных документов.

Требования к содержанию документации

2.1. Постановка проблемы

Содержание документации пользователя программного продукта определяется основной задачей — обеспечить целевое применение программного продукта. Иными словами, в документации пользователя содержатся сведения, обеспечивающие целевое применение программного продукта, а именно:

- сведения о назначении программного продукта, области его целевого применения и ограничениях в части его применения;
- описание интерфейса программного продукта и назначения его элементов;
- перечень пользовательских задач, решаемых с помощью программного продукта;
- описание конкретных действий пользователя, с помощью которых решается та или иная пользовательская задача;
- условия успешного решения задач, список возможных неудач и ошибок и способы их устранения.

Впрочем, вероятно, любой технический писатель представляет себе в общих чертах, что именно должен включать тот или иной документ. Гораздо более важно сформулировать требования, предъявляемые к содержанию документации в целом и отдельных документов. Редко встречаются документы, написанные совсем не о том, о чем следует, а вот документы, написанные

не совсем о том, о чем следует, не вполне так, как следует, встречаются довольно часто.

Некогда мы уже договорились, что документация пользователя закономерным образом распадается на отдельные документы. Поэтому в дальнейшем мы будем говорить о требованиях к содержанию не документации в целом, а отдельного документа.

Требования к содержанию документа подразделяются на следующие группы:

- требования объективной корректности;
- требования соответствия сведений, сообщаемых в документе, поставленным перед этим документом задачам;
- требования доступности сведений для читателя документа.

2.2. Объективная корректность

К сведениям, содержащимся в документе, предъявляются следующие требования объективной корректности:

- фактическая достоверность;
- соответствие сведений требуемому уровню знаний в области информационных технологий;
- соответствие сведений требуемому уровню знаний в предметной области.

Фактическая достоверность сведений, сообщаемых в документе, является наиболее фундаментальным требованием, предъявляемым к содержанию документа. При этом следует иметь в виду, что естественный язык как средство коммуникации весьма коварен и очень часто допускает не одну, а множество интерпретаций одного и того же высказывания.

Пример. Если в документе содержится утверждение: «Программа допускает импорт графических файлов формата TIFF», — а программа этого не допускает, то утверждение является недостоверным и может поставить пользователя в трудное положение. Если в документе содержится утверждение «Программа не допускает импорта графических файлов формата TIFF», а программа такой импорт допускает, то

утверждение является недостоверным, хотя, возможно, на практике доставит пользователю меньше хлопот. Однако если в документе содержится утверждение: «Программа допускает импорт графических файлов формата GIF и JPEG», — пользователь может сделать вывод о том, что программа не допускает импорта файлов формата TIFF. Если этот вывод неверен — можно ли назвать такое утверждение недостоверным?

Мы здесь понимаем под фактической достоверностью буквальное соответствие фактическому положению дел всех сведений, сообщаемых в документе. Однако необходимо помнить, что фактическая достоверность есть необходимое, но не достаточное условие формирования у пользователя адекватного представления о программном продукте.

Фактическая недостоверность сведений, сообщаемых о статичном, более не развивающемся программном продукте, связана обычно с тем, что технический писатель по невнимательности или недостаточному знанию допустил ошибку. Единственный путь избежать подобных ошибок или, вернее, вовремя исправить их — подвергнуть документ сквозному тестированию (проверке).

Однако если программный продукт развивается и разработчик постоянно выпускает новые версии продукта, возникает дополнительный фактор появления недостоверных сведений — отставание документации от текущего состояния продукта. Чтобы предотвратить такое отставание, необходимо вести параллельный учет всех изменений в программном продукте и в документации. Мы не можем здесь себе позволить даже краткий экскурс в методики ведения подобного учета. Существенно, что в рамках учета изменений соблюдаются следующие правила:

- при любых изменениях в программном продукте отслеживаются те изменения, которые требуют отражения в документации;
- выполнение задач по документированию тех или иных изменений в программном продукте отслеживается по стадиям вплоть до завершения.

При соблюдении этих правил ни одно изменение в программном продукте не будет изъято из общего процесса документи-

рования. В любой момент времени технический писатель будет владеть исчерпывающей информацией о том, какой версии продукта соответствует документация в ее текущем виде и какие изменения еще только предстоит отразить в документации. Это позволяет даже в том случае, если работа над документацией задерживается, предупредить читателей о том, что документация отражает более раннее состояние продукта, и тем самым не сообщать им заведомо недостоверных сведений.

К сожалению, не всегда и не во всех случаях разработчики корректно поддерживают даже версионность самого программного продукта, что негативно сказывается и на достоверности документации пользователя.

Соответствие сведений требуемому уровню знаний в предметной области и в области информационных технологий подчас не менее важна, чем фактическая достоверность сведений о самом документируемом программном продукте. Дело в том, что хотя документирование программного продукта предусматривает изложение сведений, в первую очередь относящихся к самому продукту, на деле техническому писателю приходится высказывать достаточно много соображений, выходящих за эти пределы. Чаще всего эти соображения относятся или к информационным технологиям (в целом либо в какой-то части, выходящей за рамки описания одного программного продукта), или к предметной области (к фигурирующим в ней сущностям, бизнес-процессам, действиям, лицам и т. п.). Кроме того, даже когда речь идет строго о документируемом программном продукте, многие высказывания о нем основаны на определенных знаниях из области информационных технологий или из предметной области.

Предполагается, что все эти соображения и высказывания соответствуют некоторому уровню знаний. С одной стороны, они должны быть доступны для понимания целевым пользователем, от которого, в свою очередь, требуется обладать определенным уровнем знаний. С другой стороны, они не должны выглядеть в глазах этого пользователя (носителя требуемого уровня знаний) нелепыми, неверными, необоснованными или несущественными. Объяснения, суждения, трактовки отдельных понятий не должны противоречить ни положениям и определениям, приня-

тым в профессиональной среде разработчиков программ и автоматизированных систем, ни опыту специалистов в той области, которую призван обслуживать документируемый продукт. В общих вопросах предметной области и эксплуатации вычислительной техники и автоматизированных систем автор документации должен выдавать как минимум тот уровень знаний, которого он требует от пользователя.

В практике документирования встречаются отклонения от гармоничного идеала в две противоположные стороны. В одних случаях автор документации попросту не вполне владеет освещаемой темой и поэтому допускает ошибки, неточности и несостоятельные суждения. В такой ситуации нет ничего исключительного: каждый технический писатель может столкнуться в своей работе с новой для него предметной областью или с незнакомыми понятиями из области информационных технологий.

Чтобы не выносить на публику свидетельства собственной некомпетентности, следует, во-первых, тесно взаимодействовать с консультантами в процессе работы над документацией, а во-вторых, прислушиваться ко всем замечаниям рецензентов (даже если эти замечания звучат расплывчато и обобщенно). В окончательном виде документация пользователя должна производить впечатление подготовленной профессионалами — как в предметной области, так и в области информационных технологий. В противном случае сообщаемые пользователю сведения попросту не будут вызывать у него доверия.

В других случаях автор документации вполне владеет освещаемой темой, но стремится упростить изложение, подстроить его под уровень пользователя — и невольно допускает неверные или некорректные суждения.

Следует учитывать, что всякое упрощение знаний требует их дополнительной проработки, уточнения основных понятий, четкого понимания, какой аспект и с какой целью мы упрощаем. Если какой-то аспект функционирования программного продукта или важный общий вопрос представлены в упрощенном виде, необходимо отметить сам факт упрощения и по возможности объяснить пользователю его причины.

2.3. Соответствие документа поставленным перед ним задачам

2.3.1. Требования, определяемые задачами документа

Выше нами были рассмотрены требования объективной корректности сведений, содержащихся в документе. Однако соблюдения этих требований недостаточно, чтобы документ соответствовал поставленным перед ним задачам.

Задачами, поставленными перед документом, определяются также следующие требования к его содержанию:

- актуальность;
- полнота;
- детальность.

2.3.2. Актуальность

Под *актуальностью* сведений мы понимаем их важность для пользователя, их необходимость для целевого применения программного продукта. Требование актуальности состоит в том, чтобы в документ включались те, и только те сведения, которые необходимы пользователю для работы с продуктом.

На первый взгляд, это требование настолько очевидно, что непонятно, следует ли о нем говорить. На самом деле достаточно часто в документ включаются сведения, актуальность которых для пользователя вызывает серьезные сомнения. В первую очередь к ним относятся:

- технические подробности функционирования программного продукта (за исключением минимальных сведений, необходимых пользователю для понимания сути происходящих процессов);
- сведения из предметной области (за исключением краткого введения, позволяющего уточнить терминологию и сориентировать пользователя в дальнейшем изложении);
- сведения из области информационных технологий, не относящиеся непосредственно к документируемому продукту;

но описывающие в целом те или иные классы программных средств, технологии, алгоритмы и т. д.;

- произвольные предположения относительно предпочтений пользователя, его желаний и намерений, а также о возможных нецелевых применениях документируемого продукта;
- суждения о достоинствах и недостатках документируемого продукта, а также о достоинствах и недостатках других технических решений.

Следует по возможности избегать включения в документ подобных сведений. В тех случаях, когда все же приходится включать в документ сведения общего характера, рекомендуется ограничиваться лишь теми знаниями, которые необходимы для корректного целевого применения программного продукта. Едва ли целесообразно превращать документацию пользователя в обзор различных программных средств сходного назначения, в учебник по предметной области или в рекламный буклет.

При этом не стоит забывать, что работа технического писателя тесно связана с работой других людей, занятых внедрением программного продукта: разработчиков, установщиков, специалистов по маркетингу и т. п. Иногда приходится идти навстречу их пожеланиям и включать в документ дополнительные сведения, в том числе и те, что перечислены выше. К этому надо относиться с пониманием, одновременно стремясь минимизировать количество лишних, ненужных пользователю сведений. В любом случае, технический писатель всегда должен понимать, кому и для чего он сообщает те или иные сведения.

В документации пользователя программного продукта, вообще говоря, неуместны положения, относящиеся к служебным обязанностям пользователя. Такого рода сведения включаются в документацию только в том случае, если программный продукт является органичной составной частью автоматизированной системы, обслуживающей определенную организацию. В этом случае пользователь не столько применяет его по своему усмотрению, сколько следует параграфам технологической инструкции, регламента либо должностной инструкции, которые потом инкорпорируются в состав документации пользователя (строго

говоря, это противоречит принципу разделения программной и системной документации, но иногда практикуется).

2.3.3. Полнота

2.3.3.1. Понятие полноты

Под *полнотой* сведений мы понимаем их достаточность для того, чтобы документ выполнил стоящую перед ним задачу. Полнота сведений складывается из *обширности* сведений, их *завершенности* и их *точности*.

2.3.3.2. Обширность сведений

Обширность сведений определяется темой и назначением документа. Если документ представляет собой руководство пользователя, в нем должны быть представлены все пользовательские задачи и способы их решения. Если справочник пользователя — все функциональные возможности и все элементы интерфейса с исчерпывающей информацией о них. В любом случае действует правило: любые сведения, подпадающие под определение, данное в заголовке документа, должны присутствовать в документе, если непосредственно в тексте документа не указано обратное.

Пример. В руководстве пользователя к текстовому редактору описаны все возможности этого программного продукта, однако по ряду причин не описан язык макрокоманд, используемый в этом редакторе. Это означает, что во Введении будет присутствовать оговорка:

В настоящем Руководстве не описан язык макрокоманд и не содержится рекомендаций по его применению. Макрокоманды применяются лишь в тех случаях, когда основных возможностей редактора не хватает. Язык макрокоманд и методика программирования макрокоманд описаны в Руководстве программиста.

Обширность сведений, включаемых в документ, косвенным образом зависит от общего состава комплекта документации и содержания других документов.

2.3.3.3. Завершенность сведений

Завершенность сведений определяется тем, насколько исчерпывающий характер имеют сведения по тому или иному вопросу.

Пример. В п. 2.2 приводится фраза:

Программа допускает импорт графических файлов формата GIF и JPEG.

Если программа также допускает импорт графических файлов формата TIFF, а в документе об этом не говорится, то можно сказать, что сведения о возможности импорта графических файлов в документе присутствуют, но неполны, а конкретно — не имеют исчерпывающего характера.

Завершенность сведений должна быть полной: если автор заявляет о своем намерении осветить тот или иной вопрос, который включает в себя ряд аспектов, ни один из этих аспектов не должен остаться неосвещенным, если непосредственно в тексте документа нет на этот счет специальной оговорки.

Требование завершенности сведений приобретает особую важность, коль скоро речь идет об описаниях последовательно выполняемых действий — процедурах. Иногда стремятся опустить действия, которые кажутся очевидными. Часто спрашивают: зачем писать в конце: «Нажмите на кнопку **ОК**», — ведь это и так очевидно.

Мы стремимся предостеречь от подобных рассуждений. Если заявлено описание процедуры, оно должно быть исчерпывающим и включать в себя все, в том числе очевидные, шаги. Во-первых, так проще следить за ходом изложения, во-вторых, никогда нельзя сказать заранее, какое «очевидное» замечание может показаться неочевидным, в-третьих, за «очевидными» действиями могут

стоять весьма и весьма неочевидные процессы, и рубеж, по пересечении которого эти процессы иницируются, не может не быть отражен в документации. То же еще в большей степени верно для тех случаев, когда шаг, за его очевидностью, пропускается где-то в середине процедуры.

2.3.3.4. Точность сведений

Точность сведений определяется следующими положениями:

- все конкретные объекты, элементы интерфейса, лица, программные средства и т.п. должны быть точно идентифицированы;
- все классы объектов, процедур, процессов должны быть охарактеризованы предельно четко и конкретно;
- все характеристики, количественные и качественные, должны быть приведены с максимально возможной конкретностью и точностью.

Конкретные объекты, элементы интерфейса, лица, программные средства и т.п. имеют уникальные идентификаторы — имена, идентификационные номера, графические значки. Характер идентификатора зависит от природы идентифицируемого объекта (элемента интерфейса, лица и т.п.): например, для лица это может быть фамилия с инициалами, для элемента интерфейса — значок, для устройства — условное кодовое наименование и т.п. Однако во всех случаях, если речь идет о чем-либо конкретном, единичном, обязательно следует указывать идентификатор.

Пример. Если какой-либо процесс запускается нажатием кнопки **Выполнить**, надлежит писать:

Нажмите на кнопку **Выполнить**.

Но не:

Нажмите на кнопку.

Нельзя рассчитывать на то, что пользователь сам догадается, какая кнопка имеется в виду. Каждый раз, когда упоминается конкретный объект (элемент интерфейса, лицо и т. п.), используется его идентификатор.

Здесь мы касаемся важной проблемы, о которой речь пойдет в п. 4.1.3. Если какие-либо объекты (элементы интерфейса, лица, программные средства и т. п.) многократно упоминаются в изложении, у технического писателя возникает желание заменить идентификатор местоимением. Например:

По завершении процесса кнопка **Выполнить** становится доступной. Нажмите на эту кнопку.

Может возникнуть вопрос: неужели и в этом случае нельзя рассчитывать на то, что пользователь сам догадается, какая «эта» кнопка имеется в виду?

Текст на естественном языке — сложный и деликатный организм. Он подчиняется особым законам, нарушение которых приводит к неприятным последствиям. Читатель знает: если какой-то предмет уже назван ранее, для его обозначения используют местоимения и разного рода указательные слова. Если мы без конца будем повторять идентификатор и всеми силами избегать местоимений, у читателя может возникнуть сомнение: это действительно тот же предмет или какой-то другой?

По завершении процесса кнопка **Выполнить** становится доступной. Нажмите на кнопку **Выполнить**.

Для неподготовленного читателя это звучит так, как будто есть две разные кнопки, по недоразумению получившие одно и то же название. Вот почему здесь не просто можно, а весьма и весьма желательно употребить местоимение, отсылающее к предыдущему упоминанию объекта.

Однако таких отсылок в техническом тексте должно быть как можно меньше. Их можно (а порой, повторяем, и нужно) употреблять только в том случае, если их смысл совершенно очевиден и быстро, без напряжения воспринимается пользователем. Если

предыдущее упоминание объекта встречается в том же или предыдущем предложении, тогда лучше использовать местоимение. Однако если есть минимальный риск, что читатель запутается или хотя бы затруднится с ходу понять, к чему это местоимение относится, лучше использовать идентификатор. Ни в коем случае не следует использовать слова и выражения, отсылающие не к упоминанию объекта в том же тексте, а к восприятию или осмыслению читателем реальности: «...нажмите на соответствующую кнопку»; «введите значение в предназначенном для этого поле» и т. п. Более подробно мы вернемся к этой теме в п. 4.1.3.

Иногда приходится сталкиваться с неудачно спроектированным интерфейсом, в котором некоторые элементы не имеют идентификаторов. В таких ситуациях приходится заменять идентификатор не слишком удобными указательными конструкциями: «кнопка в левом верхнем углу диалогового окна»; «флажок рядом с полем **Ширина**» и т. п.

Классы объектов, процедур, процессов и т. п., упоминающиеся в документе, должны быть охарактеризованы предельно точно и конкретно; их словесные характеристики не должны быть слишком широкими или слишком узкими и не должны быть неопределенными.

Пример. Если программа обрабатывает текстовые документы в формате RTF, не следует писать:

Программа обрабатывает текстовые документы.

Или:

Программа обрабатывает текстовые документы определенного формата.

Первая формулировка была бы слишком широкой, вторая — слишком неопределенной. Допустима только предельная точность и конкретность:

Программа обрабатывает текстовые документы в формате RTF.

Пример. Если программа допускает импорт графических файлов и растровых, и векторных форматов, не следует использовать, даже если по контексту это более уместно, более частную формулировку:

Программа допускает импорт растровых файлов.

Или неопределенную:

Программа допускает импорт графических файлов.

Следует писать:

Программа допускает импорт графических файлов любых, как графических, так и растровых форматов.

Характеристики, количественные и качественные, должны быть приведены с максимально возможной конкретностью и точностью.

Для количественных характеристик это означает необходимость переходить на язык конкретных числовых значений, если только это возможно.

Пример. Не следует писать:

Значение, вводимое в поле **Год**, имеет ограничение.

Но только:

Значение, вводимое в поле **Год**, ограничивается 2000 и более поздними номерами; при попытке ввести номер года 1999 или более ранний выдается сообщение об ошибке.

Для качественных характеристик это означает необходимость использовать из всех возможных словесных выражений наиболее конкретное, специализированное и передающее наиболее точный смысл.

Пример. Вместо: «более мощный компьютер» — используется более точное выражение, скажем: «компьютер, оснащенный процессором с более высокой производительностью».

2.3.3.5. Презумпция полноты сведений

Требование полноты сведений, представленных в документе, является очень важным. Документ может быть сколь угодно достоверным, но если пользователь не найдет в нем ожидаемых сведений, своей задачи документ не выполнит.

С другой стороны, полнота сведений, в отличие от достоверности, является относительной, поскольку оценка полноты сведений зависит от того, какие задачи решает данный конкретный документ в составе документации и достаточно ли содержащихся в нем сведений для решения этих задач.

Эти задачи декларируются в заглавии документа и в его вводных разделах, и именно из них исходит пользователь, обращаясь к этому документу. Важно помнить, что пользователь исходит из *презумпции полноты сведений*: он считает само собой разумеющимся, что в документе содержатся все сведения, определяемые тематикой и задачами этого документа. Если ему в явном виде не хватает каких-то сведений, он испытывает разочарование и раздражение. Но бывает и так, что пользователь считает сведения, представленные в документе, полными, а они такими не являются: не рассмотрена важная пользовательская задача, упомянуты не все форматы входных или выходных данных, указаны не все способы решения той или иной задачи и т. д. Такая «скрытая» неполнота может оказаться еще большим недостатком, чем открытая, поскольку пользователь не знает, что ему сообщили не все, и не пытается восполнить недостаток сведений. Самоочевидным примером «скрытой» неполноты являются случаи отсутствия в тех местах, где это требуется, предупреждений и предостережений. Еще раз повторяем: пользователь рассчитывает получить из документа все важные для себя сведения, а не какую-то их часть.

2.3.4. Детальность изложения

2.3.4.1. Понятие детальности изложения

Под *детальностью* изложения мы понимаем уровень подробности, достаточный для того, чтобы документ выполнял поставленные перед ним задачи.

Мы исходим из того, что документация пользователя адресована лицу, которое непосредственно применяет функциональные возможности программного продукта для решения своих задач. Поэтому документы, входящие в состав документации пользователя, обычно содержат описание конкретных действий пользователя в интерфейсе программного продукта, конкретной реакции системы на эти действия и конкретных результатов, которых пользователь добивается в результате этих действий. Исключение могут представлять лишь документы общего, обзорного характера, если таковые по тем или иным причинам включаются в документацию пользователя. Отсутствие в таких документах конкретных сведений в обязательном порядке должно восполняться их наличием в других документах комплекта, причем должно быть явным образом указано, в каких именно документах их следует искать. Например:

В настоящем документе содержится только общее описание методики взаимодействия носителей различных функциональных ролей с программным продуктом и процессов обработки и передачи данных. Конкретные действия носителей различных функциональных ролей описаны в руководствах по работе с программой для этих ролей.

Руководства и справочники пользователя, инструкции и другие подобные документы содержат детальные сведения об использовании тех или иных функций и решении тех или иных пользовательских задач. Это означает, что в документе в явном виде описываются следующие детали функционирования программного продукта:

- действия пользователя в интерфейсе программного продукта;
- внешние реакции системы и их интерпретация;

- изменения в данных, настройках программ и состоянии системы.

2.3.4.2. Действия пользователя в интерфейсе программного продукта

Действия пользователя в интерфейсе программного продукта описываются так, чтобы пользователь получил исчерпывающее представление о том, как именно выполнить эти действия. Обычно это описание включает в себя упоминания элементов интерфейса и глаголы, обозначающие собственно действия пользователя: «введите значение в поле...»; «щелчком мыши выберите в списке объект...»; «нажмите на кнопку...» и т. п. В практике документирования программных продуктов закрепились устойчивые слова и выражения, используемые для обозначения действий пользователя (мы называем такие устойчивые слова и выражения *фигурами описания*; см. п. 3.5.2).

Обычно пользователь, раскрывая очередной документ, уже готов к восприятию этих фигур описания, поскольку многократно встречался с ними ранее (поэтому, в частности, рекомендуется придерживаться в этом отношении устоявшейся практики, привычной пользователю).

Иногда заказчик настаивает: типичный пользователь программного продукта не имеет никакого опыта работы с компьютером и компьютерными программами, поэтому ему надо «подробнее» объяснить, как выполнять то или иное действие. В этом случае приходится давать пояснения к устоявшимся и кажущимся очевидными фигурам описания. Важно, чтобы уровень детализации описания позволял типичному пользователю программы понять, как именно выполняется то или иное действие.

Иногда, напротив, заказчик стремится сократить конкретные описания действий, мотивируя это тем, что типичный пользователь обладает большим опытом и в состоянии самостоятельно догадаться о том, как выполняются те или иные действия. Мы полагаем, что эти рассуждения не слишком убедительны. Если в программном продукте используется процедура или функциональная возможность, которую можно считать стандартной и

общеизвестной, можно сослаться на ее общеизвестность и не описывать. Однако такая ссылка должна присутствовать в явном виде. Если же процедура или функциональная возможность таковой не является, а предлагается лишь рассчитывать на догадливость искушенного пользователя, — такая практика, как нам кажется, не заслуживает поощрения. Пользователя нельзя заставлять о чем-либо догадываться, поскольку, во-первых, на размышления уходит время, а во-вторых, он может догадаться неправильно. Кроме того, может так случиться, что завтра понадобится снабдить одно из «очевидных» действий неочевидным примечанием; где мы разместим это примечание, если само действие у нас не описано? Например, многих как будто раздражает детальное описание работы с формой ввода данных: ведь все поля в форме подписаны, а порядок ввода хорошо известен любому опытному пользователю. Все это так, но у каждой формы ввода данных есть свои тонкости: в каком порядке можно вводить данные, какие ограничения (формальные и/или содержательные) накладываются на вводимые значения, какие именно изменения происходят в программе или в обрабатываемых данных после нажатия кнопки **ОК** и т. д. Есть, вероятно, совсем очевидные случаи, но если описывать форму в одном, неочевидном, случае и не описывать ее в другом, очевидном, — это нарушит универсальность подхода к пользовательскому интерфейсу, собьет пользователя с толку, лишит изложение естественного ритма.

Если же пользователю кажется, что его обременяют нудным рассказом о том, что и так очевидно, пусть он пропустит это место, действуя на свой страх и риск, а не потому, что мы за него приняли решение. Разумеется, это не означает, что технический писатель должен испытывать терпение своего читателя, постоянно повторяя то, что действительно очевидно (о логических повторах см. п. 3.5.4).

2.3.4.3. Внешние реакции системы и их интерпретация

Внешние реакции системы являются, по сути дела, «репликами» системы в ее диалоге с пользователем. В одних случаях пользователь совершает какие-либо действия — нажимает на

кнопку, выбирает пункт меню, щелкает мышью, — а внешняя реакция системы — появление диалогового окна, изменение отображаемого значения, перемещение пункта из списка в список — является ответом на эти действия. В других случаях, напротив, внешняя реакция системы — уведомление о поступившем письме, сообщение о возникшей проблеме, предупреждение об истечении определенного срока — наступает на какое-то независимое от пользователя событие. В этом последнем случае к внешним реакциям системы относятся:

- визуальные изменения на экране монитора;
- звуковые сигналы на встроенных или подключаемых динамиках компьютера;
- изменения в индикации или работе периферийных устройств или иной аппаратуры.

Визуальные изменения на экране монитора могут относиться к интерфейсу программного продукта или других программ. Рекомендуется учитывать, в первую очередь:

- исчезновение либо появление элемента интерфейса, нескольких элементов интерфейса, области интерфейса или целого окна;
- изменение доступности одного или нескольких элементов интерфейса (например, кнопка была недоступна, а теперь она доступна и может быть нажата пользователем);
- изменение состояния одного или нескольких элементов интерфейса (например, флажок был установлен, а теперь он автоматически снят);
- изменение внешнего вида области интерфейса: уменьшение или увеличение, изменение фона, появление в области тех или иных новых объектов или исчезновение тех объектов, которые отображались в ней ранее (например, очистка рабочей области графического редактора);
- изменение внешнего вида указателя мыши (например, со «стрелки» на «песочные часы» на время выполнения какого-либо процесса);
- появление сообщений в виде окон сообщений или в специально отведенной области рабочего окна (например, появ-

ление в статусной строке сообщения **Объект не найден** — в случае если поиск не дал результатов).

Звуковые сигналы на встроенных или подключаемых динамиках компьютера могут играть различную роль в диалоге системы и пользователя. В одних случаях они сводятся к сигналам в узком смысле, то есть к предупреждениям о смене режима, критическом значении параметра, важном событии и пр. В других случаях звуковой сигнал имеет более сложную структуру и несет более сложную информацию — допустим, в речевой форме. В третьих — звуковой сигнал важен сам по себе, является результатом, ценным для пользователя (программы звукозаписи, аудиомонтажа, синтеза речи).

Изменения в индикации или работе периферийных устройств или аппаратуры могут играть для пользователя вспомогательную роль, а могут представлять собой существенный результат в рамках решения пользовательской задачи. Например, активизация принтера или сканера обычно имеет прямое отношение к решаемой задаче, поскольку она в этом случае, скорее всего, связана с выводом данных на печать или со сканированием.

Внешние реакции системы обычно соответствуют определенным процессам в ней. Скажем, изменение внешнего вида указателя мыши свидетельствует о том, что запрошенное пользователем действие начало или закончило выполняться. Или: изменение значения параметра, отображаемого в информационном поле, свидетельствует о том, что параметр сменил значение. Поэтому надлежит, описав внешнюю реакцию системы, дать ее интерпретацию.

Иногда кажется, что интерпретация очевидна. Но и в этом случае ее лучше проговорить: возможно, догадок, которые возникают у пользователя, недостаточно.

Пример. По ходу решения пользовательской задачи на экране не появляется сообщение: **База данных не обнаружена. Поиск невозможен.** Интерпретация этого сообщения, казалось бы, очевидна. Тем не менее, пользователю, скорее всего, понадобятся разъяснения: что это значит — не обнаружена? как следует понимать выданное сообщение? Поэтому следует не только описать, что происходит на экране, но и объяснить, как следует понимать происходящее:

Иногда в ходе выполнения команды на экране появляется сообщение: **База данных не обнаружена. Поиск невозможен.** Это означает, что система не в состоянии обратиться к базе данных, — обычно из-за отсутствия подключения к локальной сети или из-за временной недоступности сервера баз данных.

Весьма часто подобное отражение внешних реакций системы в документации пользователя вызывает недоумение: зачем дублировать то, что и так появится на экране? Тому есть несколько причин:

- при использовании документа в процессе работы с программой это упрощает отслеживание разных этапов работы, при чтении документа в отрыве от компьютера — позволяет понимать логику работы программы;
- в случае ошибочных действий пользователя: неверных шагах, перепутанных кнопок, вводе некорректных данных и т. п. — это позволяет сразу же заметить признаки ошибки и скорректировать свои действия;
- в случае сбоя программы или неисправности компьютера это позволяет сразу же заметить признаки неисправности и принять меры.

Коротко говоря, отражение в документации пользователя детальных описаний внешних реакций системы позволяет в каждый момент времени судить о том, насколько штатным образом развивается диалог пользователя и системы.

2.3.4.4. Изменения в данных, настройках программ и состоянии системы

Нередко изменения в данных, настройках программ и состоянии системы не слишком хорошо отражаются в документации — по той простой причине, что они скрыты от глаз и других органов чувств, и технический писатель, увлеченный описанием действий пользователя и внешних реакций системы, подчас пренебрегает ими. Между тем, тому же пользователю — читателю документации — совершенно необходимо знать об этих изме-

нениях, в противном случае ему придется лишь догадываться о них, самостоятельно экспериментировав с программой.

Отражая в документации разного рода скрытые изменения, следует по возможности сообщить, в результате чего они происходят: действий пользователя, работы документируемого программного продукта, работы других программ или иных компонентов системы.

К изменениям в данных в первую очередь относятся:

- передача данных из одного модуля в другой, из одного места хранения в другое;
- сохранение данных в том или ином месте (в файле на диске, в разделе базы данных и т. п.);
- удаление данных, хранящихся в том или ином месте;
- создание копии данных и передача этой копии на хранение в то или иное место;
- обновление данных в соответствии с действиями пользователя;
- изменение значений данных: численных, текстовых, графических.

К изменениям в настройках программ в первую очередь относятся:

- задание значений для параметров работы программ (с указанием того, в пределах какого времени будет действовать данное значение параметра: в продолжение текущего сеанса работы, вплоть до отмены и т. д.);
- изменения состояния программы или смена ее рабочих режимов.

К изменениям в состоянии системы в первую очередь относятся:

- начало и завершение работы компонентов системы;
- начало и завершение процессов обработки данных;
- вступление компонентов системы во взаимодействие: подключение, отключение, вызов одними компонентами других.

Изменения в данных, настройках, состоянии системы тесно связаны с действиями пользователя и внешними реакциями системы. Внешние реакции системы обычно свидетельствуют о визуально не наблюдаемых, но важных для пользователя изменениях. Действия же пользователя направлены либо на то, чтобы инициировать — в более или менее отдаленной перспективе —

подобные изменения, либо на то, чтобы прореагировать на них тем или иным образом.

В документации обязательно должна быть отражена логика этих взаимосвязей.

Пример. В документе описано применение одной из функций программы:

Нажмите на кнопку **Расчет**. Указатель мыши изменит свой вид со «стрелки» на «песочные часы». Это означает, что в системе начался процесс обработки данных. По завершении процесса новые значения данных будут сохранены в базе данных программы. О завершении процесса можно судить по изменению вида указателя мыши с «песочных часов» опять на «стрелку».

Здесь первая фраза описывает действия пользователя, вторая — внешнюю реакцию системы, третья содержит интерпретацию этой реакции, и эта интерпретация отсылает пользователя к скрытому, но важному для него процессу. Четвертая фраза описывает завершение этого процесса, пятая же предупреждает о том, какая внешняя реакция системы будет свидетельствовать о завершении.

Более подробно и более предметно об этом рассказывается в разделе, посвященном изложению процедурной и функциональной информации (п. 3.4.3).

2.3.4.5. Условная самодостаточность текста

Детальность изложения лучше всего проверяется с помощью теста на *условную самодостаточность текста*. Это означает, что если в документе содержатся сведения о той или иной функциональной возможности, процедуре, о том или ином процессе обработки данных и т. п., читатель должен быть в состоянии, не обращаясь к программному продукту, только из текста понять, как выполняются действия пользователя, каковы внешние реакции системы, какие происходят изменения в данных, настройках про-

грамм и состоянии системы. Если у читателя, типичного пользователя, возникает потребность запустить программный продукт и свериться с ним, детальность изложения недостаточна.

2.4. Доступность

Требование *доступности* сведений, сообщаемых в документе, означает, что содержание документа должно быть понятно типичному пользователю.

С требованием доступности документа для восприятия не следует путать требования, предъявляемые к структуре документа (простота обозрения, логичность), требования к стилю (не слишком запутанный синтаксис, четкость и однозначность выражений). Все эти требования относятся к форме, в которой сообщаются сведения, но не к самим сведениям. Форме выражения, ее оптимизации посвящена значительная часть нашей книги. Однако здесь речь идет о доступности самих сведений, объективной способности типичного пользователя усвоить эти сведения.

Наиболее проблемными с этой точки зрения являются сведения из области информационных технологий, имеющие чересчур специальный характер. Типичный пользователь прикладного программного продукта не всегда обладает специальными познаниями в части компьютерной техники и программного обеспечения; обычно он знаком на пользовательском уровне с интерфейсом и основными рабочими объектами операционной системы. Ему могут быть вовсе незнакомы даже основные понятия теории баз данных, он часто имеет довольно-таки смутное представление о символических языках, даже о языках разметки. У него в большинстве случаев нет своего опыта программирования, а возможно, и самостоятельной настройки операционной системы или каких-либо программ.

Если сведения не соответствуют требованию доступности для пользователя, прежде всего, следует задать вопрос: а соответствуют ли они требованию актуальности? Если нет, их смело можно, и даже нужно исключить из документации пользователя. Если же сведения актуальны для пользователя, но есть осно-

вания опасаться, что он не готов к их восприятию, необходимо подготовить его, так сказать, на ходу. Для этого в отдельной главе или даже в отдельном документе дается краткий очерк необходимых понятий. Следует помнить, что документация пользователя не место для полноценного курса по той или иной важной теме, будь то системы управления базами данных или программирование на алгоритмических языках. Здесь освещаются только наиболее важные для освоения программного продукта вопросы. Трудные для понимания концепции допускается излагать в сокращенном и упрощенном виде (об этом необходимо предупредить пользователя, отослав его за более полными сведениями к специальной литературе).

Если есть сомнения по поводу доступности тех или иных сведений, лучше всего, излагая их, предупредить пользователя о возможных затруднениях. Например:

Сведения, изложенные в настоящем разделе, адресованы в первую очередь пользователям, знакомым с основами программирования. Основная часть функций программы может использоваться без обращения к материалу настоящего раздела.

Разумеется, следует постараться разместить эти сведения отдельно от общедоступных — в особом разделе или разделах, а возможно, и в особом документе. Иногда такое обособление делается даже вопреки логике изложения; в таких случаях необходимо предупредить читателя, что если он намерен ознакомиться с темой в полном объеме, то за разъяснениями по некоторым частным вопросам ему придется обратиться к другому разделу или документу.

Не следует, однако, впадать в крайность и видеть в каждом не вполне тривиальном рассуждении угрозу для пользователя. Надо иметь в виду, что целевое применение прикладного программного продукта — деятельность, требующая некоторой подготовки, и далеко не всегда эта подготовка может быть возложена на автора документации пользователя. Следует предполагать в своих читателях некоторый начальный уровень знаний, в том числе и в области информационных технологий. Если есть опа-

сения, что у кого-то этот уровень недостаточно высок, рекомендуем обсудить эту проблему с заказчиком. Возможно, ему откроется новое видение тех сложностей, которые ожидают его на пути внедрения программного продукта. Это не вызовет у него радости, но заставит принимать технического писателя всерьез.

Глава 3

Структура документации пользователя

3.1. О структуре документации пользователя

3.1.1. Структурированность — сквозное свойство документации

Под *структурой* документации часто подразумевают явное деление ее на отдельные документы, а документов на разделы, подразделы, пункты, подпункты и пр. Однако на самом деле *структурированность* — сквозное свойство документации пользователя (да и вообще технического текста). Она выражается не только в делении документов на более мелкие части, каждая из которых имеет заголовок, но и в рациональном размещении разных видов полезной для пользователя информации, в представлении тех или иных сведений в удобной для восприятия форме (маркированного или нумерованного списка, таблицы), в обособлении сведений, на которые пользователю рекомендуется обратить внимание. Структурированность документации выражается также и в употреблении похожим образом построенных словосочетаний, фраз, фрагментов текста для описания сходных явлений — эта повторяемость значительно облегчает восприятие текста.

Причина, по которой мы предпочитаем структурированное изложение материала неструктурированному, кроется в особенностях человеческого восприятия. Вступая во взаимодействие с тем или иным техническим средством, например с компьютером, на котором установлен, в числе прочего, документируемый программный продукт, человек сталкивается с чужим, весьма сложным миром. В этом мире действуют особые законы, которые необходимо как можно скорее понять, усвоить и затем держать в памяти, чтобы эффективно эксплуатировать техническое средство. Человеческое же сознание устроено таким образом, что информация лучше всего воспринимается в том случае, когда в обширном массиве вычленяются какие-то более мелкие фрагменты, причем вычленение этих фрагментов подчинено более или менее понятным закономерностям. Это может быть деление текста на тематические разделы, выделение повторяющихся приемов описания сходных явлений, определенная методика подачи информации и т. п.

3.1.2. Содержательная и формальная структура

Будем различать структуру *содержательную* и *формальную*. Содержательная структура присуща любому документу независимо от того, обозначена ли она какими-либо явными признаками — *структурными маркерами*. Например, если пространственный текст посвящен достаточно крупной теме, то в нем, даже если отсутствует явная рубрикация, вычленимы структурные элементы, соответствующие различным аспектам общей темы. Если эти структурные элементы вычленяются легко и естественно, мы говорим, что текст хорошо структурирован, даже если в нем нет явных структурных маркеров и он предстает нашим глазам как сплошной.

Формальная структура определяется автором документа и имеет конкретное внешнее выражение: деление документа на разделы, специальное форматирование отдельных фрагментов текста и т. п. Формальная структура вносится в документ с по-

мощью структурных маркеров — заголовков, отступов, значков и т. п.

Документация пользователя должна быть хорошо структурирована как в содержательном, так и в формальном отношении. Закономерно возникает вопрос о том, что первично, а что вторично. Со школьной скамьи нас учат сначала составить план, а потом уже писать «сочинение». На первое место в таком случае выступает формальная структура, а содержательная ей подчиняется. С другой стороны, многие (и не без основания) полагают, что главное — это начать работу над текстом; при ясном и четком понимании материала изложение автоматически получит желаемую структурированность, а уже поверх содержательной структуры накладывается формальная.

Нам кажется, что в этом споре обе стороны до некоторой степени правы. На самом деле нельзя ни чересчур жестко следовать заранее составленному плану, ни пускаться в путь без плана, полагая, что материал сам подскажет, как его разбить на структурные элементы. Следует начинать работу с составления плана, а в дальнейшем — следовать ему в целом, при необходимости внося коррективы, порой весьма существенные.

3.1.3. Первопроходец или экскурсовод?

Дело в том, что среди технических писателей обычно преобладает методика, которую мы назовем *методикой первопроходца*. Технический писатель проходит там, где до него, фигурально выражаясь, не ступала нога пользователя, пути же, которыми он идет, потом будет повторять каждый пользователь. Исходя из этого пишется и документация пользователя.

Мы являемся сторонниками *методики экскурсовода*. Сначала технический писатель изучает программный продукт и в том или ином виде составляет базу знаний о продукте. Затем он формулирует для себя основные задачи, которые будет решать документация пользователя, — выделяет типы пользовательской активности, которые необходимо поддерживать, сортирует сведения, как более или менее важные для пользователя, определяет

уровень подготовки пользователя и особенности изложения. И уже на основе этой работы планирует структуру, содержательную и формальную, тех документов, которые впоследствии будут предоставлены пользователю.

Методика экскурсовода несколько более трудоемка, нежели методика первопроходца. Однако нам кажется очень важным, чтобы технический писатель проводил границу между своими знаниями о программном продукте, с одной стороны, и представлением необходимых пользователю знаний в документации, с другой. Путешественника не должно беспокоить, на какие проблемы наткнулся первопроходец; он нуждается в гиде, который удобными путями проведет его по интересным местам и предостережет от неприятностей.

Методика экскурсовода предполагает совершенно определенный подход к структуре документации пользователя: содержательная и формальная структура документации разрабатываются в расчете на восприятие пользователя — с тем чтобы каждый документ выполнил свою задачу с минимальными для пользователя трудозатратами и проблемами. Разумеется, когда технический писатель собирает сведения о программном продукте, он также структурирует их — в неструктурированном виде представить себе знания о программном продукте, да и о любом сколько-нибудь сложном техническом решении, невозможно. Однако структура комплекта документов, предоставляемого пользователю, и каждого в отдельности документа разрабатывается отдельно, причем так, чтобы служить основной задаче документации пользователя — обеспечивать целевое применение программного продукта пользователем.

3.1.4. Вводные главы: торжество формальной структуры

Работа над структурой документации начинается уже на стадии определения состава комплекта — ведь комплекс знаний о программном продукте един и делится на части по-разному, в зависимости от характера программного продукта и особенностей его использования (см. п. 1.3.4). Затем разрабатывается структу-

ра каждого в отдельности документа. Что же при этом первично — формальная структура или структура содержательная?

Изначально существуют некоторые черты формальной структуры, которые предшествуют любой работе над документом. Так, в начале каждого документа помещается несколько вводных глав, в которых кратко характеризуется продукт, указывается назначение документа, перечисляются, если это необходимо, требования к квалификации пользователя, приводятся условные обозначения и сокращения и т. п. Обычно состав этих глав один и тот же для документов определенного типа, независимо от документируемого программного продукта. Получается, что формальная структура отчасти предшествует содержательной структуре, а содержательная структура отчасти подчиняется заданной с самого начала формальной структуре. Благодаря этому пользователь начинает знакомство с документом со «стандартных» заголовков разделов и только после этого переходит к материалу, специфическому для конкретного программного продукта. Такой порядок изложения облегчает восприятие материала.

3.1.5. Основная часть документа: способы изложения материала

Та часть документа, которая следует за вводными главами и в которой описывается собственно целевое применение программного продукта, структурируется в соответствии с содержательными критериями. Таким образом, здесь уже содержательная структура не подчиняется формальной, а напротив, определяет ее. Однако это не означает, что здесь не используются заранее заданные формальные клише. Эти клише уже не касаются «стандартных» глав или разделов, однако они имеют отношение к порядку изложения важных для пользователя сведений. Последовательность глав и разделов не может быть задана заранее, поскольку их тематика уже зависит от конкретного материала, однако главы и разделы следуют друг за другом в соответствии с одним из способов изложения материала.

Мы выделяем следующие способы изложения материала:

- с точки зрения пользовательских задач;
- с точки зрения функций программного продукта.

Первый способ изложения материала предполагает, что вначале формулируются задачи, которые стоят перед пользователем и которые, как предполагается, он будет решать с помощью программного продукта. Затем для каждой задачи приводится последовательность действий, с помощью которых эта задача решается.

Второй способ изложения материала предполагает, что вначале перечисляются функции программного продукта (и элементы интерфейса, позволяющие эти функции применять). Затем для каждой функции приводится назначение, а также последовательность действий, с помощью которых эта функция применяется в пользовательской практике.

Эти два способа изложения материала соответствуют двум различным видам формальной структуры изложения. Если изложение ведется с точки зрения пользовательских задач, оно естественным образом разбивается на структурные элементы, каждый из которых соответствует пользовательской задаче (последовательность этих структурных элементов желательно представить сжатым текстом обзорного характера). Если изложение ведется с точки зрения программного продукта, оно разбивается на структурные элементы, соответствующие функциям продукта или группам функций (например, разделы соответствуют различным меню главного окна программы, подразделы — отдельным пунктам).

Иными словами, заголовки и содержание разделов, подразделов, пунктов и подпунктов определяются конкретным предметом изложения. В этом смысле формальная структура определяется содержательной. Однако сам принцип структурирования задается не конкретным содержанием, а выбранным способом изложения материала. Соотношение формальной структуры и ее содержательного наполнения можно продемонстрировать следующим образом.

Пример. Изложение материала с точки зрения пользовательских задач:

Работа с сообщениями электронной почты

Сообщения электронной почты и виды работы с ними — Обзор пользовательских задач

Создание нового электронного письма и его отправка
Просмотр поступивших электронных писем
Ответ на поступившее электронное письмо

Пользовательские задачи

Пример. Изложение материала с точки зрения функций программного продукта

Функции обработки изображения

Обработка изображения и функции программы — Обзор функций

Просмотр изображения
Редактирование изображения
Управление параметрами изображения

Функции

Чем же определяется способ изложения материала? Иногда в документе последовательно или почти последовательно используется один и тот же способ изложения материала, иногда же способы изложения материала чередуются.

Выбор делается на основании целого ряда соображений:

- Разные документы, входящие в комплект документации пользователя, могут соответствовать разным способам изложения материала. Так, руководство пользователя содержит в основном изложение материала с точки зрения пользовательских задач, а справочник пользователя — с точки зрения функций программного продукта.
- Разные пользовательские перспективы (см. п. 1.3.4.3) предполагают предпочтительное использование при документировании разных способов изложения материала. Так, при документировании программы-инструмента или программы потоковой обработки предпочтительнее изложение материала с точки зрения пользовательских задач,

- при документировании программы-среды или программы реального времени — с точки зрения функций программного продукта.
- Разные темы в пределах одного документа предполагают предпочтительное использование разных способов изложения материала. Так, при описании порядка выполнения какого-либо важного для пользователя действия целесообразно строить изложение с точки зрения пользовательских задач: сначала формулируется задача, а затем по шагам описываются действия пользователя, направленные на выполнение этой задачи. Напротив, при описании настройки программного продукта или задания параметров какого-либо процесса предпочтительнее изложение с точки зрения функций программного продукта: описывается интерфейс настройки (часто таким интерфейсом является диалоговое окно, иногда содержащее несколько вкладок), а уже отталкиваясь от интерфейса настройки, автор документа разъясняет смысл каждого параметра и дает рекомендации по заданию значений параметров.

Эти соображения могут учитываться порознь или вместе, что дает в конкретных ситуациях различные варианты комбинаций способов изложения материала. Единственное, чего не следует допускать ни в коем случае, — это беспорядочное смешение способов изложения материала. Пользователь в любой момент должен понимать, с какой точки зрения ведется изложение материала; выбор же способа изложения материала всегда определяется тем, как удобнее воспринимать информацию пользователю.

Как правило, в рамках одного документа один из способов изложения материала является *основным*, другой же — *дополнительным*, используемым в тех случаях, когда первый явно неэффективен. Например, изложение ведется в целом с точки зрения пользовательских задач (основной способ изложения материала), однако настройка описывается с точки зрения функций программного продукта (дополнительный способ изложения материала).

Способ изложения материала проявляется не только на уровне формальной структуры документа, но и на других уровнях

изложения: он проявляется в формах подачи информации, построении фразы и т. п. Так, если выбран способ изложения с точки зрения пользовательских задач, он проявляется, во-первых, на уровне формальной структуры: документ разбит на разделы, соответствующие пользовательским задачам или тематическим группам пользовательских задач, разделы — на подразделы, соответствующие этапам пользовательских задач или целым задачам и т. д. Во-вторых, способ изложения проявляется на уровне отдельного структурного элемента, не делимого далее на структурные элементы: изложение начинается с постановки пользовательской задачи или выделения этапа в ее решении, а затем описываются конкретные пошаговые действия пользователя, направленные на решение задачи или реализацию этапа. В-третьих, отдельные абзацы и даже фразы тяготеют к той же последовательности изложения: сначала постановка задачи, затем ее решение. Например:

Чтобы перейти к следующему этапу решения задачи, нажмите на кнопку **ОК**.

Напротив, если выбран способ изложения с точки зрения функций программного продукта, будут иными и характер формальной структуры (разделы будут соответствовать группам близких по назначению функций программы, подразделы — отдельным функциям и т. д.), и принцип построения отдельного структурного элемента, и структура отдельных фраз.

Пример. При изложении с точки зрения пользовательских задач более уместна такая структура фразы:

Чтобы отредактировать документ, в меню **Правка** выберите ту или иную функцию редактирования документа.

При изложении с точки зрения функций программного продукта — такая:

В меню **Правка** доступны функции редактирования документа.

3.2. Деление документа на основные структурные элементы

3.2.1. Основные структурные элементы: тематический принцип

Документ делится на разделы, разделы — на подразделы, подразделы — на пункты, пункты — на подпункты. Таким образом, в документе нами выделяется до четырех уровней структурных элементов, каждый из которых снабжается заголовком. Эти структурные элементы мы называем *основными*, в отличие от *дополнительных* структурных элементов: нумерованных и маркированных списков, процедур, таблиц, иллюстраций, фрагментов выделенного формата («врезок») и т. п.

Мы уже говорили о том, что текст документации пользователя всегда структурирован, причем на многих уровнях. Однако это было общее замечание. Здесь мы делаем более конкретное утверждение: документ делится на структурные элементы, каждый из которых имеет заголовок. И сразу же возникает вопрос: для чего, с какой целью?

На первый взгляд все очевидно: внутри большой темы выделяются темы поменьше, каждая такая тема и становится содержанием отдельного раздела. Внутри раздела по тому же принципу выделяются подразделы и т. д. Это абсолютно верно: в основе деления документа на основные структурные элементы лежит *тематический принцип*.

Это отнюдь не такое тривиальное замечание, как может показаться на первый взгляд. Для сравнения скажем, что дополнительные структурные элементы часто выделяются не по тематическому, а, например, по *конструктивному* принципу, т. е. просто потому, что те или иные сведения удобнее представить в виде списка или таблицы (об этом подробнее см. п. 3.3.2). Основные же структурные элементы всегда выделяются по тематическому принципу. Заголовок структурного элемента отражает тему последнего.

Тематический принцип состоит не только в том, чтобы каждый структурный элемент соответствовал отдельной теме. Он включает в себя следующие положения:

- Каждый структурный элемент содержит сведения по какой-либо теме, более общей или более частной, в зависимости от его уровня.
- Если структурный элемент входит в структурный элемент предыдущего (более высокого) уровня, это всегда означает, что тема первого представляет собой какой-либо аспект темы второго.
- Уровень структурного элемента хотя бы в первом приближении соотносится с уровнем важности темы этого элемента. В частности, если структурный элемент имеет высший уровень иерархии (такой структурный элемент называется разделом), он включает в себя сведения по важной самостоятельной теме, которую целесообразно выносить в заголовок первого уровня¹³.

Работая над различными текстами на протяжении всей своей жизни, мы привыкаем сначала составлять «план», а потом заполнять его пункты текстом. Формальная структура, если на нее взглянуть со стороны, напоминает развернутый план документа. И в самом деле, работа над документом должна начинаться с составления плана, который впоследствии станет основой для формальной структуры документа. Приступая к работе над документом, технический писатель берет за основу типовую структуру и затем «доставляет» ее — добавляет структурные элементы, содержащие специфические для документируемого программного продукта сведения, располагает их тем или иным образом, продумывает порядок изложения и т. п. В результате у него получается эскиз формальной структуры документа, но ни в коем случае не окончательная формальная структура. Вплоть до завершения работы над текстом документа технический писатель вносит в формальную структуру коррективы, и порой очень значительные.

¹³ В документах очень большого объема иногда используются структурные элементы еще более высокого уровня, чем разделы, — части и тома.

3.2.2. Глубина и сечение формальной структуры документа

Итак, каждый раздел может быть поделен на подразделы, подраздел — на пункты, пункт — на подпункты. *Глубиной формальной структуры* называется число ее уровней от самого крупного до самого мелкого структурного элемента. На разных участках формальной структуры ее глубина различна, однако мы рекомендуем соблюдать следующее ограничение: глубина по возможности нигде не должна превосходить 4.

Если структурный элемент делится на структурные элементы следующего уровня, последние образуют формально однородную последовательность (формально — потому что в содержательном плане, как будет показано в п. 3.2.3, между этими структурными элементами могут быть различные отношения). *Сечением* формальной структуры называется число структурных элементов в структурном элементе предшествующего уровня. На разных участках формальной структуры ее сечение различно, однако мы рекомендуем соблюдать следующее ограничение: сечение по возможности нигде не должно превосходить 9. Иначе говоря, в разделе не более 9 подразделов, в подразделе не более 9 пунктов, в пункте не более 9 подпунктов¹⁴.

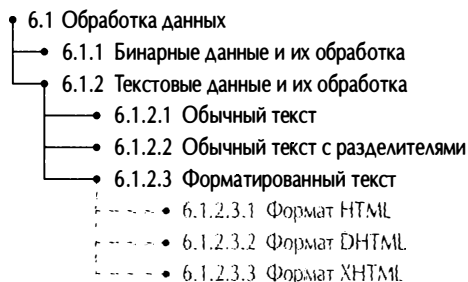
Ограничения, накладываемые на глубину и сечение формальной структуры документа, не произвольны, но предлагаются нами как результат компромисса между стремлением сделать документ как можно более простым для восприятия, с одной стороны, и необходимостью отразить в документе объективную сложность документируемого программного продукта.

Если глубина формальной структуры документа выходит за пределы предлагаемого нами ограничения, или проще говоря, возникает необходимость разбить подпункт на подподпункты, следует попытаться перестроить формальную структуру таким образом, чтобы избежать этого. В первую очередь необходимо задаться вопросом: если глубина формальной структуры на этом участке так

¹⁴ Число следующих подряд разделов документа формально не ограничивается. Однако существует важное содержательное ограничение, о котором мы говорили выше: раздел должен включать в себя сведения по важной самостоятельной теме, которую целесообразно выносить в заголовок первого уровня.

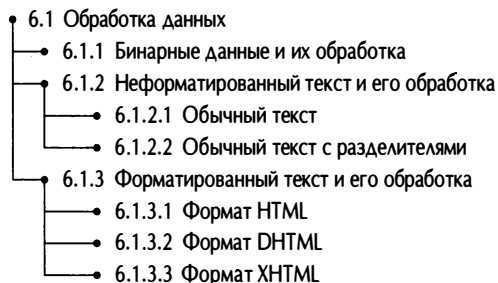
велика, то, возможно, мы недооценили значимость какого-либо структурного элемента и следует повысить уровень его иерархии. Для этого либо выносят подпункт в самостоятельный пункт, либо содержащий его пункт — в самостоятельный подраздел, либо содержащий его подраздел — в самостоятельный раздел.

Пример. Имеется фрагмент структуры (бледным шрифтом справа от пунктирной черты показаны подподпункты, которые пришлось бы выделить в пределах подпункта):



Ниже даны варианты исправленной структуры: в первом варианте подпункт повышается до пункта, во втором — пункт до подраздела, в третьем — подраздел до раздела. Автору документа предстоит выбрать, какой вариант в большей степени соответствует потребностям пользователя при чтении подряд и при поиске нужных сведений по случаю. Результат выбора зависит от того, какие темы лучше объединить под одним заголовком, а какие — рассмотреть по отдельности.

Вариант 1



Вариант 2

- 6.1 Обработка бинарных данных
- 6.2 Обработка текстовых данных
 - 6.2.1 Обычный текст
 - 6.2.2 Обычный текст с разделителями
 - 6.2.3 Форматированный текст
 - 6.2.3.1 Формат HTML
 - 6.2.3.2 Формат DHTML
 - 6.2.3.3 Формат XHTML

Вариант 3

- 6 Обработка данных
 - 6.1 Бинарные данные и их обработка
 - 6.2 Текстовые данные и их обработка
 - 6.2.1 Обычный текст
 - 6.2.2 Обычный текст с разделителями
 - 6.2.3 Форматированный текст
 - 6.2.3.1 Формат HTML
 - 6.2.3.2 Формат DHTML
 - 6.2.3.3 Формат XHTML

В данном случае мы остановили бы свой выбор на третьем варианте. Он сохраняет исходную структуру и просто сдвигает ее на один уровень вверх. Это допустимо, и даже желательно, поскольку заголовок «Обработка данных» звучит вполне внушительно, чтобы вынести его на верхний уровень иерархии. Если бы это было невозможно (например, тема была бы слишком частной), следовало бы выбирать из оставшихся двух вариантов.

Если сечение формальной структуры документа выходит за пределы предлагаемого нами ограничения, или проще говоря, возникает потребность создать в рамках одного структурного элемента 10 и более структурных элементов следующего уровня, лучше попытаться перестроить формальную структуру таким образом, чтобы избежать этого. В первую очередь необходимо задаться вопросом: если этот структурный элемент допускает членение на такое большое количество тематически обособлен-

ных частей, то, возможно, вместо него следует создать два или более структурных элемента того же уровня. Если это оказывается нецелесообразным, можно подумать над тем, чтобы углубить структуру — объединить два или несколько структурных элементов следующего уровня под единым заголовком. В этом случае сечение исходного структурного элемента уменьшится, а упомянутые два или несколько структурных элементов следующего уровня спустятся еще на один уровень вниз.

Пример. Имеется фрагмент формальной структуры:

- 8.5 Редактирование параметров пользователя
 - 8.5.1 Фамилия, имя, отчество
 - 8.5.2 Фотография
 - 8.5.3 Дата рождения
 - 8.5.4 Адрес
 - 8.5.5 Телефон
 - 8.5.6 Email
 - 8.5.7 Интересы
 - 8.5.8 Тип аккаунта
 - 8.5.9 Идентификационные параметры
 - 8.5.10 Используемый веббраузер
 - 8.5.11 Сервер

Количество пунктов в подразделе слишком велико, поэтому возможны два варианта улучшенной структуры. В первом варианте подраздел разбит на два независимых подраздела, во втором — углублена формальная структура. Автору документа предстоит выбрать, какой вариант в большей степени соответствует потребностям пользователя.

Следует отметить, что в интерфейсе программного продукта не было таких понятий, как личные данные пользователя и пользовательские настройки. Их ввел технический писатель для того, чтобы лучше структурировать изложение. Этот прием применяется довольно часто, однако им не стоит злоупотреблять, чтобы не создать слишком много новых понятий, никак не присутствующих в самом продукте.

Вариант 1

- 8.5 Личные данные пользователя и их редактирование
 - 8.5.1 Фамилия, имя, отчество
 - 8.5.2 Фотография
 - 8.5.3 Дата рождения
 - 8.5.4 Адрес
 - 8.5.5 Телефон
 - 8.5.6 Email
- 8.6 Пользовательские настройки и их редактирование
 - 8.6.1 Интересы
 - 8.6.2 Тип аккаунта
 - 8.6.3 Идентификационные параметры
 - 8.6.4 Используемый веббраузер
 - 8.6.5 Сервер

Вариант 2

- 8.5 Редактирование параметров пользователя
 - 8.5.1 Личные данные пользователя
 - 8.5.1.1 Фамилия, имя, отчество
 - 8.5.1.2 Фотография
 - 8.5.1.3 Дата рождения
 - 8.5.1.4 Адрес
 - 8.5.1.5 Телефон
 - 8.5.1.6 Email
 - 8.5.2 Интересы
 - 8.5.3 Тип аккаунта
 - 8.5.4 Идентификационные параметры
 - 8.5.5 Используемый веббраузер
 - 8.5.6 Сервер

Легко видеть, что глубина формальной структуры и ее сечение связаны обратной связью: стремясь уменьшить глубину, автор документа рискует растянуть сечение, и наоборот, сокращая сечение, он часто невольно увеличивает глубину. Это означает, что идеальная формальная структура получается в результате отыскания правильного баланса между глубиной и сечением на каждом участке.

Описанные нами приемы достаточно просты и, по сути дела, очевидны; мы лишь систематизировали их. Может, однако, возникнуть вопрос: а стоит ли заниматься оптимизацией формальной структуры по глубине и сечению? Велика ли беда, если глубина ее будет более 4 или сечение — больше 9?

Во-первых, разумеется, ничего страшного от отдельно взятого нарушения правила не произойдет. Но если не следить за соблюдением правила вообще, формальная структура будет соответствовать ему все меньше, а это неудобно для пользователя.

Во-вторых, как уже было отмечено, нарушение приводимых здесь ограничений не только создает неудобства пользователю, но и заставляет задуматься: а правильно ли организована формальная структура в целом? верно ли выбран способ изложения материала? оптимальным ли образом структурирован материал внутри основных структурных элементов? Поэтому, столкнувшись с феноменом разрастания формальной структуры вглубь или по сечению, рекомендуется в первую очередь попытаться перестроить ее, используя описанные выше приемы. Если же это не получается, следует — это слишком общий совет, но более конкретные рекомендации, к сожалению, дать трудно, — задуматься над тем, правильно ли построено изложение в целом.

Если не удастся справиться с разрастанием формальной структуры путем ее перестройки, стоит задуматься о выборе средств изложения материала (уже на уровне текста, а не его формальной структуры). Иногда за счет использования дополнительных структурных элементов — например таблиц — можно сократить количество основных и одновременно сделать изложение более удобным для пользователя (подробнее об это рассказано ниже; см. п. 3.3.2).

Никакие приводимые здесь положения нельзя рассматривать как догму. Каждый из читателей может столкнуться с особыми случаями, в которых придется нарушать предлагаемые нами ограничения. Тем не менее, следует предпринять все необходимые усилия, чтобы не дать формальной структуре чересчур разрастись вглубь или вширь. И если максимальная глубина и максимальное сечение формальной структуры, устанавливаемые нами, в том или ином конкретном случае окажутся слишком жесткими огра-

нениями, необходимо, разумеется, их пересмотреть. Однако это ни в коем случае не отменяет самого принципа — стремиться держать стихийный процесс разрастания формальной структуры под контролем.

3.2.3. Парадокс формальной структуры и пути его разрешения

С формальной точки зрения документ выглядит как нечто целое, поделенное на основные структурные элементы. Однако для читателя важен не только каждый структурный элемент в отдельности, но и логика перехода от одного структурного элемента к другому. Внешне, формально эти переходы все выглядят одинаково. Однако с содержательной точки зрения следует принимать во внимание следующие обстоятельства:

- Некоторые структурные элементы предусмотрены типовой структурой, тогда как другие — нет. Напомним, что типовая структура задает ту часть формальной структуры, которая роднит документ с аналогичными документами для других программных продуктов. Грубо говоря, структурные элементы, предусмотренные типовой структурой, содержат типичные для любого или почти любого программного продукта сведения, а не предусмотренные типовой структурой — специфические для данного конкретного продукта. Однако внешне эти две разновидности структурных элементов никак не различаются. Читателю же чрезвычайно важно понимать, что роднит программный продукт с другими программными продуктами, а что является его отличительной чертой.
- Некоторые структурные элементы содержат сведения, не относящиеся напрямую к работе программного продукта, но важные для понимания изложения. К таким сведениям могут относиться: сведения из предметной области, обслуживаемой программным продуктом; сведения об определенных форматах файлов или данных; сведения о тех или иных технологиях, алгоритмах, протоколах; сведения об определенном типе программных продуктов, к которому

принадлежит документируемый программный продукт, и т. д. Читателю полезно отграничивать эти структурные элементы от тех, которые содержат конкретные сведения о функционировании программного продукта.

- Структурный элемент может находиться в различных отношениях с предшествующими структурными элементами того же уровня. Он может логически продолжать их или же, наоборот, открывать новую тему, относительно независимую от тематики предшествующих структурных элементов. Читателю же необходимо знать о том, где заканчивается одна цепочка логически связанных рассуждений и начинается другая.

Может возникнуть вопрос: почему бы, если структурный элемент открывает новую тему, это обстоятельство не отразить непосредственно в формальной структуре. Дело в том, что формальная структура документа — инструмент мощный, но не всесильный. Она в состоянии (и то лишь в известной степени) отразить тематическую иерархию: какие функции программы к чему относятся, для чего предназначены, с чем связаны и т. п. Однако логика изложения разворачивается как бы поперек этой иерархии. Иногда частные аспекты той или иной темы непосредственно вытекают один из другого, а иногда они логически независимы.

Пример. В раздел «Контакт и работа с ним в системе» входят подразделы «Создание контакта», «Редактирование контакта», «Удаление контакта» и «Формирование сводной таблицы контактов». Все подразделы, кроме последнего, представляют собой этапы жизненного цикла контакта в системе, а последний подраздел посвящен некоторой очень важной для пользователя возможности, которая никак не связана с жизненным циклом контакта. Подраздел «Формирование сводной таблицы контактов» вполне обоснованно занимает место в указанном разделе, однако логически он с предыдущими подразделами никак не связан.

Парадокс формальной структуры состоит в том, что она предусмотрена для упрощения работы пользователя с документом, однако на самом деле порой придает единообразный вид логи-

чески разнородным переходам и тем самым в известной мере усложняет восприятие.

Эту опасность необходимо иметь в виду и стремиться разграничивать перечисленные выше ситуации. Общих правил здесь не существует, однако можно дать несколько рекомендаций:

- Типовая структура по возможности составляется таким образом, чтобы «типовые» структурные элементы по возможности выносились на верхний уровень иерархии и в начальную часть документа, независимо от их размера и важности содержащихся в них сведений.

Пример. Устанавливается, что руководство пользователя открывается разделами «Введение», «Назначение программы», «Условные обозначения и сокращения», «Требования к программному и аппаратному обеспечению», «Требования к квалификации пользователя», «Установка программы» и «Запуск программы и завершение работы с ней». Эти разделы могут быть не слишком пространными, но они остаются самостоятельными разделами (структурными элементами первого уровня) и размещаются в установленном порядке в начале документа.

- Структурные элементы, не содержащие конкретных сведений о функционировании программного продукта, располагаются в начале документа или структурного элемента более высокого уровня.

Пример. Если в разделе «Функции обработки изображений» необходимо дать описание графический форматов, подраздел «Графические форматы» помещают в начале раздела.

- Рекомендуются разрабатывать формальную структуру документа таким образом, чтобы разрыв в логической последовательности соответствовал началу нового раздела или подраздела, тогда как пункты (в пределах подраздела) или подпункты (в пределах пункта) были связаны логической

последовательностью. Таким образом, членение документа на разных уровнях имеет разную смысловую нагрузку: на верхних уровнях — разграничение тем, на нижних — поэтапное логически последовательное изложение одной темы¹⁵.

Пример. Ниже приведены два варианта фрагмента структуры — до оптимизации структуры в соответствии с приведенными рекомендациями и после. В первом варианте между пп. 7.1.1 и 7.1.2, а также между пп. 7.1.4 и 7.1.5 явно намечаются разрывы логической связности. Это заставляет думать о том, что, во-первых, необходимо обособить как отдельные темы понятие учетной записи пользователя, действия с учетной записью пользователя и предоставление пользователю прав доступа; во-вторых, что структурный элемент «Пользователь» недооценен и ему следует повысить уровень иерархии. Второй вариант значительно лучше соответствует нашим требованиям.

До оптимизации структуры

- 7.1 Пользователь
 - 7.1.1 Учетная запись пользователя
 - 7.1.2 Создание учетной записи пользователя
 - 7.1.3 Редактирование учетной записи пользователя
 - 7.1.4 Удаление учетной записи пользователя
 - 7.1.5 Предоставление пользователю прав доступа к базе данных
 - 7.1.6 Предоставление пользователю прав доступа к функциям системы

¹⁵ Граница может пролегать «выше» или «ниже»: например, (а) разделы, подразделы и пункты используются для разграничения тем, а подпункты — для поэтапного логически последовательного изложения той или иной темы; или наоборот, (б) разделы — для разграничения тем, а более глубокое членение — для поэтапного логически последовательного изложения темы. Мы рекомендуем придерживаться средней линии как оптимального варианта.

После оптимизации структуры

- 7 Пользователь
 - 7.1 Учетная запись пользователя
 - 7.2 Действия с учетной записью пользователя
 - 7.2.1 Создание учетной записи пользователя
 - 7.2.2 Редактирование учетной записи пользователя
 - 7.2.3 Удаление учетной записи пользователя
 - 7.3 Предоставление прав доступа пользователю
 - 7.3.1 База данных и предоставление прав доступа к ней
 - 7.3.2 Функции системы и предоставление прав доступа к ним

Заголовки структурных элементов используются для того, чтобы подчеркнуть различную тематику структурных элементов и логику перехода между ними.

Заголовки «типовых» структурных элементов всегда должны иметь установленную форму.

Заголовкам структурных элементов, не содержащих конкретных сведений о функционировании программного продукта, рекомендуется придавать специальную форму. Например: «Графические форматы: обзор», «Протоколы обмена данными: общие замечания». Если структурный элемент содержит общетеоретические сведения об объекте, явлении, процессе, технологии и т. п., вполне допустимо использовать заголовок, начинающийся с предлога «о»: «О реляционных базах данных».

Изменение формы заголовка сигнализирует пользователю о том, что завершилась одна логически связанная цепочка изложения и, вероятно, начинается другая. Поэтому рекомендуется придавать заголовкам структурных элементов, связанных единой логической цепью рассуждений, однородную форму; там же, где очередной структурный элемент открывает новую тему, форма заголовка должна меняться.

Пример. Ниже приведены два варианта фрагмента структуры. В первом варианте все заголовки разнородны, тогда как во втором варианте заголовки, связанные единой логической цепочкой, имеют сходную форму — описание некоего действия, совершаемого с учетной записью. Благодаря это-

му пользователь легко опознает момент, когда заканчивается логически связанное изложение и начинается другое.

Разнородные заголовки

- 7.1 Параметры пользователя
 - 7.1.1 Учетная запись пользователя
 - 7.1.2 Добавление пользователя в систему
 - 7.1.3 Редактирование идентификационных параметров пользователя
 - 7.1.4 Удаление учетной записи
 - 7.1.5 Сведения о пользователе, их ввод и редактирование

Однородные заголовки

- 7.1 Параметры пользователя
 - 7.1.1 Учетная запись пользователя
 - 7.1.2 Создание учетной записи пользователя
 - 7.1.3 Редактирование учетной записи пользователя
 - 7.1.4 Удаление учетной записи пользователя
 - 7.1.5 Сведения о пользователе, их ввод и редактирование

Перечисленные выше рекомендации отнюдь не являются обязательными во всех ситуациях. Напротив, их следует применять только тогда, когда это покажется автору уместным. Необходимо научиться видеть проблемы, порождаемые унифицированной формальной структурой, и искать пути их решения. Для решения же этих проблем имеет смысл воспользоваться какими-либо из наших рекомендаций. В некоторых случаях лучше использовать в качестве вспомогательного инструмента тщательно продуманную форму заголовков; в других следует придавать дополнительный смысл положению структурного элемента в иерархии или в последовательности однородных структурных элементов. Работая с документом, читатель постепенно усвоит навязанную ему логику и будет воспринимать ее как что-то естественное.

3.3. Сплошной текст и его структура

3.3.1. Структура сплошного текста

Сплошным текстом называется текст, внутри которого нет членения на основные структурные элементы. Каждый из основных структурных элементов первого, второго или третьего уровня может члениться на структурные элементы следующего уровня или, напротив, содержать сплошной текст¹⁶. Структурный элемент четвертого уровня (подпункт), как правило, содержит сплошной текст (как уже говорилось, подпункт не рекомендуется членить на подподпункты; см. подробнее п. 3.2.2).

Формальное структурирование документа не ограничивается его членением на основные структурные элементы. Внутри сплошного текста нет членения на основные структурные элементы, проще говоря — нет заголовков, однако выделяются *дополнительные структурные элементы*.

Дополнительные структурные элементы имеют следующие отличия от основных структурных элементов:

- выделяются независимо друг от друга и не образуют ни иерархии, ни единого членения всего документа в целом;
- выделяются не на основании тематического принципа, а на основании других принципов (см. ниже).

Дополнительные структурные элементы выделяются на основании следующих принципов:

¹⁶ Мы исходим из того, что структурный элемент либо членится на структурные элементы следующего уровня (и тогда не включает в себя сплошного текста иначе как внутри этих элементов), либо содержит только сплошной текст (и тогда не имеет членения на структурные элементы следующего уровня). Иначе говоря, при членении структурного элемента на структурные элементы следующего уровня мы — из соображений структурной строгости и удобства поиска информации в документе — не рекомендуем оставлять сплошной текст между заголовком исходного структурного элемента и первым по счету заголовком структурного элемента следующего уровня (как если бы, например, в нашей книге непосредственно под заголовком «Сплошной текст и его структура» перед заголовком «Структура сплошного текста» присутствовал какой-либо текст). В то же время некоторые авторы придерживаются на этот счет противоположного мнения. Этот спор, видимо, вечен, однако большинство наших рекомендаций приложимы к документам, какую бы сторону в нем ни занимать.

- Конструктивный принцип. Дополнительный структурный элемент выделяется на том основании, что определенные сведения требуют определенной формы подачи: маркированного списка, нумерованного списка, таблицы и т. п.
- Маркировочный принцип. Дополнительный структурный элемент выделяется на том основании, что определенные сведения требуют привлечения внимания.
- Дозировочный принцип. Дополнительный структурный элемент выделяется на том основании, что любые сведения лучше усваиваются, если подаются относительно небольшими порциями.

В соответствии с этим, дополнительные структурные элементы подразделяются на *конструктивные*, *маркировочные* и *абзацы*.

3.3.2. Конструктивные структурные элементы

3.3.2.1. Перечисления

Перечисления используются для изложения однородной информации, например относящейся к одному и тому же объекту, или действию, или отношению, или процессу. Однородные сведения излагаются в виде *пунктов перечисления*. Перечислению всегда предшествует *титulusная фраза*, которая описывает то общее основание, которое лежит в основе перечисления. Например:

В состав документа входят следующие составные части:

- текст;
- иллюстрации;
- таблицы.

Копирование записи осуществляется одним из следующих способов:

- по выбору (одна запись);
- пакетом (несколько записей);
- всей таблицей (все записи).

Основой для создания документа служит любой из следующих объектов:

- другой документ;
- шаблон документа;
- текст в формате RTF.

Решение задачи моделирования завершается одним из следующих результатов:

- выдача итоговых данных;
- сообщение об отсутствии решения;
- принудительное прерывание процесса.

Перечисления играют важнейшую роль в структурировании излагаемых сведений. Речь идет не только о том, чтобы уже существующую последовательность однородных слов, выражений или фрагментов текста представить в виде маркированного или нумерованного списка. Необходимо вовремя заметить, где такое оформление действительно необходимо.

При работе над документом часто возникает соблазн фиксировать информацию о программном продукте произвольным образом, в том порядке, в котором она приходит в голову. Полезные для пользователя сведения при этом ложатся как бы слоями, причем один слой закрывает другой, тот — еще один и т. д. В итоге изложение строится вольно, в описательном стиле, и пользователь часто не в состоянии понять, какого рода сведения ему собираются преподнести в следующем предложении. Скажем, один из приведенных выше примеров мог бы выглядеть так:

Может быть выполнено копирование любой записи, по выбору пользователя. Если возникает необходимость скопировать несколько записей, можно воспользоваться возможностью копирования пакетом. Кроме того, существует возможность скопировать всю таблицу целиком.

Безусловно, пользователь в состоянии разобраться в подобном тексте и понять, что речь идет просто о разных способах копирования записи. Однако он потратит для этого лишние усилия. Если же все изложение строится таким образом, сориентироваться в нем весьма непросто. К тому же, когда два однородных явления описываются по-разному, у читателя возникает ошибочное ощущение, что между этими явлениями существует какая-то принципиальная разница, просто технический писатель о ней ничего не сказал или читатель сам что-то упустил. Это ощущение также препятствует адекватному пониманию излагаемых сведений.

При разработке документа всегда следует придерживаться базового принципа, сформулированного нами ранее: путешественника не должно беспокоить, на какие проблемы натыкался первопроходец. Иначе говоря, пользователь должен получить ценные для него сведения не в том виде, в котором они откладывались в сознании технического писателя по мере изучения последним программного продукта, а в виде, оптимальном для быстрого восприятия и правильного понимания.

Поэтому, если в процессе работы над документом получается слишком большой фрагмент, выдержанный в вольном, описательном стиле, резонно задуматься: нельзя ли структурировать его? Не излагается ли тут однородная информация, которую можно было бы представить в виде перечисления? Особенно закономерны подобные подозрения в тех случаях, когда излагаются сведения следующего характера:

- возможности или способы выполнения какого-либо действия;
- этапы или стадии какой-либо процедуры либо процесса;
- свойства или характеристики какого-нибудь объекта либо явления.

Если информация представлена в виде перечисления, это очень удобно для пользователя. Он сразу видит ситуацию в целом, может охватить ее одним взглядом, оценить общий объем и характер сообщаемых ему сведений. Перечисления очень удобны с мнемонической точки зрения: пользователь легко запоминает сведения, представленные в виде перечисления¹⁷.

Перечисления — это способ структурирования сплошного текста. Чтобы использовать их наиболее эффективно, стоит воспользоваться приведенными ниже рекомендациями.

Титульной фразе рекомендуется уделять серьезное внимание. Обычно она представляет собой содержательное утверждение, в котором обязательно имеется обобщающее выражение, которое характеризует каждый из пунктов перечисления, как то «...следующие составные части»; «...одним из следующих способов»;

¹⁷ Особым случаем перечисления является описание многошагового действия — пользовательская процедура. Она всегда представляется в виде нумерованного списка. Более подробно см. п. 3.4.3.

«...любой из следующих объектов»; «...одним из следующих результатов» и т. п.

Иногда бывает трудно подобрать обобщающее выражение. В таких случаях используются такие слова, как «объект», «действие», «отношение», «процесс», «составная часть», «элемент», «операция» и т. п. Если обобщающее выражение не удастся подобрать вовсе, естественно задуматься о том, действительно ли пункты перечисления однородны.

В титульной фразе не рекомендуется отражать количество пунктов перечисления, поскольку это количество может измениться. Кроме того, обычно количество пунктов перечисления не играет существенной роли в дальнейшем изложении.

Если пункты перечисления представляют собой имена существительные (с зависимыми словами или без таковых), то предпочтительнее ставить эти имена существительные в именительном падеже. А поскольку обобщающее выражение в титульной фразе согласуется (в соответствии с правилами синтаксиса) с пунктами перечисления, то и титульную фразу предпочтительнее строить так, чтобы обобщающее выражение стояло в именительном падеже.

Пример. Если это возможно, не стоит писать:

Документ состоит из следующих частей.

Лучше:

В состав документа входят следующие части.

Пунктами могут быть отдельные слова, выражения, а также целые абзацы из одной или нескольких фраз.

Перечисления обычно оформляются в виде маркированных или нумерованных списков. Рекомендуется использовать нумерованные списки только в следующих случаях:

- Порядок следования пунктов важен для дальнейшего изложения.

Пример. Перечисляются этапы процесса.

Процесс продажи товара в системе состоит из следующих этапов:

1. Создание и выставление счета.
2. Оплата счета и создание счета-фактуры.
3. Создание товарной накладной.

- Предполагается в дальнейшем ссылаться на пункты перечисления.

Пример. При выполнении некоторой операции соблюдаются определенные правила. В дальнейшем на любое из этих правил можно сослаться на номер:

В этом случае соблюдены правила 1 и 2, но нарушается правило 3.

В остальных случаях используются маркированные списки.

Наконец, не рекомендуется начинать несколько пунктов перечисления подряд с одного и того же слова, а тем более — словосочетания.

Пунктов перечисления не должно быть слишком много. Не рекомендуется создавать перечисления более чем из 9 пунктов. Если возникает необходимость создать более длинное перечисление, его лучше оформить в виде таблицы (см. п. 3.3.2.2).

Не рекомендуется использовать без крайней необходимости многоуровневые маркированные и нумерованные списки¹⁸.

3.3.2.2. Таблицы

Таблицы представляют собой специальные формы представления однородной текстовой информации — в этом они могут быть сближены с перечислениями. Таблицы отличаются от пере-

¹⁸ Процедуры оформляются в виде нумерованных списков, причем каждый пункт — шаг — может содержать маркированные списки, таблицы, иллюстрации и т. д. Однако включение в шаг процедуры другой процедуры нежелательно; о проблеме ветвления процедур см. п. 5.1.

числений прежде всего тем, что каждый «пункт» имеет сходную структуру, т. е. состоит из одних и тех же полей. Например, ниже сведения о задании параметров скважины представлены в виде перечисления и в виде таблицы.

Таблица представляет собой еще более структурированный вид текстовой информации, чем перечисление. Когда пользователь встречается перечисление, он сразу понимает, что пункты перечисления представляют собой фрагменты однородной информации, но вынужден сам догадываться, в чем состоит их однородность. В таблице же однородная структура информации представлена в явном виде.

Пример.

Перечисление

В окне задаются следующие параметры скважины:

- глубина (целое число метров, вводится в поле **Глубина** с клавиатуры);
- диаметр (целое число метров, вводится в поле **Диаметр** с клавиатуры);
- профиль скважины (вид профиля выбирается в раскрываемом списке **Профиль**, из классификатора).

Таблица

Параметры скважины и порядок их задания приведены в таблице 1.

Таблица 1. Параметры скважины и их задание

Параметр	Элемент интерфейса	Пояснение
<i>Глубина</i>	Поле Глубина	Целое число. Ввод значения в метрах с клавиатуры
<i>Диаметр</i>	Поле Диаметр	Целое число. Ввод значения в метрах с клавиатуры
<i>Профиль</i>	Раскрывающийся список Профиль	Вид профиля скважины выбирается в раскрываемом списке из классификатора

Однако между таблицей и перечислением есть и более глубокое различие: перечисление в большей степени включено в текст и ориентировано на восприятие по ходу чтения текста подряд. Таблица, напротив, обособлена от остального текста и рассчитана скорее на чтение по строкам: пользователь находит строку, соответствующую, например, параметру *Глубина* (см. приведенный выше пример), и знакомится со сведениями по этому параметру; затем он находит строку, соответствующую параметру *Диаметр*, знакомится со сведениями по этому параметру и т. д. Именно поэтому допускается создавать таблицы с любым количеством строк, тогда как количество пунктов в перечислении по возможности ограничивается (см. п. 3.3.2.1). Перечисление должно восприниматься как единое целое, со своей логикой и последовательностью, тогда как строки таблицы объединены только общей темой таблицы, а читаются независимо друг от друга. Если перечисление слишком разрастается в длину, стоит задуматься: возможно, пользователю будет не так интересна последовательность однородных пунктов, как структурированный массив сведений по каждому из них. В этом случае целесообразно переоформить перечисление как таблицу.

Таблицы удобны для представления функциональной информации как в руководстве пользователя, так и (в особенности) в справочнике пользователя — в тех случаях, когда описывается назначение большого количества кнопок, пунктов меню и т. д.

3.3.2.3. Символические записи

В документации пользователя могут встречаться символические записи: формулы, строки программного кода и т. д. При использовании символических записей следует придерживаться следующих правил:

- после символической записи в обязательном порядке давать точную, полную и однозначную расшифровку всех условных обозначений, используемых в ней;
- использовать, где это возможно, общепринятые условные обозначения и знаки;

- прибегать к символической записи только в тех случаях, когда это считается общепринятым, и воздерживаться от изобретения своих собственных символических записей.

Использование символической записи сразу повышает уровень требований к квалификации пользователя. Поэтому, прежде чем использовать символическую запись, следует задаться вопросом: органично ли для целевого пользователя воспринимать информацию в символической форме? В то же время следует принимать во внимание, что существуют виды сведений, которые невозможно или затруднительно представить иначе как в символической форме: формулы, уравнения, фрагменты текста программ и т. д. Если нет уверенности в том, что целевой пользователь имеет достаточно высокую квалификацию для восприятия этих сведений, возможно, имеет смысл исключить их из документа или вынести их в приложение.

3.3.3. Маркировочные структурные элементы

Маркировочные структурные элементы выделяются в тексте документа для того, чтобы обратить внимание пользователя на те или иные сведения, или *маркировать* их. Обычно маркировочный структурный элемент включает в себя один или несколько абзацев и оформляется с помощью *средств маркировки*: увеличенного отступа слева или (реже) справа, увеличенных интервалов, отделяющих структурный элемент от предшествующего или следующего за ним, специальной пиктограммы, рамки, фона, шрифтового выделения и т. п.

Маркировка тех или иных сведений имеет следующие цели:

- дать знать читателю в процессе чтения, что ему предстоит ознакомление со сведениями особого рода;
- помочь читателю при беглом просмотре выхватить взглядом сведения особого рода для внимательного чтения.

В зависимости от характера сведений более важна та или другая цель маркировки. Поэтому в документе могут быть предусмотрены один или несколько *видов маркировки*. В первом случае рекомендуется использовать разного рода подзаголовки, шриф-

товые выделения, специальные значки. Во втором — лучше использовать увеличенное расстояние между абзацами, измененный абзацный отступ, рамки, значки на полях.

Маркировке обычно подвергают следующие виды сведений:

- предупреждения;
- предостережения;
- советы;
- примеры;
- сведения для отдельных групп пользователей.

Предупреждением называется сообщение о важных последствиях того или иного действия либо бездействия пользователя: изменении данных без возможности восстановления, запуске долговременного процесса, утрате доступа к данным или функциям программы и т. п.

Предостережением называется сообщение о важных негативных последствиях того или иного действия либо бездействия пользователя: безвозвратной порче данных, выходе из строя оборудования и/или программного обеспечения, возможном причинении вреда жизни и здоровью человека и т. п.

Между предупреждением и предостережением имеется важное различие. Предупреждение относится к штатному поведению пользователя, которое, вообще говоря, необязательно является неверным, однако может повлечь за собой непредвиденные последствия, сильно влияющие на дальнейшую работу пользователя с программой либо данными или даже осложняющие эту работу. Предостережение же относится к неправильному поведению пользователя, которое может повлечь за собой вредные последствия.

Для выделения предупреждений и предостережений используются похожие, но различающиеся средства маркировки (для предостережений — более категоричного стиля, для предупреждений — более мягкого).

Предупреждения и предостережения, как правило, важны при внимательном чтении, а не при беглом просмотре. Средства маркировки выбираются так, чтобы не нарушать связности текста, не провоцировать пользователя пропустить важное сообщение.

Обычно предупреждения и предостережения помещаются в том месте, где они наиболее актуальны по контексту. Предупреждение имеет смысл помещать непосредственно перед описанием действия, чреватого серьезными последствиями, а предостережение — там, где, как предполагает автор документа, есть риск совершения пользователем нежелательного действия.

Советом называется сообщение о дополнительных возможностях программы, способах решения некоторых частных, но часто встречающихся задач, упрощенных путях выполнения тех или иных процедур и т. д.

Совет обычно менее важен в контексте изложения и более интересен тем пользователям, которые бегло просматривают уже знакомый документ в поисках дополнительных сведений для внимательного чтения. Следовательно, средства маркировки должны выделять совет на общем фоне, делать его заметным даже при беглом просмотре документа; связь совета с остальным изложением не так важна.

Примером называется разбор каких-либо действий пользователя или автоматической обработки данных на конкретных значениях данных, как правило, подобранных специально для наглядной демонстрации работы программы. Пример важен только в контексте изложения, знакомство с ним отдельно от остального изложения может оказаться бесполезным и даже вредным.

Дело в том, что пример вреден тем же, чем и полезен: он чересчур нагляден. Пользователю гораздо удобнее и приятнее знакомиться с конкретным примером, чем с отвлеченной процедурой, поэтому он склонен подменять усвоение общих правил работы с программой ознакомлением с частными случаями ее применения. При этом, однако, он часто принимает общее за частное и частное за общее, отвлекается на конкретные детали и упускает из виду важные аспекты функционирования программы (если они никак не проявляются в конкретных примерах). Иногда это приводит к неполному знакомству с возможностями программы, иногда, что гораздо опаснее, формирует у пользователя ложное представление о способах работы с программой.

Это не означает, что использование в изложении примеров должно быть полностью исключено. Напротив, при описании

действий, смысл и назначение которых пользователю трудно оценить с первого взгляда, рекомендуется сопровождать изложение примером. Однако пример всегда следует за общим, отвлеченным описанием процедуры и в обязательном порядке маркируется.

Средства маркировки должны быть такими, чтобы:

- пользователь, читая документ подряд, получал предупреждение: далее разбирается конкретный пример, цель которого не сообщить что-то новое, а проиллюстрировать то, что было изложено ранее (должны быть маркированы и начало, и окончание примера);
- при беглом просмотре документа пользователем пример по возможности не слишком бросался в глаза (чтобы не возникало соблазна переходить сразу к чтению примера).

К *сведениям для отдельных групп пользователей* относят, например, сведения для пользователей, имеющих большой опыт работы с программами подобного типа или специфическим образом использующих программу. Важно, чтобы сведения для отдельных групп пользователя бросались в глаза при беглом просмотре документа.

Если слишком часто приходится выделять сведения для отдельных групп пользователей, есть основание задуматься: а не следует ли для этих групп пользователей предусмотреть особый документ, а остальных пользователей не обременять лишним знанием.

3.3.4. Абзацы

Абзацы выделяются в тексте документа на основании дозировочного принципа: любая информация гораздо лучше воспринимается читателем в том случае, если она разбита на небольшие отрезки. Разбиение текста на абзацы по возможности имеет осмысленный характер; граница между абзацами обычно означает, что началось обсуждение нового понятия, или что ранее обсуждались более общие аспекты темы, а теперь будут обсуждаться более частные, или что какой-либо аспект темы, который ранее был только заявлен, теперь рассматривается подробно и т. п.

Рекомендуется разбивать текст на абзацы протяженностью не более 7–9 предложений. Нижняя граница размера абзаца не устанавливается, но следует помнить: если абзац слишком короткий (1–2 предложения), читатель невольно ожидает, что этот абзац содержит какие-то очень важные сведения (иначе бы, полагает он, автор не стал обособлять эту пару предложений от соседних). Поэтому следует избегать дробления всего текста на короткие абзацы из 1–2 предложений, что практикуют некоторые авторы.

Обобщая эти соображения, можно предложить такие параметры разбиения текста на абзацы: в среднем длина абзаца 3–6 предложений; если логика изложения требует удлинить абзац, его протяженность может составить 7–9 предложений.

Если какие-то сведения представляют большую важность и самостоятельность, но могут быть изложены в 1–2 предложениях, их вполне можно выделить в отдельный абзац, но злоупотреблять таким «телеграфным» членением текста не стоит.

3.4. Типы информации и их взаимное расположение

3.4.1. Общие принципы

Документация пользователя неоднородна; она содержит разные типы информации. Часть сведений, сообщаемых пользователю, состоит в конкретных описаниях действий пользователя в интерфейсе программы.

Если изложение ведется с точки зрения пользователя и его задач, эти сведения предстают в виде перечней одно- или многошаговых действий, которые пользователь предпринимает, чтобы достичь той или иной цели. Такая информация называется *процедурной*.

Если изложение ведется с точки зрения программы, эти сведения представляют собой перечни функциональных возможностей, предоставляемых в интерфейсе программы. Такая информация называется *функциональной*.

В целом информация о конкретных действиях пользователя называется *процедурно-функциональной*.

Процедурно-функциональная информация и представляет основной интерес для пользователя, обратившегося к документации. Однако документация пользователя не может целиком и полностью состоять только из процедурно-функциональной информации. Прежде чем указать пользователю, какие именно действия он должен выполнить, необходимо описать задачи, которые ему предстоит решить, перечислить и охарактеризовать объекты, с которыми ему предстоит работать, разъяснить смысл используемых параметров и т. д. Подобные сведения составляют *структурную* информацию.

Наконец, очень часто пользователь нуждается в разнообразных сведениях дополнительного характера. Эти сведения не необходимы для успешной эксплуатации программы; в принципе, пользователь и без них сможет решать с помощью программы свои задачи (коль скоро они не выходят за рамки ее целевого применения). Однако наличие подобных сведений сильно упрощает работу пользователя, позволяет избежать многих досадных ошибок, сокращает период освоения программы. Например, существуют какие-то важные случаи применения описанных в текущем разделе функций или процедур, какие-то возможности в части оптимизации рабочего процесса и т. д. Такие сведения составляют *справочную* информацию¹⁹.

Мы исходим из того, что человеку кажется естественной отчасти диктуемая логикой, а отчасти воспитываемая культурой последовательность изложения. Сначала он рассчитывает понять, что и с какой целью ему предстоит сделать, а затем — каким образом ему это сделать. Исходя из этого, мы считаем, что взаимное

¹⁹ Термин «справочная информация» в данном случае означает совсем не «информация из справочника», а скорее «информация для справки». Не следует путать эти две категории сведений. Если для успешного применения функций программы или решения задач пользователю необходимы какие-либо сведения, они, в принятой здесь терминологии, относятся не к справочной, а к структурной или процедурной/функциональной информации. Если эти сведения, в силу их природы, нарушают ход изложения (например, это громоздкая таблица с описанием режимов работы оборудования), то их можно вынести в приложение, а по ходу изложения непременно поставить на это приложение ссылку. Однако под справочной информацией мы понимаем иное.

расположение информации разных типов внутри сплошного текста должно отвечать следующим принципиальным требованиям:

- информация разных типов по ходу изложения не смешивается;
- последовательность изложения строится так, что пользователь сначала знакомится со структурной информацией, потом с процедурно-функциональной, потом, если сочтет необходимым, со справочной.

Практически это означает, что в документации пользователя совершенно недопустим свободный, так сказать, повествовательный стиль изложения, при котором автор в произвольном порядке то сообщает сведения общего характера, то обращается к описанию конкретных действий, то приводит замечания справочного характера. Такой стиль характерен для технических писателей, пишущих «в манере первопроходца» и не задумывающихся об удобстве восприятия текста пользователем.

3.4.2. Структурная информация

Структурная информация образует небольшое введение общего характера, предпосланное основной, процедурно-функциональной части. Подобно тому как документ в целом начинается с вводной части, в которой рассматриваются цели и задачи программы, описывается модель действительности, используемая в программе, указываются условия успешного функционирования программы и т.д., — точно так же каждый связный фрагмент сплошного текста начинается с такой же вводной части, но в миниатюре.

В структурную информацию включается краткий обзор содержания раздела (подраздела, пункта, подпункта), в котором конспективно рассматриваются:

- цель и назначение рассматриваемых функциональных возможностей, практический смысл рассматриваемых пользовательских задач;
- типы данных или объектов, их краткая характеристика и точное и лаконичное описание их формата;

- общее описание действий пользователя, ответных действий программы и изменений, происходящих в результате действий пользователя;
- условия успешного выполнения действий.

Столь амбициозная программа не должна нас смущать: каждый из этих аспектов может быть описан кратко, одним предложением или даже частью предложения. Например:

Редактор фотографий предназначен для оперативного редактирования загружаемых в базу данных фотографий сотрудников (обрезки фотографии, увеличения яркости и т. п.). Редактор обрабатывает фотографии, загруженные в программу из файлов формата JPEG и GIF. Результаты работы сохраняются в базе данных. Для успешной работы редактора и сохранения результатов работы в базе данных требуется наличие дополнительного места на жестком диске.

Собственно говоря, стремление к лаконичности не только допускается, но приветствуется. Если структурная информация требует слишком много места, следует задуматься: возможно, было бы целесообразно часть ее перенести в общий обзор во вводной части документа, а в данном разделе оставить только самые необходимые сведения со ссылкой на общий обзор.

Так, если перед описанием конкретных действий приходится давать определения тем или иным понятиям, вероятно, следует вынести эти определения и поясняющие их рассуждения в особый раздел, а перед описанием конкретных действий вводить понятия без явных определений, со ссылками на вводную часть. Например:

В журнале событий фиксируются *события*. Каждое событие имеет тот или иной *тип* и соответствует определенному *шаблону* (понятия события, типа события и шаблона события подробно разъяснены в разделе 6).

Однако иногда разумнее поместить определение понятия и сопутствующие ему объяснения непосредственно перед процедурно-функциональной информацией. Например, если значительная часть работы с программой представляет собой задание значений для большого числа параметров (настроек) — будет,

скорее всего, неразумно сначала про все эти параметры говорить в общем и целом, а затем начинать разговор заново и описывать конкретные действия по заданию значений. Впрочем, сведения концептуального характера (относящиеся ко всем параметрам или к их отдельным группам) все-таки и в этом случае лучше выносить в особый раздел во вводной части.

3.4.3. Процедурно-функциональная информация

Процедурно-функциональная информация двулика: она может быть процедурной (изложение с точки зрения пользовательских задач) или функциональной (изложение с точки зрения функций программного продукта).

Процедурная информация излагается в виде нумерованной последовательности *шагов процедуры*. Перед нумерованной последовательностью располагается *титульная фраза*, в которой указывается назначение выполняемой процедуры, то есть решаемая пользовательская задача, например:

Чтобы создать учетную запись пользователя...

Чтобы сформировать список адресатов письма...

Чтобы отправить сообщение на тот или иной электронный адрес...

Для каждого шага процедуры приводятся следующие сведения:

- Действия пользователя, описанные с максимальной степенью детализации, принятой в данном документе (о детальности изложения см. п. 2.3.4). Например:

Нажмите на кнопку **Копировать**.

- Наблюдаемая реакция системы на действия пользователя и ее интерпретация. Например:

Имя объекта исчезнет из правого списка и отобразится в левом списке. Это означает, что объект включен в список объектов, предназначенных для копирования.

- Последствия действий пользователя для данных, их внешнего отображения, процессов их обработки, настроек программы, ее текущего состояния. Например:

В результате добавления в этот список объект будет назначен для копирования, но само копирование происходит не на данном шаге, а на следующем (при нажатии на кнопку **ОК**; см. шаг 6 настоящей процедуры).

Пример. Приведем описание шага процедуры целиком:

Нажмите на кнопку **Копировать**. Имя объекта исчезнет из правого списка и отобразится в левом списке. Это означает, что объект включен в список объектов, предназначенных для копирования. В результате добавления в этот список объект будет назначен для копирования, но само копирование происходит не на данном шаге, а на следующем (при нажатии на кнопку **ОК**; см. шаг 6 настоящей процедуры).

Функциональная информация состоит из описания одной или более функциональной возможности, относительно каждой из которых разъясняется ее смысл и назначение, ее реализация в интерфейсе программы и, при необходимости, даются разного рода пояснения и дополнения. Если структурный элемент целиком посвящен одной функции, то сначала излагается структурная информация (смысл и назначение функции), затем функциональная информация (реализация функции в пользовательском интерфейсе программы) и, наконец, справочная информация.

Если же, как это чаще всего бывает, в структурном элементе описано несколько функций, а именно функции, доступные в диалоговом окне, рабочем окне, меню, динамическом меню и т. п., то часто используют таблицу. В левой части таблицы помещают структурную информацию, в середине функциональную, в правой части справочную (о двух разных принципах компоновки раздела см. п. 3.4.5). При этом таблице предшествует титульная фраза, в которой описываемым функциям дается общая характеристика, например:

В диалоговом окне **Общие настройки** задаются следующие параметры процесса (табл. 3).

Посредством команд меню **Формат** выполняются следующие действия по форматированию текста (табл. 5).

Для каждой функции приводятся следующие сведения:

- Наименование функциональной возможности. Например:

Создание учетной записи по образцу.

- Содержательный смысл функциональной возможности (какие изменения происходят в системе в результате использования этой функциональной возможности).

Например:

Создается новая учетная запись, все атрибуты которой, кроме фамилии, имени и отчества пользователя, совпадают с существующей учетной записью.

- Реализация функциональной возможности в пользовательском интерфейсе программы (какие действия следует предпринять пользователю, чтобы воспользоваться этой функциональной возможностью). Например:

1. Щелчком мыши выберите в таблице пользователей существующую учетную запись в качестве образца.
2. Щелкните правой клавишей мыши и в динамическом меню выберите пункт **По образцу**. На экране появится форма создания учетной записи. Все поля, кроме полей **Фамилия**, **Имя** и **Отчество**, будут автоматически заполнены в соответствии с выбранным образцом.
3. Введите фамилию, имя и отчество пользователя в полях **Фамилия**, **Имя** и **Отчество** соответственно.
4. Нажмите на кнопку **ОК**. В таблице пользователей появится новая учетная запись.

Пример. Приведем описание шага функции целиком:

Создание учетной записи по образцу	Создается новая учетная запись, все атрибуты которой, кроме фамилии, имени и отчества пользователя, совпадают с существующей учетной записью.	- Щелчком мыши выберите в таблице пользователей существующую учетную запись в качестве образца. - Щелкните правой клавишей мыши и в динамическом меню выберите пункт По образцу. На экране появится форма создания учетной записи. Все поля, кроме полей Фамилия, Имя и Отчество, будут заполнены в соответствии с выбранным образцом. - Введите фамилию, имя и отчество пользователя в полях Фамилия, Имя и Отчество соответственно. - Нажмите на кнопку ОК. В таблице пользователей появится новая учетная запись.
------------------------------------	---	---

Иногда практикуют иной порядок описания: элемент интерфейса — функциональная возможность — ее описание. Мотивируют это тем, что пользователь видит перед собой интерфейс программы и ему удобнее, когда в документации изложение отталкивается от интерфейса.

С одной стороны, такая аргументация кажется несостоятельной. К программе обращаются для того, чтобы решить определенную задачу, воспользоваться определенной функцией. Поэтому начинать изложение следует с ответа на вопрос «для чего», и лишь затем сообщать «как». Это общее соображение подкрепляется простым практическим соображением: функциональная возможность часто бывает реализована несколькими элементами пользовательского интерфейса, но элемент интерфейса реализует, как правило, только одну функциональную возможность (см. приведенные выше примеры). Понятно, что идти при изложении функциональной информации от элемента интерфейса к реализуемой им возможности еще и технически неудобно.

С другой стороны, во многих программах предусмотрены функции, которые сами по себе не нужны пользователю, однако

необходимы для корректной и эффективной работы программы. Таковы, скажем, функции настройки программы. Пользователю необходимо настроить программу, чтобы ею потом можно было успешно пользоваться. Часто функции настройки располагаются в диалоговом окне настройки, включающем одну или несколько вкладок. От пользователя требуется, чтобы он ввел значения определенных параметров, выбрал определенные варианты работы программы, включил или выключил определенные режимы. Пользователь получает от системы запрос и отвечает на него, заполняя поля, выбирая варианты в раскрывающихся списках и переключателях, устанавливая или снимая флажки. При описании подобных диалоговых окон кажется возможным отталиваться не от функции как таковой, а от элемента интерфейса. Например:

Формат даты	Выбор формата, в котором дата будет отображаться в выходных документах	Установите переключатель в положение, соответствующее требуемому формату
Тип валюты	Выбор валюты, в которой предполагается выполнять расчеты	В раскрывающемся списке выберите один из доступных пунктов (из классификатора валют)
Предварительный просмотр	Включение режима предварительного просмотра документов	Чтобы включить режим предварительного просмотра, установите флажок, иначе снимите его

Процедурная информация может содержать описания одношаговых действий. Например:

Чтобы сформировать отчет по текущему состоянию базы данных, нажмите на кнопку **Отчет**.

Описание одношагового действия лучше всего помещать в отдельный абзац. Однако тут возникает вопрос: а правильно ли был выбран способ изложения материала? Обычно процедурная информация соответствует изложению материала с точки зрения пользователя, а это предполагает, что у пользователя есть ряд до-

статочны конкретных практических задач и их решение описано в виде последовательности шагов. Что же касается одношаговых (а иногда и двух- или трехшаговых) действий, совершаемых пользователем по ходу работы, их удобнее описывать как предоставляемые программой функциональные возможности. Это не мешает включать упоминание тех же функциональных возможностей программы и соответствующих им элементов пользовательского интерфейса в пользовательские процедуры — многошаговые последовательности, описывающие решение более конкретных практических задач.

Пример. Функция формирования отчета в общем виде описана в составе функциональной информации:

Формирование
отчета по базе
данных

Формируется выходной
документ, включающий
сводную информацию
о записях, хранящихся в
базе данных

Кнопка **Отчет**

Та же функция упомянута в контексте решения пользовательской задачи в составе процедурной информации:

Чтобы подготовить данные для анализа состояния дел в компании:

1. Нажмите на кнопку **Загрузить**. Откроется диалоговое окно **Список баз**.
2. Щелчком мыши выберите базу в списке, после чего нажмите на кнопку **ОК**.
3. Нажмите на кнопку **Отчет**. В окне просмотра отчета сформируется отчет по базе данных.
4. Нажмите на кнопку **Excel**. Откроется стандартное окно Microsoft Windows **Сохранить файл**.
5. В стандартном окне Microsoft Windows укажите имя файла и путь в нему.

В результате данные для анализа будут подготовлены и сохранены в формате Microsoft Excel. Дальнейший анализ данных рекомендуется выполнять в приложении Microsoft Excel.

3.4.4. Справочная информация

К справочной информации относятся, в первую очередь, следующие сведения:

- Рекомендации по использованию тех или иных функциональных возможностей программы для специфических целей или важных частных случаев применения программы. Например:

Формирование отчета по базе данных рекомендует-
ся использовать для последующего анализа данных,
например для анализа и обработки показателей со-
стояния дел в компании.

- Указание упрощенных способов выполнения тех или иных действий. Например:

Чтобы создать несколько учетных записей пользо-
вателей, различающихся только фамилией, именем
и отчеством, необязательно создавать каждую учет-
ную запись сызнова. Достаточно создать ряд учет-
ных записей по образцу уже существующей.

- Советы по оптимизации использования и/или обслужива-
ния программ. Например:

Рекомендуется время от времени запускать проце-
дуру оптимизации базы данных. В результате этой
процедуры сильно сокращается место, которое
база данных занимает на диске, и повышается про-
изводительность системы при поиске и обработке
данных. Эта рекомендация особенно актуальна для
пользователей компьютеров с относительно невысо-
кой производительностью и ограниченными ресур-
сами дисковой памяти.

3.4.5. Компоновка различных типов информации

Различные типы информации, как уже говорилось, не должны смешиваться; их взаимное расположение должно быть таково, что пользователь сначала знакомится со структурной информацией, затем с процедурно-функциональной, и затем — со справочной.

Мы различаем два основных принципа компоновки информации: *последовательный* и *табличный*.

При последовательной компоновке информации в начале сплошного текста размещается структурная информация, затем следует описание пользовательских процедур или функциональных возможностей. В конце помещается справочная информация. При этом пользователь легко вычленяет из общего хода изложения все три типа информации, поскольку процедурная или функциональная информация выделяется особым характером форматирования (нумерованные списки, таблицы); поэтому мы не рекомендуем вводить какие бы то ни было дополнительные текстовые или графические указания на тот или иной тип информации.

Последовательная компоновка информации кажется естественной и логичной, но она не всегда удобна. Иногда необходимо описать ряд функциональных возможностей, причем каждая из функциональных возможностей нуждается в подробном комментарии общего характера. Так бывает, например, в тех случаях, когда в диалоговом окне задаются значения параметров, причем каждый из параметров нуждается в комментарии общего характера: что он означает, зачем задается и пр. В этой ситуации едва ли целесообразно сначала давать общее описание всех параметров подряд (структурная информация), затем отдельно указывать для каждого из них порядок задания (функциональная информация), а затем, при необходимости, по каждому из них давать дополнительные сведения (справочная информация). Кажется более рациональным использовать табличный принцип компоновки информации, то есть разместить структурную, функциональную и справочную информацию в одной таблице — так, как это было сделано в п. 3.4.3.

Когда пользователь знакомится с документом, он читает таблицу по строкам и воспринимает информацию именно в таком порядке, как предложено выше: сначала структурную, потом

функциональную, потом справочную. При этом он может или читать всю таблицу подряд, или быстро отыскать интересующий его параметр и ознакомиться с его описанием. Порядок ознакомления остался тем же, только направление чтения изменилось с вертикального на горизонтальное (вдоль строк таблицы).

Если изложение материала строится с точки зрения пользователя, т. е. взаимодействие пользователя с программой описывается в виде процедур, обычно используется последовательный принцип компоновки информации. Если же изложение материала строится с точки зрения программы, т. е. основой изложения является описание функциональных возможностей, то часто оказывается удобнее табличный принцип компоновки информации.

3.5. Проблема логического повтора

3.5.1. Повторяемость как свойство технического текста

Программный продукт по своей природе хорошо структурирован; он весь состоит из стандартных элементов интерфейса, серийных разработок, повторяющихся решений. Можно сказать, что он складывается, словно из кубиков, из сходных между собой составляющих, мелких и крупных. Нет ничего удивительного, что «кубики» имеют тенденцию повторяться — эти повторы не случайны, но, напротив, вполне закономерны. Например, чтобы задать значения серии параметров, используется форма ввода данных. Серии параметров могут иметь разный смысл и разную природу, относиться к разным объектам и т. д., однако во всех случаях используется стандартное решение — форма ввода данных.

Однако то, что является сильной стороной в конструктивном отношении, порождает известные проблемы при документировании. Подобно тому как программа состоит из постоянно повторяющихся (с модификациями или без таковых) элементов, а деятельность пользователя складывается из повторяющихся элементарных операций, — точно так же документ, создаваемый техническим писателем, состоит из повторяющихся больших и малых фрагментов, описывающих сходные функции, процедуры,

процессы, явления и т. п. Иногда эти фрагменты повторяются в точности, иногда — с точностью до замены некоторых фигурирующих в них сущностей на другие, аналогичные.

Пример. Фрагмент «Нажмите на кнопку **ОК**» встречается в конце описания работы с большинством диалоговых окон, а фрагмент «В поле ... введите ...» встречается каждый раз, когда в ходе выполнения процедуры предлагается ввести значение некоторого параметра в некотором поле. При этом первый фрагмент воспроизводится каждый раз в точности, а второй — с учетом разных названий поля и имен (наименований) параметра.

3.5.2. Фигуры описания

Фигурами описания мы называем повторяющиеся фрагменты небольшого размера, хорошо обозримые и воспринимаемые читателем как единое целое. Обычно речь идет о словосочетаниях, самое большее — об отдельных коротких простых предложениях. Их повторяемость закладывается на уровне работы над текстом, поскольку они являются своеобразными клише, которыми технический писатель пользуется, когда создает документ: «на экране отображается процесс...»; «...формируется массив данных»; «чтобы выполнить это действие...» и т. п.

Фигуры описания подразделяются на *терминологические* и *унифицирующие*.

Терминологические фигуры описания представляют собой словосочетания, являющие строгими терминами, например «создать копию файла», «форматировать диск», «удалить программу с компьютера» и т. д. Употребление терминологических фигур описания определяется не столько волей технического писателя, сколько общепринятой практикой, а также корпоративными правилами (коль скоро речь идет о документировании в компании со сложившейся системой управления терминологией; см. об этом в п. 4.1.1.1). Терминологические фигуры описания фактически являются терминами и вводятся как термины (об этом см. п. 4.1.1).

Унифицирующие фигуры описания представляют собой повторяющиеся модели, по которым строится описание сходных ситуаций, функций, процедур, объектов, процессов, явлений и т. д. Унифицирующая фигура описания может представлять собой законченное словосочетание или же словосочетание, в котором задана только часть слов, а часть определяется конкретным контекстом.

Пример. В п. 3.5.1 приведена фигура описания

в поле ... введите...

На этой фигуре описания равным образом основаны следующие формулировки:

В поле **Диаметр** введите значение диаметра штуцера в метрах.

В поле **Дата** введите дату подписания документа.

В поле, расположенном в правом нижнем углу, введите номер копии документа.

В некоторых случаях фигура описания вообще не содержит наперед заданных слов, а лишь определяет грамматическую форму слова или нескольких слов. Например, для обозначения конкретных действий пользователя в интерфейсе программы употребляются глаголы в повелительном наклонении (в вежливой форме, то есть во множественном числе).

Унифицирующие фигуры описания играют важнейшую роль — они поддерживают единообразие формулировок в документации пользователя. Единообразие же формулировок чрезвычайно важно для пользователя. Если технический писатель употребляет в двух сходных случаях две непохожие формулировки, это всегда чревато для пользователя затруднениями и недоразумениями. Дело в том, что технический писатель знает, что две эти формулировки употребляются для обозначения сходных явлений, но пользователь этого не знает. Два участника коммуникации видят вещи с противоположных сторон: автор документации, техниче-

ский писатель, отталкивается от изученной им реальности и ищет для ее описания подходящие формулировки, тогда как читатель документации, пользователь, видит формулировки и по ним восстанавливает картину реального положения дел. Увидев две непохожие формулировки, читатель, во-первых, не сразу осознает, что за ними скрываются два сходных явления, не усмотрит (по крайней мере, с первого взгляда) важного для понимания структурного единства программы. Во-вторых, даже догадавшись, что речь идет о похожих вещах, он будет сомневаться: ведь автор текста зачем-то использовал разные формулировки; вдруг он имел в виду важное различие? Наконец, в-третьих, читателю придется каждый раз вдумываться в смысл каждого слова, тратить время на интерпретацию новой формулировки — тогда как встретив формулировку, по построению сходную с употребленными прежде, он поймет ее смысл и место в изложении гораздо быстрее.

Унифицирующие фигуры описания — это элементы структуры документа (документации), хотя и наиболее мелкие. Составление списка унифицирующих фигур описания является, по сути дела, частью проектирования структуры и, как следствие, осуществляется «в два эшелона»: предварительный список составляется до начала работы над текстом, окончательный список получается в результате уточнения и дополнения по ходу работы над документом (документацией).

Действительно, существует целый ряд унифицирующих фигур описания, которые легко заготовить заранее, предвидя, что они обязательно понадобятся; скажем, для описания конкретных действий пользователя (см. выше), для титульных фраз описаний функций и процедур, для описания некоторых аспектов деятельности пользователя или функционирования программного продукта и т. д.

Пример. Для конкретных действий пользователя, как уже говорилось, последовательно употребляется повелительное наклонение:

Нажмите на кнопку **ОК**.

Для титульной фразы в описании процедуры — союз «чтобы» в сочетании с глаголом в неопределенной форме:

Чтобы выполнить полнотекстовый поиск...

Для упоминания наличия у программы примечательной функции — выражение «программа оснащена функцией...»:

Программа оснащена функцией полнотекстового поиска.

Все эти устойчивые формулы являются ничем иным, как унифицирующими фигурами описания.

Исходя из общих соображений, технический писатель составляет предварительный список унифицирующих фигур описания. При работе над текстом он, однако, может столкнуться с необходимостью пополнить этот список.

Пример. При описании различных функций выясняется, что для каждой функции следует указать перечень режимов работы программы, в которых эта функция доступна. Следовательно, появляется фигура описания «Данная функция доступна в следующих режимах...», которая употребляется для единообразного формулирования титульных фраз при перечислении режимов. Если не зафиксировать эту фигуру описания, в роли титульной фразы будут встречаться разные по своему построению формулировки:

Данная функция доступна в следующих режимах...

Применение данной функции возможно в следующих режимах...

Данную функцию поддерживают следующие режимы...

Таким образом, технический писатель, обнаружив в процессе работы, что какая-то формулировка, какой-то способ формулирования встречаются не один раз, добавляет новую фигуру описания в общий список. Подобная сопутствующая деятельность

чрезвычайно важна даже при работе в одиночку; при коллективной работе ведение общего, доступного всем участникам работы репозитория фигур описания могло бы кардинальным образом улучшить качество работы (иначе непонятно, как можно было бы достичь единообразия формулировок в частях документации, написанных разными людьми). Список фигур описания впоследствии мог бы стать серьезным подспорьем для переводчиков документации на другие языки²⁰.

Список фигур описания может иметь, в зависимости от постановки задачи документирования, различные области применения:

- отдельный документ;
- часть документации пользователя или документация пользователя в целом;
- практика документирования в целом.

Формулировки в отдельном документе должны быть единообразны. Поэтому без единого списка фигур описания при создании того или иного документа не обойтись. Безусловно, следует говорить о единообразии формулировок в пределах нескольких документов, входящих в комплект документации пользователя, или даже в пределах всего комплекта; однако здесь необходима оговорка. Одно и то же положение в принципе может требовать разных формулировок в зависимости от адресата (прежде всего, в силу разного уровня знаний в предметной области и/или в области информационных технологий). Однако это не отменяет возможности, и даже желательности существования общего списка фигур описания для всего комплекта документации (некоторые фигуры описания могут иметь более узкую область применения — один или несколько документов).

Наконец, во многих случаях целесообразно вести общий список фигур описания, имеющий еще более широкую область применения и охватывающий всю деятельность коллектива технических писателей в рамках более или менее стабильного организационного решения. Как правило, в таких случаях идет

²⁰ Это особенно актуально при использовании систем и баз памяти переводов (translation memory). В этих случаях употребление фигур описания в соответствии с заранее оговоренным списком заметно ускорит перевод текстов и существенно снизит его стоимость.

речь о близких по назначению программных продуктах либо о программных продуктах, использующих сходные технологии. Документирование в таких случаях представляет собой обширную практику, охватывающую работу по подготовке документации пользователя на целый ряд программных продуктов. Разумеется, здесь необходима еще большая осторожность в использовании одних и тех же фигур описания в документации на разные программные продукты. Однако ведение общего списка фигур описания, тем не менее, кажется весьма полезным.

Часто приходится сталкиваться с желанием технического писателя получить готовый словарь фигур описания, с тем чтобы в дальнейшем применять его в своей практике. Это желание понятно, но не слишком обоснованно. Терминологические фигуры описания тесно связаны с различными отраслями информационных технологий, их значения и практика их употребления определяются профессиональным сообществом, а не волевым усилием технических писателей. Список же унифицирующих фигур описания трудно предоставить в готовом виде — он слишком сильно зависит от условий задачи документирования: программного продукта, его назначения, специальных задач, стоящих перед техническим писателем и т. д. Методически было гораздо более корректным включить разработку списка фигур описания в конкретный процесс документирования и возложить ответственность за разработку этого списка на технического писателя, знакомого со спецификой программного продукта и условиями его применения.

При разработке унифицирующих фигур описания следует помнить: они всегда рассчитаны на интуитивное понимание читателем, пользователем, по ходу чтения. Поэтому унифицирующие фигуры описания должны быть по возможности краткими и легкими для восприятия целевым пользователем; порождаемые на их основе формулировки должны соответствовать требованиям, предъявляемым к формулировкам (см. п. 4.2.1), — быть строгими, лаконичными, завершенными и однозначными.

3.5.3. Логические повторы и перекрестные ссылки

Повторяющийся фрагмент может быть достаточно пространственным и представлять собой законченное изложение небольшой темы. Так бывает, главным образом, в следующих случаях:

- Какая-либо функция (процедура, объект и т. п.) используется при решении различных пользовательских задач и упоминается в документе многократно.

Таковы функция поиска, функция формирования рабочего списка объектов, функция формирования отчетов и т. п.

Пример. В программе учета движения товаров на складе в самых разных ситуациях используется функция поиска записи по атрибутам.

- Разные по смыслу и назначению функции (процедуры, объекты и т. п.) имеют сходную структуру и описываются по общей модели.

Пример. В программе учета кадров предполагается создание учетных записей для пользователей, групп пользователей и служебных пропусков. Содержательно объекты этих трех типов имеют совершенно разную природу, смысл, назначение, жизненный цикл; однако формально они создаются одинаково: открывается диалоговое окно общего образца, в котором задаются параметры, в первую очередь — идентификатор(ы).

Если фрагмент достаточно велик, он уже не воспринимается читателем как единое целое. Поэтому если его воспроизводить всякий раз, когда он актуален (например, всякий раз, когда необходимо воспользоваться функцией формирования рабочего списка объектов, воспроизводить целиком описание этой функции), документ может показаться читателю слишком громоздким. В этом случае от повтора отказываются, — например, помещая описание функции (процедуры, объекта) один раз, а затем лишь ссылаясь на это описание (посредством перекрестной ссылки).

Такую ситуацию мы называем *логическим повтором*: логически описание присутствует столько раз, сколько оно необходимо, но в действительности его можно заменить, например, ссылкой на уже имеющееся описание. Проблема логического повтора состоит в том, что для одних читателей удобнее, когда каждый раз воспроизводится описание, а другие предпочитают перекрестные ссылки.

Так, если читатель обращается к документу с целью получить справку по тому или иному конкретному поводу, он настроен на чтение небольшого отрывка и желал бы именно из него получить все необходимые сведения. Перекрестные ссылки, которые заставили бы его обращаться к другим частям документа, покажутся ему неуместными. Если в тексте упоминается какая-либо важная функция, такому читателю было бы удобнее, если бы описание этой функции помещалось прямо здесь (независимо от того, было это описание уже помещено в других частях документа или нет).

Напротив, если читатель знакомится с документом систематически, читает его подряд, в том порядке, в котором он написан, и постепенно усваивает материал, ему было бы неудобно, если бы описания некоторых функций постоянно повторялись: во-первых, как уже говорилось, это загромождало бы текст, во-вторых, приучало бы читателя к мысли, что автор документа постоянно топчется на месте и говорит, в сущности, все время одно и то же. Это побуждает пропускать знакомые места или места, кажущиеся знакомыми, и в конечном итоге приводит к плохому усвоению или игнорированию важных сведений. Изложение с постоянными повторами становится бесформенным, теряет структурированность и не вполне отвечает читательским ожиданиям (он хочет ознакомиться с новыми сведениями, а ему навязывают уже известное). Таким образом, «систематическому» читателю удобнее текст, где логические повторы реализованы как перекрестные ссылки.

Полный повтор ранее сказанного в ходе изложения и краткая перекрестная ссылка на приведенное выше описание — наиболее радикальные решения проблемы логического повтора. Используются и компромиссные варианты: перекрестная ссылка

с краткой аннотацией (пояснением, какого рода информация дается по ссылке); частичный повтор с перекрестной ссылкой (в первый раз приводится полное описание, во все последующие — краткое со ссылкой на полное); описание, по формату обособленное от остального текста. Кроме того, каждый случай логического повтора допускает отдельное решение: например, описания некоторых из постоянно используемых функций могут повторяться каждый раз, когда эти функции упоминаются; для некоторых же описание дается один раз, а в дальнейшем приводятся лишь ссылки. Решение проблемы логического повтора подыскивается таким образом, чтобы предполагаемому читателю было удобно пользоваться документом.

3.5.4. Логические повторы в структуре документа

Логические повторы неизбежны и являются типичной чертой структуры технического текста. Для восприятия текста читателем они играют двойственную роль: может оказаться, что они помогают усвоить новые сведения, подчеркивают взаимосвязь между различными функциями и модулями программного продукта, делают более ясной логику работы с этим продуктом, а может — что они сбивают с толку, мешают запомнить важное, затеяют логику. Для того чтобы логические повторы помогали, а не мешали пользоваться документом, необходимо:

- Продумать систему логических повторов.
- В каждом случае выбрать оптимальный способ реализации логического повтора.

Система логических повторов тесно связана со структурой документа. Порой структура документа корректируется в процессе работы над текстом, если выявляется необходимость логического повтора.

Скажем, во время работы над текстом выясняется, что функция формирования рабочего списка объектов является «сквозной», используется время от времени для решения разных пользовательских задач. При подготовке первого, чернового варианта текста этот факт иногда проходит незамеченным: в одном случае

функция описана более подробно, в другом менее, в третьем дана ссылка на предшествующее описание и т.д. Однако впоследствии текст и структура документа корректируются: разрабатывается единое, исчерпывающее и строгое описание функции, которое по возможности выделяется в отдельный структурный элемент (раздел, подраздел, пункт или подпункт) и помещается в начале документа, до первого случая, когда эта функция упоминается по логике изложения.

Вообще, возникновение у автора потребности в логическом повторе само по себе является поводом задуматься: не следует ли поместить повторяющийся фрагмент в начале документа под отдельным заголовком? Методически это позволит читателю сразу обратить внимание на наличие важной функции, подготовит его к той вспомогательной, но важной роли, которую она будет играть в дальнейшем изложении. Неудачным следует признать распространенное решение, когда такого подготовительного раздела (подраздела, пункта, подпункта) нет, а функция (процедура, объект и т.п.) описывается подробно при первом упоминании в документе, а при втором и последующих упоминаниях помещаются перекрестные ссылки на первое. Ведь при этом читатель, знакомясь с одним разделом, посвященным какой-либо конкретной теме, будет вынужден обратиться к другому разделу, посвященному также конкретной теме, но совершенно другой. Это нелогично, а потому сбивает читателя с толку.

После того как логические повторы выявлены, следует для каждого из них выбрать способ реализации (таблица 3).

Таблица 3. Основные способы реализации логических повторов и рекомендации по их применению

Способ	Область применения
Полный повтор описания каждый раз, когда оно актуально	Применяется в справочнике пользователя. Иногда используется в руководстве пользователя, если оно ориентировано на малоопытного, но дисциплинированного пользователя и содержит только четкие, пошаговые инструкции
Перекрестная ссылка на полное описание	Применяется в руководстве пользователя или ином документе, ориентированном на систематическое освоение. В контекстной справке или в веб-документации, доступной через Интернет или интранет, используются гиперссылки. Иногда перекрестная ссылка для удобства сопровождается кратким пояснением (аннотацией). Например: Создайте список объектов, для чего воспользуйтесь функцией формирования списков объектов (см. п. 5.1). Функция формирования списков объектов является вспомогательной функцией, постоянно используемой в тех случаях, когда необходимо создать список объектов для последующей обработки
Краткое описание с перекрестной ссылкой на полное описание	Применяется в руководстве пользователя в тех случаях, когда тема логического повтора слишком сложна или повторяется слишком редко и необходимо напомнить читателю основные сведения по этой теме. Например: Создайте список объектов. Для этого: 1. Нажмите на кнопку Список. Откроется диалоговое окно Список объектов. 2. В левой части окна сформируйте список объектов. Чтобы поместить объект в список, выберите его в правой части окна и нажмите на кнопку <. 3. Нажмите на кнопку ОК. Подробно функция формирования списков рабочих объектов рассматривается в п. 5.1

Оптимальным решением проблемы логического повтора мог бы быть документ, формируемый, в зависимости от целей использования и адресата, с установкой на явные повторы или же, напротив, на перекрестные ссылки.

Заметим еще раз: логические повторы — неизбежная примета технического текста. У читателя ни в коем случае не должно оставаться ощущения, что автор документа не замечает логических повторов; наоборот, они должны быть подчеркнуты, маркированы, что называется — отработаны.

Повторяющиеся фрагменты текста должны быть одинаковыми или аналогичными; перекрестные ссылки должны вести в начало документа, где помещена исчерпывающая информация о «сквозной» функции, процедуре, объекте и т. п.

3.6. Перекрестные ссылки и случаи их использования

В предыдущем разделе мы упоминали о важнейшей функции перекрестной ссылки: отработке логических повторов. Однако существуют и другие случаи использования перекрестных ссылок.

Перекрестные ссылки могут быть разделены на две группы по формальному критерию: одни перекрестные ссылки ведут назад (к более ранним структурным элементами документа), другие — вперед (к более поздним). Это различие актуально для бумажной документации и гораздо менее актуально для гипертекста. Важнее разграничение перекрестных ссылок по содержательному критерию; мы выделяем следующие основные разновидности ссылок:

- *Оторные* — отсылающие к сведениям, необходимым для правильного и полного усвоения излагаемого материала. Например:

Сформируйте рабочий список объектов (формирование рабочего списка объектов описано в п. 6.1).

- *Навигационные* — помещаемые в обзорных замечаниях и указывающие на конкретное изложение той или другой темы. Например:

Программа оснащена функцией обработки графических изображений (см. п. 10.3).

- *Ограничительные* — сопровождающие замечания о наличии тех или иных сведений в другой части документа (но не в текущей). Например:

В настоящем разделе создание макрокоманд не рассматривается. Язык макрокоманд описан в п. 13.

- *Справочные* — сообщающие о наличии в документе других сведений по той же или близкой тематике. Например:

В программе предусмотрены также возможности экспорта данных в другие форматы. Об этом подробнее см. п. 14.1.

Опорные ссылки обычно используются там, где возникает проблема логического повтора или где для решения текущей пользовательской задачи необходимо предварительно решить другую. В любом случае мы рекомендуем строго придерживаться *принципа последовательного изложения*: если читатель знакомится с документом подряд, то на каждом этапе чтения и усвоения материала он располагает всеми необходимыми сведениями. Это означает, что если он читает, условно говоря, пятую главу, ему не могут понадобиться для ее понимания сведения, содержащиеся в шестой или седьмой. Иными словами, опорные ссылки могут вести только назад, но никак не вперед. Они должны указывать на пройденный уже материал, обращение к которому может понадобиться читателю лишь затем, чтобы вспомнить подзабытые детали и нюансы.

Навигационные ссылки используются в больших документах, где предусмотрены разделы или подразделы обзорного характера, позволяющие читателю лучше сориентироваться в изложении.

Ограничительные ссылки используются там, где у читателя могут возникнуть неверные ожидания относительно текущей темы. Часто в той или иной части документа затрагиваются вопросы, которые целиком освещаются в другой части документа. Необходимо пояснить читателю, где он может найти исчерпывающую информацию по затронутому вопросу, для чего и используется ограничительная ссылка.

Справочные ссылки используются в тех случаях, когда автор документа считает нужным указать читателю, где еще в документе рассматриваются темы, близкие к текущей. Такие ссылки часто сопровождаются краткими поясняющими выражениями вроде «см. также». Справочные ссылки в принципе можно опустить: пониманию текста это не мешает. Однако умело подобранная справочная ссылка позволит читателю лучше понять общую логику структуры программного продукта, выявить общие приемы работы с ним.

Перекрестных ссылок не должно быть слишком много, поскольку их избыток мешает последовательному восприятию текста. Поэтому следует использовать этот инструмент только тогда, когда он действительно необходим читателю или хотя бы небесполезен для него.

Глава 4

Слова и формулировки

4.1. Словарный запас технического писателя

4.1.1. Терминология

4.1.1.1. Виды терминологии

В документации пользователя программного продукта широко употребляется *терминология*. Собственно говоря, это объясняется тем, что в документации пользователя необходима очень высокая точность формулировок, а это, в свою очередь, предполагает высокий уровень терминологизированности. Любое специализированное понятие должно быть обозначено термином.

В документации пользователя употребляется как *компьютерная терминология*, так и *терминология предметной области*, которую обслуживает документируемый программный продукт²¹.

²¹ Строго говоря, упорядоченное употребление терминологии сотрудниками чрезвычайно важно для эффективной деятельности любого предприятия или организации. Это относится не только к документации пользователя, а к любым видам письменной и устной коммуникации, встречающимся в корпоративной практике, — к рекламным текстам, служебной переписке, текстовым фрагментам в пользовательском интерфейсе, к общению сотрудников между собой, к ответам специалистов службы технической поддержки на вопросы пользователей. Терминология присутствует всюду, а поэтому проблемы ее упорядочения не должны и не могут ложиться на плечи тех, кто занят документированием: во-первых, для этого нужны более широкие полномочия, чем те, которыми располагает технический писатель, во-вторых, необходимо решать не только проблемы, связанные с документированием, но и совершенно иного порядка — эргономические, маркетинговые, юридические. В современных крупных компаниях все чаще создаются особые терминологические службы — для управления терми-

Компьютерная терминология, употребляемая в документации пользователя, должна по возможности соответствовать общепользовательской. Пользователь, как нам уже приходилось отмечать, чаще всего не является специалистом по информационным технологиям и в своем понимании терминологии ориентируется на документацию, с которой ему уже доводилось знакомиться. В случае если в документации используются специализированные термины, значения которых может оказаться непонятным пользователю, следует принять меры: либо предусмотреть специальный раздел, в котором разъяснить смысл основных понятий, либо отказаться от изложения сведений слишком специального характера, либо попытаться изложить те же сведения без использования малопонятной терминологии (о требовании доступности сведений для восприятия см. п. 2.4).

Терминология предметной области употребляется в соответствии с нормами, принятыми в литературе, посвященной этой области, а также в соответствии с принятой в профессиональном сообществе практикой.

Терминология предметной области обязательно должна обсуждаться с экспертами, причем выбор экспертов лучше всего согласовать с заказчиком, чтобы — если в дальнейшем возникнет спор о терминах — ссылка на мнение «специалистов» была для него солидным аргументом.

При использовании терминологии предметной области перед техническим писателем часто возникают следующие проблемы:

- существуют разногласия между экспертами по поводу значений того или иного термина или по поводу термина для обозначения того или иного понятия;

ногие (terminology management) в масштабе всей компании. Существование терминологической службы сильно облегчило бы работу технических писателей, но, к сожалению, в отечественных компаниях редко встречается компетентный и современный подход к управлению терминологией. Техническому писателю приходится самостоятельно решать некоторые терминологические проблемы (см. об этом ниже), но это не значит, что он в состоянии «заодно» решить их все в масштабе всей компании. Создание терминологической службы и управление терминологией в компании — дело сложное, требующее специальных навыков, технических средств и организационных решений; в этой книге подобные вопросы не рассматриваются.

- в интерфейсе программного продукта используется иная терминология, чем утвержденная экспертами как общепринятая;
- термин предметной области случайно совпадает с компьютерным термином или иным словом, постоянно используемым в документации и обозначающим определенное понятие.

Все перечисленные проблемы необходимо выявить и обсудить на возможно более раннем этапе работы. Никакого универсального решения здесь не существует, поэтому в каждом конкретном случае придется подбирать оптимальный способ разрешения терминологической коллизии.

Если речь идет просто о различных мнениях по поводу терминологии, надо постараться выявить, какое употребление наиболее соответствует профессиональной практике потенциальных пользователей. Следует всегда помнить, что пользователь не принимает участие в дискуссиях, а воспринимает тот или иной термин так, как он привык. Поэтому даже если есть основания полагать, что термин не слишком удачный или должен употребляться в другом значении, — не стоит на этапе работы над документацией бороться со сложившимся употреблением.

Если разработчик пользовательского интерфейса, в отличие от технического писателя, не дал себе труда согласовать терминологию с экспертами и используемые им названия полей, кнопок, флажков и т. д. противоречат принятой терминологии, имеет смысл попытаться добиться изменения некорректных названий на более правильные. Если это по тем или иным причинам невозможно, с этим приходится смириться, однако изложение необходимо строить так, чтобы не возникало недоразумений.

Пример. Если в программе допускается ввод названия населенного пункта из официального классификатора городов, а в интерфейсе рядом с полем **Город** помещена кнопка **Из словаря** (то есть классификатор неудачно назван словарем), то название кнопки, если уж его нельзя скорректировать, в документации лучше всего сопровождать пояснением:

При затруднениях с вводом названия города нажмите на кнопку **Из словаря** (выбор названия из классификатора) и в открывшемся списке выберите название.

Если термин предметной области совпал со словом, которое уже используется в каком-то значении (например, компьютерным термином), приоритет по возможности отдается предметной области. К счастью, компьютерные термины зачастую имеют синонимичные варианты.

Пример. Мы рекомендуем использовать для обозначения структурного элемента файловой системы термин «каталог», но если этим термином уже обозначен, например, библиотечный каталог (в программе библиотечного учета), вместо компьютерного термина «каталог» разумнее использовать термин «папка».

В документации пользователя следует проводить четкую границу между профессиональным жаргоном и строгой терминологией. Это касается как терминологии предметной области, так и, в особенности, компьютерной терминологии. Бурное развитие информационных технологий привело к тому, что во многих областях не успела утвердиться русскоязычная терминология, зато произошло стихийное усвоение англоязычных заимствований; многие из таких заимствований, иногда нарочито искаженных, становятся жаргонными словами. Жаргон недопустим не только и даже не столько потому, что его употребление идет вразрез с нормами культуры речи. Дело в том, что значение термина строго определено, причем, как правило, существуют источники: специальная литература, технические спецификации программных продуктов и стандартов, — где можно ознакомиться с определением термина. Жаргонные слова удобны для употребления, когда разговор идет между понимающими друг друга с полуслова коллегами, причем можно переспросить собеседника, уточнить смысл сказанного им.

Документация пользователя адресована людям, которые могут и не понять пересыпанной жаргонизмами речи или понять сказанное не так, как должно. Вот почему мы настоятельно рекомендуем не только самим воздерживаться от жаргона, но и, по возможности,

не давать разработчикам и экспертам навязать документации профессиональный жаргон вместо терминологии. Речь идет не только пресловутых «багах», «фичах» и «прогах», но и, например, о выражениях типа «сервер упадет» или «программа подвесит компьютер». Приведенные примеры, кстати, показывают, что жаргонная речь — это обиходная речь профессионалов, которые не просто наблюдают работу программы, но и постоянно высказывают догадки о причинах того или иного течения дел. В документации пользователя необходимо дать точное и терминологически строгое описание того, что произойдет (может произойти) в том или ином случае: «сервер перестанет отвечать на запросы клиентов»; «для работы программы может потребоваться слишком много ресурсов, что приведет к сбоям в работе системы и потребует ее перезагрузки». Возможно, это несколько сложнее и более громоздко, но именно это мы понимаем под терминологической строгостью. И именно этого от нас в конечном итоге ожидает пользователь.

4.1.1.2. Терминология программного продукта

Иногда для программного продукта приходится разрабатывать особую терминологию. В первую очередь это касается объектов, с которыми ведется работа: для них чаще всего используются термины «объект» (предпочтительно с поясняющими словами: «географический объект», «объект охраны», «объект учета» и т. д.), «элемент» (предпочтительно с поясняющими словами: «элемент чертежа», «типовой элемент» и т. д.), «документ» (иногда также с поясняющими словами: «текстовый документ», «документ формата...» и т. д.).

Нередко возникает потребность в специальных терминах для следующих понятий:

- модели, по которым порождаются новые объекты, например: «шаблон», «образец», «модель»;
- категории объектов, выделяемые на тех или иных общих основаниях, например: «класс», «тип», «вид»;
- действия, выполняемые пользователем с помощью программного продукта, например: «редактировать», «просматривать», «обрабатывать», «вести учет», «управлять».

Терминологию программного продукта нельзя обходить вниманием. Пользователь сталкивается с документацией в целом; ему необходим терминологически строгий рассказ о работе с программой. Если терминология программного продукта не продумана, в документации одно и то же понятие обозначается то одним, то другим способом.

Пример. Если в программе предусмотрена категоризация объектов, то необходимо сразу решить, как будет называться категория объектов: «класс», «тип», «вид», как-либо иначе²². В противном случае в документации будут встречаться то выражение «класс объектов», то выражение «объект того же типа», то «...другие виды объектов»; пользователь, сбивтый с толку, не сразу поймет, что речь идет об одной и той же категоризации.

Предлагая специфическую терминологию для программного продукта, следует избегать двух нежелательных коллизий: с одной стороны, пересечения этой терминологии с общепринятой терминологией предметной области или области информационных тех-

²² Слова «класс», «тип» и «вид» используются как синонимы, однако мы рекомендуем следующее распределение оттенков их значений. Слово «тип» целесообразно использовать для внутренней, присущей механизмам программного продукта классификации объектов, как правило, имеющей отражение в пользовательском интерфейсе. Например:

Чтобы создать объект того или иного типа, в меню **Объект** выберите название этого типа; так, чтобы создать объект типа **Счет-фактура**, в меню **Объект** выберите пункт **Счет-фактура**.

Слово «вид» можно использовать для дополнительной классификации. Например:

Объект всегда принадлежит к одному из следующих основных видов: документы; отчеты; формы. Так, объект типа **Счет-фактура** всегда имеет вид **Документ**.

Слово «класс» рекомендуется использовать для классификации объектов в зависимости от их качеств, возможностей, сложности обработки и т. д. Например:

Программы этого класса позволяют обрабатывать только растровые (но не векторные) изображения. Задачи этого класса требуют для своего решения использования встроенного языка программирования.

нологий, с другой — выбора в качестве специфических терминов широко употребляемой околотерминологической лексики: «объект», «элемент» и т. п. На второй коллизии следует остановиться более подробно. Недостаток упомянутых околотерминологических слов в том, что они имеют слишком отвлеченный характер, а специфический для данного продукта термин должен обозначать более чем конкретное понятие. Слово «объект» принадлежит к числу отвлеченно-описательных слов (см. п. 4.1.2.3) и используется для того, чтобы в самом общем виде охарактеризовать то, что впоследствии будет определено в более конкретных терминах.

Пример. Так, говорят: «Объектом обработки в программе является...» — или: «Объектом учета в программе является...» — и далее приводится термин, обозначающий этот объект. Если он сам по себе называется объектом (особенно если при нем нет поясняющего слова), такая фраза выглядит невятной. Поэтому следует либо выбирать более конкретный термин, либо запретить себе формулировки типа «Объектом обработки является...»; вместо этого используются, например, формулировки:

Обработке в программе подлежат географические объекты.

Или:

Данные, учет которых ведется в программе, объединяются и хранятся в виде объектов учета.

4.1.1.3. Ввод термина в обращение в документе

Термин может вводиться в обращение различными способами (таблица 4).

По факту употребления обычно вводятся общеизвестные термины, понимание которых профильным пользователем не вызывает никаких сомнений.

Таблица 4. Способы ввода термина в обращение

Способ	Применение
По факту употребления	Термин употребляется в тексте документа в расчете на то, что пользователь опознает его при первом же употреблении как знакомый или общепонятный. В этом случае термин, по сути дела, специальным образом не вводится
По контексту	Для первого употребления термина подбирается контекст, в котором смысл термина становится более или менее понятен; термин рекомендуется выделять в тексте, предпочтительно курсивом. Например: В зависимости от физической природы, объект относится к одному из трех <i>типов</i>... В этом случае внимание читателя привлекается к факту употребления нового термина, причем через контекст дается общее представление о смысле термина, хотя строгое определение не дается
Посредством определения	При первом употреблении термина его значению дается формальное определение. Определения, даваемые в документации пользователя, должны иметь строгий стиль, по возможности унифицированный. Например: <i>Контактом</i> называется запись в базе данных, содержащая сведения о физическом лице или организации, с которыми осуществляется взаимодействие. Термин вводится явным образом
Посредством определения в специальном разделе в начале документа	Определение термина дается в разделе «Термины и определения» или аналогичном разделе. Термин вводится явным образом, причем внимание пользователя с самого начала привлекается к вводимому термину и обозначаемому им понятию; подчеркивается, что понятие является одним из центральных для дальнейшего изложения

По контексту вводятся термины, значение которых нет необходимости определять точно, читателю лишь необходимо примерно представлять себе, какого рода понятие обозначается данным термином, как и в каких ситуациях упоминается, как связано с другими понятиями. Документация пользователя не является ни учебником, ни научной монографией; для понимания практических сведений о целевом применении программы часто достаточно приблизительных представлений о значении тех или иных терминов.

Иногда в основе функционирования программы лежит достаточно сложная концепция, систематическое изложение которой потребовало бы слишком много места и едва ли необходимо в документации пользователя. Однако терминология, связанная с этой концепцией, не может вводиться просто по факту употребления — нужны минимальные пояснения. В этом случае лучшим решением может стать компактное введение в концепцию, состоящее не из явных определений и тезисов, а из поясняющих контекстов.

Пример. Вместо систематического изложения, включающего в себя строгие определения понятий «сервер», «клиент» и др., приводится следующий текст:

Система состоит из *сервера* и одного или нескольких *клиентов*. Сервер устанавливается на том же компьютере, что и центральная база данных (*компьютере-сервере*), клиенты — на *пользовательских компьютерах*, подключенных к той же локальной сети, что и компьютер-сервер. Пользователь работает с установленным на его компьютере клиентом, в результате чего в клиенте формируются *запросы* и направляются серверу. Сервер отвечает на эти запросы, направляя информацию клиенту. В результате пользователь в ходе работы на своем компьютере получает оперативный доступ к информации из центральной базы данных. Внешне этот доступ выглядит так же, как и доступ к информации, хранящейся на пользовательском компьютере; важно иметь в виду, что при отключении пользовательского компьютера от локальной сети или при выключении компьютера-сервера доступ к центральной базе данных невозможен.

Пользователь, для которого важна практическая работа с информацией, получает лишь самое общее представление о клиент-серверной архитектуре — в объеме, необходимом для понимания происходящего на рабочем месте во время сеанса работы с программным продуктом.

Посредством определения вводятся те термины, которые должны быть, по мысли автора документа, интерпретированы абсолютно точно, без каких-либо разночтений. Скажем, в программе предлагается ввести значение какого-либо сложного параметра, а это, в свою очередь, требует строгого понимания его смысла.

При этом следует помнить, что центральная задача документации все же состоит не в просвещении пользователя, а в сообщении ему сведений, необходимых для эксплуатации программы. Строгое определение термина вполне можно несколько упростить, в случае если этого упрощенного понимания достаточно для усвоения содержания документации. При этом определение рекомендуется снабдить примечанием, в котором будет указано, что в документации дано упрощенное определение и что более строгое и полное определение следует искать в специальной литературе.

Посредством определения в специальном разделе в начале документа вводятся те термины, которые связаны со спецификой документируемого программного продукта, играют ведущую роль в документации пользователя и необходимы для понимания на протяжении всего изложения.

Пример. Программа представляет собой электронный органайзер, в котором фиксируются контакты с физическими или юридическими лицами, а также назначаются встречи. В этом случае в раздел «Термины и определения» следует включить определения терминов «лицо», «организация», «контакт», «встреча».

Термины предметной области, если они не очевидны, чаще вводятся посредством определения, термины из области информационных технологий — по контексту. Это связано с тем, что в предметной области пользователю необходимо скорее уточнение границ между понятиями, тогда как в области информационных

технологий — примерное понимание смысла понятий. Впрочем, это правило отнюдь не имеет всеобщего характера.

Термины, специфические для данного программного продукта, предпочтительнее вводить посредством определения в специальном разделе в начале документа.

4.1.1.4. Термины-синонимы и термины-омонимы

Очень часто оказывается, что у одного понятия имеются конкурирующие терминологические обозначения — *термины-синонимы*. Причины этого многообразны, но сводятся к одному и тому же: разные авторы, институты, корпорации и т. д. в разное время или одновременно пришли к необходимости подыскать термин для данного понятия. Из этого следует вывод: чем важнее и емче понятие, тем выше вероятность того, что за это понятие конкурируют термины-синонимы.

В документации пользователя, вообще говоря, не следует допускать одновременного употребления терминов-синонимов. Однако в некоторых случаях возникает необходимость обозначать одно и то же понятие, так сказать, в разных проекциях.

Пример. В программе складского учета один и тот же объект называется *записью* (полностью — *записью о товаре в базе данных*), когда необходимо охарактеризовать его с информационной точки зрения, и *товаром*, когда важно указать на его содержательную природу. В этих случаях термины-синонимы допускаются, но их синонимиям надлежит явно оговорить.

В дальнейшем мы будем называть запись о товаре в базе данных товаром всюду, где это не приводит к смешению реальных предметов и их образов в базе данных.

Очень часто также оказывается, что одним и тем же термином обозначаются разные понятия. Мы имеем в виду не ту распространенную ситуацию, когда определение термина несколько «плывет» и термин в одном случае обозначает не совсем то, что в другом (например, термин «раздел» может означать только

структурный элемент верхнего уровня или же любой структурный элемент вообще). Такое размывание в документации пользователя категорически недопустимо.

Однако случается так, что один и тот же термин независимо используется для разных и достаточно далеких понятий, причем они могут принадлежать к области информационных технологий, или к предметной области, или к разным областям.

Пример. Термин «каталог» используется для обозначения структурного элемента файловой системы и структурного элемента базы данных.

Пример. Термин «документ» используется для обозначения объекта делопроизводства в программе электронного документооборота и для обозначения объекта обработки текстового редактора.

Такие термины-близнецы называются *терминами-омонимами*. В документации пользователя, вообще говоря, не следует допускать употребления одного и того же термина в разных значениях. Если обнаруживаются термины-омонимы, следует подыскать одному из них замену.

Пример. В упомянутой выше ситуации с разными значениями термина «документ» возможна для второго значения (объект обработки текстового редактора) замена термина «документ» на термин «текст».

Если такую замену найти не удастся, необходимо явно оговорить омонимию терминов. Например:

В настоящей документации термин «документ» используется для обозначения объекта делопроизводства (обычно с атрибутами «входящий», «исходящий», «внутренний» или «организационно-распорядительный»). В разделах, посвященных текстовому редактору, термином «документ» обозначается любой текст, редактируемый и сохраняемый в файле как единое целое.

4.1.1.5. Полный и краткий варианты термина

Термин может состоять из одного слова: «файл», «каталог», «параметр» — или из нескольких: «база данных», «почтовый сервер», «форматированный текст». В последнем случае целесообразно, кроме полного варианта термина, использовать краткий вариант. Дело в том, что если каждый раз повторять полный вариант термина, это очень сильно загромождает изложение.

Существуют следующие способы образования кратких вариантов термина:

- усечение словосочетания до главного слова, например: «почтовый сервер — сервер», «база данных — база»;
- образование нового слова, иногда сложносокращенного, например: «географическая база — геобазы»;
- образование аббревиатур, например: «база данных — БД».

Предпочтение следует отдавать первым двум способам; широко применять в документации пользователя аббревиатуры не рекомендуется. Большое количество аббревиатур мешает восприятию текста, в особенности если читатель знакомится с документом не систематически, а выборочно.

Для каждого термина мы настоятельно рекомендуем ровно один краткий вариант. Если в документе упоминается географическая база, то пусть в дальнейшем она фигурирует только как «географическая база» или «геобазы». Если автор называет один и тот же объект то географической базой, то геобазой, то базой, читатель легко может запутаться.

4.1.1.6. Сочетаемость термина с другими словами и фигуры описания

Любой термин представляет собой, с точки зрения языка, обычное слово или словосочетание. Как и любое другое слово или словосочетание, он включается в связи с другими словами. Однако термин обозначает вполне конкретное понятие с четко определенными и достаточно узкими границами; поэтому его сочетаемость с другими словами имеет весьма ограниченный характер и определяется свойствами понятия.

Лучше всего это можно продемонстрировать на примере термина «файл». Этот термин закономерно образует следующие словосочетания: «открыть файл», «закрыть файл», «сохранить файл», «сохранить данные в файле», «копировать файл», «создать файл», «удалить файл», «файл находится в папке» «файл содержит», «файл имеет формат» и т. д. За каждым словосочетанием здесь стоит вполне определенная ситуация: с файлом совершаются действия, он вступает в отношения, ему приписываются атрибуты и т. д. Каждая из этих ситуаций характерна для файла, и значит, термину «файл» свойственно сочетаться подобным образом с этими или аналогичными словами. Поскольку мы придерживаемся принципа единообразия в описании, сходные ситуации описываются сходными словосочетаниями. Мы уже говорили о таких словосочетаниях, называя их фигурами описания (см. п. 3.5.2). Фигуры описания постоянно повторяются от одной ситуации к другой — идентичной или аналогичной.

Таким образом, сочетаемость термина с другими словами ограничивается его способностью образовывать принятые в текущей документации фигуры описания. Никакие другие словосочетания с участием данного термина не допускаются.

Эти ограничения могут иметь чисто формальную или формально-смысловую природу. Формальные ограничения связаны исключительно с требованием единообразия.

Пример. Вернемся к термину «файл». Можно говорить «сохранить файл», а можно — «записать файл». Выражение «записать файл на диск» ничем не хуже выражения «сохранить файл на диске», оно отражает (насколько это возможно на уровне сочетания из двух слов) правильное понимание взаимоотношений между объектами, однако если выражение «сохранить файл на диске» (для обозначения операции сохранения файла на диске) уже принято в качестве фигуры описания, выражение «записать файл на диск» в этом же значении не употребляется. Никто, впрочем, не мешает поступить противоположным образом, приняв выражение «записать файл на диск» как фигуру описания (для обозначения операции сохранения файла на диске), а выражение «сохранить файл на диске» в этом же значении изъять из обращения.

Следует иметь в виду, что различные выражения не могут использоваться для обозначения одной и той же ситуации, но могут — для обозначения близких ситуаций.

Пример. В документации принимается фигура описания «задать значение» для любой операции присвоения параметру того или иного значения. Это означает, что использовать для обозначения той же операции выражения «ввести значение» некорректно. Однако вполне допустимо использовать фигуру описания «ввести значение» для обозначения частного случая присвоения параметру значения — ввода значения с клавиатуры:

Задайте значение частоты в поле **Частота**. Для этого введите значение или выберите его в раскрывающемся списке.

Формально-смысловые ограничения связаны не только с требованием единообразия, но и с некорректностью определенных выражений.

Пример. Выражение «просмотреть файл (файлы)» широко используется в обиходной речи людей, работающих с компьютером. Однако это выражение не вполне корректно и может вызвать путаницу: «просмотреть файл» означает просмотреть данные, хранящиеся в файле, тогда как «просмотреть файлы» может означать просмотреть имена файлов, отображаемое в списке. В устной речи всегда можно переспросить и уточнить, в письменной же речи требуется повышенная точность, так что можно рекомендовать использовать фигуры описания «просмотреть содержимое файла (файлов)» для обозначения просмотра данных, хранящихся в файле, и «просмотреть список файлов» для обозначения просмотра имен файлов, отображаемых в списке. Выражение «просмотреть файл (файлы)» не стоит не только использовать в указанных значениях, его вообще не следует выбирать в качестве фигуры описания, так как оно неточно.

Для каждого термина должен быть предусмотрен список фигур описания, в которые этот термин включен. Эти фигуры описания могут быть терминологическими или унифицирующими (см. п. 3.5.2). За пределами фигур описания термин использоваться не должен.

Конечно, тогда как терминологические фигуры описания заданы терминологической системой как таковой, унифицирующие фигуры описания с участием того или иного термина, как правило, выявляются только в ходе работы над текстом документа.

Пример. Некий файл является слишком объемным по меркам той или иной документируемой функции. В коллекции уже имеется терминологическая фигура описания «файл имеет объем ... байт (килобайт, мегабайт и т. д.)». Для обозначения новой ситуации вводится унифицирующая фигура описания «файл имеет слишком большой объем», после чего в аналогичных ситуациях всегда используется именно эта фигура описания, а не сходные по смыслу выражения «файл является слишком объемным», «файл слишком велик» и пр.

Разумеется, когда мы говорим, что то или иное словосочетание принято как фигура описания, мы имеем в виду, что будет использоваться не только это словосочетание, но и его естественные синтаксические трансформации.

Пример. Если у нас есть фигура описания «сохранить файл на диске», то допустимы формулировки: «сохраните файл на диске», «сохранение файла на диске», «файл, сохраненный на диске», «рекомендуется время от времени сохранять файл на диске» и т. п., — но не допускаются: «запишите файл на диске», «сохраните файл на диск», «файл, помещенный на диск».

Употребление фигур описания отнюдь не гарантирует корректности тех или иных высказываний. Помимо того что автор документа может сделать утверждение, не соответствующее действительности, он также может перепутать фигуры описания и

использовать фигуру описания не в той ситуации, для которой она предназначена.

Пример. Часто приходится сталкиваться со смешением фигур описания «сохранить файл» и «сохранить что-л. в файле». Первая из них используется в том случае, когда файл открывается для редактирования его содержимого, а затем должен быть сохранен с учетом сделанных изменений:

Откройте текстовый файл настроек, измените значения настроек, сохраните файл и закройте его.

Вторая же используется в том случае, когда ведется работа с какими-то данными, а затем эти данные, до того хранившиеся в оперативной памяти или в каких-то внутренних контейнерах программы, должны быть сохранены в отдельном файле:

По завершении работы над рисунком сохраните результаты работы в файле.

Часто эти ситуации путаются; например, в последнем случае пишут:

По завершении работы над рисунком сохраните файл.

Хотя никакого файла до сих пор не существовало.

Неточность, указанная в последнем примере, может показаться не слишком существенной, но она стирает границы между двумя разными подходами: в одном файл существует изначально, как некая первичная форма существования данных, в другом данные существуют до всяких файлов, а файлы используются для их сохранения с целью последующего использования. В разных программных продуктах используется тот или другой подход, и стирать границу между ними не стоит.

4.1.2. Вспомогательная терминология

4.1.2.1. Понятие вспомогательной терминологии

Терминология и связанные с ней проблемы отнюдь не обделены вниманием теоретиков и практиков. В то же время существует обширная группа слов, которую мы условно называем *вспомогательной терминологией* и которая остается на периферии внимания специалистов. Вспомогательные термины, как правило, вводятся по факту употребления (см. п. 4.1.1.3) и, в отличие от других терминов, принципиально рассчитаны на интуитивное понимание пользователем. К вспомогательным терминам относятся следующие группы слов и словосочетаний:

- обозначения элементов пользовательского интерфейса и действий с ними;
- отвлеченно-описательные слова и выражения;
- условные и обобщающие наименования.

Вспомогательные термины образуют общую систему понятий, которая используется для описания работы с любыми программными продуктами. Фактически из них состоит общий язык, который должен существовать между автором документа и его читателем еще до того, как последний раскроет документ. Автор вынужден исходить из того, что этот язык в значительной мере усвоен читателем по предшествующему опыту чтения документации, технической, научной и учебной литературы.

4.1.2.2. Обозначения элементов пользовательского интерфейса и действий с ними

В большинстве случаев современные программы имеют более или менее унифицированный графический интерфейс. Это означает, что, сталкиваясь с новой программой, пользователь уже знаком с такими элементами пользовательского интерфейса, как окно, кнопка, поле, раскрывающийся список и т. д., а также с такими действиями, как нажать на кнопку, ввести значение в поле, выбрать значение в раскрывающемся списке и т. д.

Следовательно, он знаком также и с терминами, обозначающими эти элементы и действия с ними, по крайней мере с некоторыми.

В некоторых случаях в профессиональной среде существуют разногласия относительно этой терминологии: один и тот же элемент интерфейса разными авторами (действующими по своему усмотрению или по указаниям какого-либо стандарта) называется «раскрывающийся список» или «выпадающий список»; «флажок» или «переключатель»; «закладка» или «вкладка». Одно и то же действие называется «нажать на кнопку», «нажать кнопку» или «щелкнуть кнопку»; «выбрать в списке» или «выбрать из списка»; «выбрать закладку» или «переключиться на закладку». Однако это практически не мешает пользователю, знакомящемуся с разными образцами документации, понимать, о каком действии идет речь. Дело в том, что современный интерфейс основан на небольшом количестве более или менее естественных операций с мышью или клавиатурой. Пользователь осваивает эти операции на стадии обучения основам компьютерной грамотности. Вспомогательная же терминология, используемая для обозначения элементов интерфейса и действий с ними, основана на простейших образных выражениях вроде: «кнопка» (внешне похожий на кнопку элемент интерфейса, на который можно «нажать» с помощью мыши); «выбрать» (подвести к объекту указатель мыши и щелкнуть левой клавишей мыши); «установить/снять флажок» (подвести указатель мыши к квадратной области «флажка» и щелкнуть левой клавишей мыши так, чтобы в квадратной области появилась/исчезла галочка). Именно в силу очевидности и органичности для человека этих образов, а также постоянного повторения одних и тех же действий пользователь легко понимает, о каких действиях идет речь, даже в тех случаях, если он не знаком с тем или иным вспомогательным термином.

Пример. Хорошо знакомый каждому пользователю элемент интерфейса, предоставляющий возможность выбирать один из нескольких вариантов, может обозначаться по-разному: «радиокнопка», «переключатель», «поле многозначного выбора». Но какой бы из этих терминов ни выбрал автор, читатель — отчасти по контексту, отчасти по опыту, отчасти на основе прилагаемой иллюстрации — моментально опознает

знакомый ему элемент интерфейса и уже после двух-трех словоупотреблений привыкает к тому, что этот элемент интерфейса в данном документе (шире: данном комплекте документации, документах, подготовленных в данной компании и т. д.) называется именно так.

Чтобы эта система терминологических обозначений была удобна для читателя, обеспечивала ему быстрое, верное и точное понимание любых инструкций и описаний, касающихся действий в интерфейсе, должны быть соблюдены два условия:

- термины, обозначающие элементы интерфейса и действия с ними, должны быть основаны на естественных образах, интуитивно понятных пользователю;
- употребление терминов должно быть унифицировано в пределах всего комплекта документации и тем более в пределах одного документа.

Первое условие объясняется тем, что, хотя читатель рано или поздно усвоит любую терминологию, ему будет гораздо удобнее, если он будет с ходу понимать образы, стоящие за терминами. Например, термин «кнопка» понятен не только потому, что пользователь уже сталкивался с ним, но и потому, что сравнение элемента интерфейса с реальной кнопкой достаточно очевидно. Именно поэтому выражение «щелкнуть кнопку» представляется нам менее удачным, чем «нажать на кнопку»: ведь реальную кнопку именно нажимают, а не щелкают по ней. Или например, рассмотрим описанную выше проблему выбора — как назвать элемент интерфейса: «радиокнопка», «переключатель» или «поле многозначного выбора». Термин «радиокнопка» лучше всего подходит для тех пользователей, которые имели дело с техникой, оборудованной подобными кнопками: одновременно может быть нажата только одна кнопка из группы (скажем, на многопрограммном радиопроductоре). Однако в последнее время техника такого рода становится менее распространенной, поэтому и термин менее понятен; лучше обойтись более нейтральным «переключатель». Термин «поле многозначного выбора» хорошо понятен профессиональным программистами или специалистам по компьютерной технике.

Второе условие объясняется тем, что читателю необходимо быть твердо уверенным в том, что он правильно понял очередную инструкцию. Каждое расхождение в обозначениях элемента интерфейса и/или действия с ним ставит читателя в тупик, заставляет задуматься: это тот же самый элемент интерфейса (действие) или другой (другое)?

Пример. В документе встречается выражение «раскрывающийся список», а тут же по соседству — выражение «выпадающий список». Почему они названы по-разному, не различаются ли правила работы с ними? Или например, в одном случае сказано «щелкните по значку», а в другом — «щелкните на значке». Одно ли это действие, или же разные? Такие вопросы кажутся нелепыми, но они непременно возникнут; ведь означает же принципиально разные действия «щелкните...» и «дважды щелкните...», так, может быть, и в этих случаях есть значимое различие?

В следующую секунду читатель во всем разберется, но естественный, плавный ход чтения будет нарушен.

Расхождения в терминах заставляют читателя задумываться над тем, над чем задумываться не стоит, не дают возможности усвоить раз и навсегда выбранную систему терминов и тем самым мешают. Поэтому рекомендуется использовать унифицированную систему терминов для элементов интерфейса и действий с ними, то есть для каждой важной ситуации предусмотреть свое, раз и навсегда закрепленное выражение — фигуру описания.

В качестве вспомогательных терминов для элементов интерфейса не рекомендуется использовать заимствованные из английского языка термины «иконка», «кликнуть», «чекбокс» и др. Они не обладают необходимой наглядностью, ухудшают общее впечатление от документа. Вместо термина «иконка» предпочтительны термины «значок» или «пиктограмма», вместо термина «кликнуть» — термин «щелкнуть», вместо термина «чекбокс» — термин «флажок».

В качестве вспомогательных терминов для действий с элементами интерфейса не рекомендуется использовать слова и выражения, обозначающие различные аспекты мышления и восприятия

пользователя, а также его намерения и решения. Вообще говоря, в документации пользователя следует как можно меньше касаться мыслей, мнений, желаний, колебаний пользователя. В отдельных случаях технический писатель обращает внимание на необходимость обдумывания какого-либо шага, изучения ситуации, но делает это крайне тактично. Например:

Перед выполнением процедуры рекомендуется внимательно изучить ситуацию и принять решение.

В основном технический писатель затрагивает отнюдь не мысли, мнения, колебания и желания пользователя, но его намерения и решения. Для обозначения же намерений и решений пользователя используются либо специальные термины, либо отвлеченно-описательные слова и выражения (см. п. 4.1.2.3). Существенно, чтобы они не совпадали с обозначениями конкретных действий пользователя; в противном случае их будет трудно соединить в рамках одного высказывания, а такая потребность будет возникать постоянно, поскольку конкретные действия предпринимаются пользователем именно исходя из его намерений и решений.

Пример. Слово «выбрать» зачастую используется в одном документе в двух разных смыслах — и как отвлеченно-описательное слово, обозначающее решение пользователя относительно какого-либо объекта, и как вспомогательный термин, обозначающий действие пользователя в интерфейсе, обычно щелчком мыши. Это приводит к неуклюжим построениям вроде:

Чтобы выбрать объект для обработки, выберите его в списке.

Целесообразно заменить либо отвлеченно-описательное слово (вариант: «назначить»), либо обозначение конкретного действия (вариант: «указать»). Можно также использовать в качестве обозначения конкретного действия уточненный вариант того же слова:

Чтобы выбрать объект для обработки, щелчком мыши выберите его в списке.

Мы говорили о том, что вспомогательные термины обычно вводятся по факту употребления. Однако это не мешает техническому писателю поместить в начале документа перечень основных обозначений для элементов интерфейса и действий с ними. Такой перечень может понадобиться пользователю, не имеющему опыта работы с компьютером: он получит возможность обращаться к этому списку за справкой, если ему окажется непонятной инструкция или описание. Лучше всего выполнить такой перечень в виде таблицы: в первом столбце помещается изображение элемента интерфейса, во втором — термин для него, в третьем — термины, обозначающие действия с этим элементом интерфейса, и необходимые пояснения. Например:

	кнопка	<i>нажать на кнопку</i> — подвести указатель мыши к кнопке, нажать левую клавишу мыши и отпустить ее. <i>отжать кнопку</i> (если кнопка после нажатия остается в нажатом состоянии) — нажать на кнопку повторно.
---	--------	--

Не рекомендуется перегружать такими перечнями документ, если он адресован пользователям, имеющим — хотя бы и сравнительно небольшой — опыт работы с компьютером и с графическим интерфейсом.

4.1.2.3. Отвлеченно-описательные слова и выражения

Отвлеченно-описательные слова и выражения представляют собой самый прочный, «низовой» пласт технической культуры. В течение столетий развития техники постепенно сложился достаточно многочисленный арсенал языковых средств для описания самых общих объектов, явлений и процессов, имеющих отношение к техническому оборудованию.

Примеры. Чтобы сказать, что какой-то процесс просто имеет место, говорят:

Процесс протекает.

При описании действий оператора какой-либо машины говорят:

Оператор выполняет следующую последовательность действий.

Каждый читатель технического текста понимает значение глаголов «протекать» и «выполнять» без дополнительных пояснений.

С появлением компьютеров, а в особенности компьютерных технологий, предполагающих диалоговое взаимодействие пользователя и программного средства, возникает новая группа отвлеченно-описательных слов и выражений, связанных со спецификой новой области; большое значение приобретают темы ввода и вывода данных, обработки этих данных в компьютере, отражения этих процессов на экране монитора и т. д.

Пример. При описании обработки данных программным продуктом говорят:

В программе выполняется обработка данных.

При описании изображения на экране монитора говорят:

В рабочей области отображается список объектов.

Смысл понятий «выполнять(ся)» и «отображать(ся)» опять-таки понимается читателем без пояснений.

Эти слова и выражения не являются в собственном смысле терминами: во-первых, они не нуждаются в определении (да и сложно давать строгие определения столь общим понятиям); во-вторых, они легко заменяются на синонимичные — можно заменить «выполняется» на «осуществляется» или «протекает» на «имеет место», смысл не пострадает. Читатель не задумывается над смыслом этих слов и выражений: они составляют тот «язык», который дан ему исходно, усвоен в процессе получения общего и/или специаль-

ного образования, закреплён чтением научной и технической литературы, а также других комплектов документации.

Отвлеченно-описательные слова и выражения используются для обозначения как предметов и явлений, так и действий и состояний. В табл. 5 перечисляются некоторые важные группы отвлеченно-описательных слов и выражений.

Таблица 5. Отвлеченно-описательные слова и выражения

Группа	Примеры
Предметы и явления	Программа оперирует следующими объектами. В состав системы входят следующие физические устройства. Главный контроллер выполняет следующие функции. На компонент возлагаются следующие задачи. Процесс решения задачи занимает некоторое время. Процедура формирования отчета описана в Приложении А. Пользователю доступны только те виды действий, которые указаны в пользовательском профиле
События, происходящие с параметрами и их значениями: присвоение значения, его изменение, отсутствие значения	Параметру <i>присваивается</i> новое значение. Значение параметра <i>увеличивается</i> (уменьшается). Когда значение параметра <i>достигает</i> 5000, выдается сообщение об ошибке. Если параметру <i>не присвоено</i> значение, работа программы невозможна
Стадии жизненного цикла объектов: возникновение, существование, завершение цикла	По умолчанию в системе <i>имеется</i> один объект с именем <i>По умолчанию</i>. Впоследствии в систему <i>добавляются</i> другие объекты: их создает пользователь либо они создаются автоматически. Если объект более не используется, он <i>исключается</i> из системной базы данных: его либо удаляют, либо переносят в архив
Характеристики действий и операций: выполнение/невыполнение, возможность/невозможность	Данная операция <i>выполняется</i> пользователем. Действия по настройке моделирования <i>доступны</i> пользователю только после задания входных параметров. Отмена данной процедуры <i>невозможна</i>

Группа	Примеры
Стадии протекания процессов: начало, течение, завершение	При загрузке операционной системы автоматически <i>запускается</i> процедура проверки оперативной и дисковой памяти на наличие вирусов. Пользователь имеет возможность <i>запустить</i> эту процедуру вручную в любой момент. Процесс моделирования <i>протекает</i> без сбоев, если значения параметров заданы корректно. Чтобы <i>приостановить</i> его, нажмите на кнопку Пауза, чтобы <i>прервать</i> — на кнопку Стоп. Если процесс моделирования не был <i>прерван</i> или <i>приостановлен</i>, он <i>завершается</i> автоматически, как только значение разности достигнет заданной величины
Действия с данными: ввод, вывод и обработка	Данные <i>вводятся</i> в базу данных следующими способами: с клавиатуры или путем импорта файла текстового формата. <i>Вывод</i> данных моделирования возможен непосредственно на экран монитора или на печать. <i>Обработка</i> данных выполняется автоматически в соответствии с выбранным алгоритмом
Показ на экране монитора образов реальных или виртуальных объектов, отображение процессов и действий пользователя	В результате действий пользователя на экране <i>появляется</i> окно поиска. Список объектов <i>отображается</i> в рабочей области окна. Удаление существующих и создание новых объектов <i>отражается</i> в списке объектов в рабочей области окна
Намерения и решения пользователя	Если пользователь <i>намерен</i> выполнить форматирование, он нажимает на кнопку Формат. Чтобы <i>выбрать</i> метод анализа данных, щелчком мыши укажите его в списке. Рекомендуется внимательно изучить данные, прежде чем <i>принять решение</i> относительно их дальнейшей обработки

Разумеется, перечислены далеко не все отвлеченно-описательные слова и выражения, которые могут употребляться в документации пользователя. Однако мы и не ставили такой цели. Важно, что используя какое-то отвлеченно-описательное слово или выражение, автор документа должен обратить на это внимание и в дальнейшем во всех аналогичных случаях использовать именно это слово (выражение), а не синонимичное.

Пример. Если автор уже принял решение говорить «...пользователь выполняет следующие операции...», то во всех случаях, когда говорится о действиях какого-либо лица, употребляется именно глагол «выполнять», а не «производить» или, к примеру, «осуществлять».

В принципе, разумеется, читатель поймет, о чем идет речь, если в одном случае будет сказано «пользователь выполняет следующие операции», а в другом «пользователь осуществляет следующие операции». Однако это может хотя бы на несколько секунд запутать читателя, сбить его с толку, помешать комфортно восприятию документации. Отвлеченно-описательные слова и выражения должны быть зафиксированы как последовательно употребляемые фигуры описания.

Следует помнить, что отвлеченно-описательные слова и выражения противостоят более конкретным формулировкам — терминологии предметной области, терминологии области информационных технологий, словам и выражениям, обозначающим действия пользователя в интерфейсе. Это противостояние вытекает из самой сущности документации пользователя: технический писатель отталкивается от самых общих категорий (передаваемых как раз отвлеченно-описательными словами и выражениями) и от них переходит к конкретным понятиям и предписаниям.

Пример. Рассмотрим фразу:

Чтобы выполнить форматирование, нажмите на кнопку **Формат**.

Здесь слово «выполните» — отвлеченно-описательное, выражение «нажмите на кнопку» — более конкретное, описывающее действия пользователя.

Другое предложение:

В части работы с объектами предусмотрены следующие возможности:

- создание нового объекта;
- редактирование свойств объекта;
- удаление объекта.

Здесь слово «возможности» — отвлеченно-описательное, а далее идет конкретное перечисление действий с объектом (каждое из действий обозначено термином).

Иначе говоря, изложение постоянно движется, как челнок, между отвлеченно-описательным и конкретно-практическим планами. Оно начинается с того, что формулируются основные задачи, решаемые программой: «Программа предусматривает решение следующих задач...» («предусматривает решение следующих задач» — отвлеченно-описательное выражение, за которым следует раскрытие в более конкретном ключе). Затем рассматривается одна из перечисленных задач: «Анализ данных требует следующих действий...» (здесь «требует следующих действий» — опять-таки отвлеченно-описательное выражение, за которым следует перечисление действий). Далее аналогичным образом раскрываются более частные аспекты решения задачи и т. д.

Чтобы подобное изложение было достаточно четким, крайне нежелательно, чтобы отвлеченно-описательные слова и выражения пересекались с терминами, с одной стороны, или обозначениями элементов пользовательского интерфейса и/или действий с ними.

Пример. Слово «запускать» часто используется как отвлеченно-описательное слово для действий пользователя, имеющих целью начать какой-либо процесс:

Запустите процесс форматирования.

Рекомендуется запустить антивирусную программу.

Однако если программный продукт, к примеру, обеспечивает управление ракетами-носителями, слово «запускать» приобретает статус термина предметной области и его уже лучше не употреблять в другой роли. В качестве отвлеченно-описательно-

го слова лучше использовать какой-нибудь синоним, например «инициировать». Или например, напомним описанную выше коллизию со словом «выбрать» в двух ролях одновременно:

Чтобы выбрать объект для обработки, выберите его в списке» (см. п. 4.1.2.2).

Таких формулировок лучше избегать, поскольку читатель может начать путать постановку задачи с ее решением. Предпочтительнее вариант:

Чтобы выбрать объект для обработки, щелчком мыши укажите его в списке.

Важно лишь проследить, чтобы обе фигуры описания употреблялись в документе (и в документации в целом) последовательно.

Иногда отвлеченно-описательные слова и выражения приводят к смешению разнородных понятий.

Пример. Слово «установить» употребляется как в отношении программы («установить программу на компьютере»), так и в отношении значений параметра («установите значение параметра равным 0»).

Пример. Слово «выполнять» употребляется как в отношении действий пользователя («пользователь выполняет следующие действия»), так и в отношении наличия у программы, устройства и т. п. определенного функционального назначения («модуль выполняет функции серверного компонента»).

Пример. Следует, по возможности, избегать подобного смешения смыслов, например: «задайте значение параметра равным 0»; «модуль реализует функции серверного компонента».

Подобные ситуации, вообще говоря, заставляют задуматься о том, насколько точны наши формулировки. Что, например, по-

нимается под формулировкой «модуль выполняет функции серверного компонента»? Что на модуль возлагаются функции серверного компонента? Что модуль наделен помимо прочих функций функциями серверного компонента? Наконец, что модуль просто реализует именно эти функции в архитектуре системы? Необходимо уточнение, а оно неизбежно приведет к повышению строгости изложения и к устранению смещения смыслов.

Отвлеченно-описательные слова и выражения часто ускользают от внимания технического писателя: терминами они не являются, все их как будто бы понимают правильно, никто не жалуется на разнობой в употреблении.

Мы считаем важным привлечь внимание к этой обделенной вниманием теме, потому что считаем: очень часто общее впечатление от текста как от невнятного, запутанного, тяжелого для восприятия объясняется хаосом в употреблении отвлеченно-описательных слов и выражений. Исправление этого недостатка чрезвычайно важно для улучшения восприятия документации и в конечном итоге — для более эффективного ее использования.

4.1.2.4. Условные и обобщающие наименования

В документации пользователя широко используются *условные* и *обобщающие наименования*.

Условное наименование, как правило, подбирается для документируемого программного продукта. Поскольку программный продукт часто имеет длинное и сложное для запоминания официальное название, проще всего договориться с самого начала, как его называть для краткости: если он имеет сравнительно узкий функциональный профиль — *программой*, если специализирован для решения небольшого числа прикладных задач — *приложением*, если состоит из нескольких компонентов — *программным комплексом*, в определенных случаях — *системой* и т. д.

Некоторые технические писатели и авторы стандартов организаций предпочитают употреблять для обозначения документируемого продукта аббревиатуру или краткий вариант его официального названия.

Пример. Интегрированный комплекс управления продажами может сокращенно называться ИКУП; многофункциональная система обеспечения движения ТРАНСПОРТ — система ТРАНСПОРТ.

Что касается аббревиатур, мы вообще не рекомендуем их использовать (за исключением общепринятых). Аббревиатуры плохо встраиваются в русский текст, так как обычно не склоняются. Кроме того, следует поставить себя на место пользователя, каждый раз мучительно вспоминающего, что эта аббревиатура значит (не говоря уж о том, что он может вообще этого не знать — если, скажем, обратился к документу по случаю, за конкретными сведениями, а вводных глав, где вводится аббревиатура, просто не читал).

Краткий вариант названия в этом смысле несколько удобнее аббревиатуры, но мы, тем не менее, не рекомендуем повторять его каждый раз, когда необходимо упомянуть документируемый программный продукт. Появление имени собственно создает дополнительное напряжение для читателя, отвлекает его. Кроме того, может так получиться, что в документе необходимо упомянуть другие программные продукты с похожими названиями.

Пример. Кроме системы ТРАНСПОРТ в документе упоминается еще комплекс мониторинга ГОРТРАНС. Если они оба названы краткими вариантами названия, читателю придется каждый раз, встречая их рядом, вспоминать, какой из них обозначает документируемый продукт, а какой — нет.

Предлагаемое нами решение состоит в следующем: в самом начале документа договориться об условном наименовании, а краткий вариант названия употреблять только в тех случаях, когда необходимо сделать какое-то важное, представляющее самостоятельную ценность замечание.

Пример. В рядовых случаях пишут:

Система оснащена возможностями импорта данных из файлов Microsoft Excel или Microsoft Access.

Но:

Внимание! В системе ТРАНСПОРТ не предусмотрено никакой автоматической верификации импортируемых данных. Рекомендуется выполнить проверку корректности импортируемых данных самостоятельно.

В случае если документ посвящен компоненту программного комплекса, возникает необходимость как-то обозначать и компонент, и программный комплекс в целом. Тут имеет смысл условным наименованием обозначать непосредственно документируемый программный продукт, а для обозначения программного комплекса в целом лучше употреблять краткий вариант официального названия. Например:

Приложение используется для мониторинга критических состояний системы. Функции мониторинга занимают ключевое место в общем перечне функций, реализуемых посредством комплекса КОНТРОЛЬ.

Аналогичным образом, если документ посвящен программному комплексу в целом, то наоборот, программный комплекс имеет смысл обозначать условным наименованием, а компонент — кратким вариантом официального названия.

Обобщающие наименования используются для обозначения некоторых групп однородных объектов и лиц (см. ниже), а также для обозначения произвольных представителей этих групп.

Пример. Если документируемый программный продукт взаимодействует с другими программами, то в качестве обобщающего наименования выступает словосочетание «другие программы»:

При необходимости программа передает данные для обработки другим программам. Программа может импортировать файлы того же формата, подготовленные в другой программе.

Посредством обобщающих наименований обозначаются следующие группы объектов и лиц:

- лица, взаимодействующие с документируемым программным продуктом;
- программные продукты, упоминаемые в документации пользователя;
- объекты, с которыми ведется работа с помощью документируемого программного продукта.

Подбор обобщающего наименования для лиц, взаимодействующих с документируемым программным продуктом, тривиален для однопользовательских программ, тогда как для многопользовательских программных продуктов и автоматизированных систем зачастую возникают затруднения. Обычно в качестве обобщающего наименования употребляется слово «пользователь». Например:

Все пользователи программы перед каждым сеансом работы в обязательном порядке проходят процедуру авторизации.

Это означает, что пользователем называется любой человек, взаимодействующий с программным продуктом, в том числе и администратор.

В этом случае возникает вопрос: как называть тех, кто использует программу непосредственно для решения своих практических (производственных, служебных и пр.) задач. Ведь иногда необходимо назвать неким обобщающим словом именно таких пользователей, исключая из этого состава администратора или администраторов.

Иногда для этого употребляется слово «оператор». Оно идеально, если речь идет о мониторинге технологических процессов, слежении за состоянием каких-либо устройств или датчиков и т. п. В том же случае, если речь идет, например, о программе управления документооборотом, слово не кажется удачным, так как программу используют и рядовые сотрудники, и руководители подразделений, и руководители организации, которых называть операторами вовсе уж не подобает. Каждый из них имеет

свой круг обязанностей, а сверх того каждый в определенной ситуации применяет программу. В таком случае необходимо придумать для «целевых» пользователей иное обобщающее наименование: «пользователь АРМ» (автоматизированного рабочего места), «обычный пользователь».

Возможно и другое решение: пользователем называть только «целевого» пользователя, а сотрудника, обслуживающего программу, — подходящим ему термином (чаще всего администратором). В случаях же, когда необходимо сформулировать некое положение относительно всех без исключения лиц, взаимодействующих с программой, то говорят, например:

Как пользователи, так и администраторы системы в обязательном порядке...

Какое из решений выбрать — ответ на этот вопрос зависит от конкретного программного продукта и от задач, которые в конкретном случае призван решить комплект документации и каждый отдельный документ. Нам кажется нежелательным лишь такое решение, при котором обобщающее слово «пользователь» то включает в себя всех лиц, взаимодействующих с программой, то только «целевых» пользователей. Такое смешение близких, но не тождественных понятий в строгом тексте недопустимо.

Программные продукты, упоминаемые в документации пользователя, обобщенно лучше всего называть просто: «другие программные продукты». Необходимо также предусмотреть ряд обобщающих наименований для отдельных групп программных средств. К ним, в частности, относятся: «другие программные продукты того же производителя», «другие программные продукты, использующие ту же технологию/тот же алгоритм» и т. д. Программные продукты, произведенные другими компаниями, нежели документируемый программный продукт, рекомендуется называть: «программные продукты сторонних производителей».

Объекты, с которыми ведется работа с помощью документируемого программного продукта, иногда нуждаются в дополнительном обобщающем наименовании.

Пример. Программа, используемая сотовым оператором, имеет дело со звонками, голосовыми сообщениями и текстовыми сообщениями. Звонки, голосовые сообщения и текстовые сообщения обрабатываются по-разному, но в документации пользователя встречаются и общие положения, касающиеся их всех. Необходимо обобщающее наименование, которым бы назывались и те, и другие, и третьи. Самое простое решение в таком случае состоит в том, чтобы назвать их универсальным словом «объект», однако это решение не слишком удачно (см. п. 4.1.2.2). От технического писателя зависит, какое слово или выражение будет выбрано на эту роль — «соединение», «контакт» или еще какое-то.

Условные и обобщающие наименования используются практически в каждом документе, причем употребляются достаточно широко. Не следует относиться легкомысленно к правильному выбору этих вспомогательных терминов. Эта проблема может показаться надуманной, но практически каждому приходилось размышлять над тем, какое обобщающее наименование выбрать для однородных, но достаточно разных объектов или какое условное наименование выбрать для документируемого продукта. Следует обратить внимание на то, чтобы условные или обобщающие наименования отвечали следующим требованиям:

- легко понимались читателем, соответствовали вкладываемому в них смыслу;
- не совпадали ни с какими терминами или отвлеченно-описательными словами.

Пример. Слово «оператор» может использоваться в качестве обобщающего слова для лиц, работающих с программным продуктом. Однако если «оператор» — термин, обозначающий одну из функциональных ролей, это слово уже не может быть обобщающим, а следовательно, надо найти другое обобщающее слово.

Пример. Слово «система» может использоваться в качестве условного наименования. Однако если в тексте постоянно упоминается какая-то еще система, хотя бы и с поясняющими словами, условное наименование лучше заменить.

Условные наименования оговариваются в самом начале документа. Например:

Интегрированный комплекс управления продажами (далее: комплекс) представляет собой...

Обобщающие наименования лиц, взаимодействующих с программой, вводятся по факту употребления, хотя если смысл вводимых наименований неочевиден, есть смысл пояснить их. Например:

Пользователь (под этим словом здесь и далее понимается любое лицо, взаимодействующее с системой) перед началом каждого сеанса работы с системой в обязательном порядке проходит авторизацию.

Обобщающие наименования программных средств очевидны и вводятся по факту употребления.

Обобщающее наименование объектов, с которыми ведется работа с помощью документируемого программного продукта, фактически представляет собой полноценный термин, вводимый техническим писателем в методических целях. Этот термин вводится по контексту или посредством определения.

4.1.3. Слова-«артикли»

Как известно, в русском языке, в отличие, например, от английского, отсутствуют артикли — служебные слова, с помощью которых говорящий или пишущий указывает, о каком предмете идет речь — о новом, упоминаемом впервые, произвольно взятом из общей совокупности таких же предметов, или о знакомом, вполне определенном, уже упоминавшемся выше или известном слушателю/читателю из других источников.

В тех языках, где они есть, артикли способствуют точности указания на предмет и облегчают понимание текста слушателем/читателем. Поэтому в техническом тексте артикли особенно удобны — они фактически представляют собой логические значки, понятные читателю и облегчающие ему восприятие тек-

ста. В русском техническом тексте артиклей, разумеется, нет, но очень часто авторы стремятся «восполнить» этот недостаток и используют в большом количестве слова и выражения «этот», «данный», «упомянутый», «описанный выше», «соответствующий» и т. п. Потребность в этих заменителях артиклей тем больше, что образцом для русскоязычной документации по историческим причинам является англоязычная документация, русскоязычные фигуры описания часто являются переводами с англоязычных аналогов, а в последних артикли играют определенную роль.

Наше отношение к этой практике трудно описать одной фразой. С одной стороны, совсем без таких «артиклей» обойтись невозможно. С другой стороны, нельзя одобрить и чрезмерное увлечение ими. Действительно, употребление «артиклей» чревато следующими недостатками:

- загромождает изложение (русские «артикли» гораздо более громоздки, чем настоящие артикли в английском, французском, немецком или каком-либо ином языке);
- создает иллюзию «уточненного» указания на предмет или явление при сохранении неопределенности.

Пример. В тексте говорится:

При передаче данных из компонента **Анализ** в компонент **Отчеты**, *данный* компонент автоматически формирует сводную таблицу. *Упомянутый* компонент используется также для выдачи справки о доходах и других документов для представления в налоговую инспекцию.

Слова «данный» и «упомянутый» только загромождают изложение, но отнюдь не способствуют большей ясности.

Прежде всего, перед тем как использовать в письменной речи «артикл», следует задуматься: а нельзя ли обойтись вовсе без уточняющих слов, которые самопроизвольно вставляются в текст по ходу работы над ним?

Пример. В тексте говорится:

Для очистки базы данных от неиспользуемых данных применяется функция оптимизации. *Данная* функция активизируется нажатием на кнопку **Оптимизация**.

Слово «данная» используется здесь для указания на предшествующее упоминание функции оптимизации, однако если его просто опустить, текст не станет менее прозрачным.

Если простым вычеркиванием «артиклей» толку не добиться, стоит попробовать перестроить текст более радикально. Если возникает потребность в использовании «артиклей», это означает, что логические связи между предложениями слишком запутаны и их следует выразить более удобными для восприятия средствами.

Пример. Исходно в тексте говорится:

Чтобы активизировать функцию оптимизации, нажмите на кнопку **Оптимизация**. Чтобы активизировать функцию архивации, нажмите на кнопку **Архив**. Чтобы активизировать функцию удаления всех данных из базы данных, нажмите на кнопку **Очистка**.

Технический писатель перечитывает свой текст и видит, что совершенно однотипные фразы, с заменой некоторых слов, повторяются несколько раз. Желая сделать изложение более лаконичным, он заменяет их на одну фразу:

Чтобы активизировать нужную функцию, нажмите на соответствующую кнопку.

Изложение стало более лаконичным, однако теперь читателю гораздо труднее его воспринимать: надо понять, какая функция в данный момент «нужная», обратиться к пользовательскому интерфейсу программы, найти, какая кнопка «соответствующая», и затем нажать ее. Короче говоря, пользователю предстоит целая аналитическая работа, не говоря уж о том, что ему придется для восполнения картины обратиться к интерфейсу, а это идет вразрез с нашими принципами (см. п. 2.3.4.5).

Между тем, гораздо лучше было бы полностью поменять структуру изложения и использовать другой способ изложения материала:

Функции и элементы интерфейса, позволяющие их активизировать, приведены в таблице 2.

Таблица 2. Функции обслуживания базы данных

Функция	Элемент интерфейса
Оптимизация хранения данных в базе данных	Кнопка Оптимизация
Архивация данных	Кнопка Архив
Удаление всех данных из базы данных	Кнопка Очистка

Структурированный текст гораздо проще для восприятия: в нем сразу видно главное, отсутствуют избыточные повторы, видны как общая логика интерфейсного решения, так и конкретные детали. Заметим, что формулировки при этом становятся лаконичнее, но текст порой несколько длиннее. Это закономерно: лаконичность формулировок важнее, чем краткость и сжатость текста в целом.

Если же все-таки приходится употреблять «артикли», следует упорядочить их употребление и, разумеется, унифицировать употребление артиклей в сходных ситуациях. Для этого следует разобраться, в каких именно ситуациях возникает необходимость в «артикле».

Артикли (в тех языках, где они существуют, например в английском) бывают неопределенными и определенными.

Потребность в неопределенном артикле в техническом тексте возникает, коль скоро надо подчеркнуть, что речь идет о произвольном экземпляре, взятом наугад, одном из многих. В английском языке для этого используется артикль «a/an», в русском же языке такого нет. В документации пользователя часто используют слова «нужный» или «необходимый», подчеркивая тем самым, что выбор экземпляра остается на усмотрение пользователя. Например:

Чтобы выбрать *нужный* товар, щелчком мыши укажите его в списке.

Эти слова не слишком удачно выполняют свою функцию: они отвлекают внимание читателя, заставляют задумываться над тем, насколько необходим ему тот или иной экземпляр и как это связано со смыслом высказывания и с его контекстом. Мы рекомендуем использовать в этом качестве выражение «тот или иной». Например:

Чтобы выбрать *тот или иной* товар, щелчком мыши укажите его в списке.

Кроме того, следует заметить, что, если по контексту выше не упоминается никакой конкретный экземпляр, можно обойтись вовсе без «артиклей». Например:

Чтобы выбрать товар, щелчком мыши укажите его в списке.

(в таком контексте слово «товар» будет воспринято именно как обозначение произвольного экземпляра, выбор которого осуществляется пользователем).

Может возникнуть желание использовать в качестве «артикла» слово «произвольный». Однако это слово слишком уж однозначное, оно неизбежно заставит задуматься: что значит «произвольный»? Идет ли речь о том, что пользователь может сделать выбор по своему произволу, или о том, что совершенно все равно, какой экземпляр он выберет? Последняя ситуация также встречается, хотя и не так часто.

Пример. Встречается такое интерфейсное решение, когда список объектов для последующей работы с ним надо активизировать, а для этого надо выбрать в списке объект, безразлично какой. Для таких случаев мы и предлагаем резервировать слово «произвольный»:

Чтобы сделать панель списка активной, щелчком мыши укажите в ней *произвольный* объект.

Потребность в определенном артикле в техническом тексте возникает, коль скоро надо подчеркнуть, что речь идет о конкретном экземпляре — либо таком, который уже упоминался выше по контексту, либо таком, который подразумевается контекстом. В английском языке для этого используется артикль «the», в русском же языке такого нет.

Если речь идет об экземпляре, уже упоминавшемся выше по контексту, то используются слова «этот», «данный», «текущий», «упомянутый (выше)», «описанный (выше)» и т. п. Мы рекомендуем упорядочить употребление этих слов следующим образом:

Таблица 6. Упорядоченное употребление указательных слов

Местоимение «этот» указывает на экземпляр, упоминавшийся в предшествующем (но не в более раннем) предложении либо в предшествующей части сложного предложения	Пользователь создает каталог для хранения архивных файлов. Этот каталог размещается на любом доступном устройстве локального компьютера
Прилагательное «данный» указывает на конкретный, ранее четко идентифицированный экземпляр, который является основной темой абзаца	Компонент Анализ используется для обработки массивов значений, полученных от других компонентов. Массивы значений импортируются из файлов, вводятся пользователем или рассчитываются автоматически. Данный компонент позволяет применить к ним методики статистического анализа
Прилагательное «текущий» указывает на экземпляр, находящийся в фокусе ввода, просмотра или подвергаемый обработке в описываемый момент	Если текущая запись не содержит сведений, задача не выполняется
Прилагательные «упомянутый (выше)» и «описанный (выше)» указывают на экземпляр, который был упомянут/описан в одном из предшествующих абзацев (но не в другом структурном элементе)	В этом случае применяется описанная выше процедура форматирования

Все перечисленные слова должны употребляться так, чтобы не возникало путаницы с тем, к какому именно экземпляру отсылает «артикуль». Чтобы текст воспринимался достаточно легко, следует минимизировать употребление таких слов. Следует иметь в виду: эти слова легко становятся словами-паразитами. Выше мы уже приводили пример, когда, попросту сняв слово «данная», автор только упростил бы текст. Конечное решение всегда зависит от конкретного контекста; мы лишь намечаем возможные пути решения.

Если речь идет об экземпляре, который не упоминался выше по контексту, но некоторым образом подразумевается этим контекстом, подобрать слово несколько труднее.

Пример. Имеется в виду ситуация, когда, например, описывается вызов различных функций с помощью кнопок панели управления:

Чтобы активизировать функцию, нажмите на соответствующую кнопку.

Очевидно, автор имел в виду, что для вызова конкретной функции используется вполне определенная кнопка. Однако слово «соответствующую» здесь явно неуместно: встретив его, читатель будет вынужден самостоятельно догадываться, какая именно кнопка имеется в виду. Весьма громоздкий «артикуль» только сделал изложение туманнее, но ничего не прояснил.

Мы вообще не рекомендуем употреблять слово «соответствующий» без поясняющих слов.

Пример. Можно сказать: «...нажмите на кнопку, соответствующую этой функции в таблице 2» (если в таблице 2 перечислены функции и кнопки, как показано выше). Однако такие формулировки, как «...нажмите на соответствующую кнопку», с нашей точки зрения, крайне нежелательны.

В таких случаях следует либо кардинально перестроить изложение (как это показано выше), либо использовать слова и

выражения, которые позволяют в явном виде сформулировать принцип соответствия.

Пример. В веб-приложении настраивается профиль пользователя по интересам. Настройка выполняется в форме, в которой устанавливаются или снимаются флажки, и в зависимости от этого в профиль включаются те или иные группы интересов. При этом дать полный список групп интересов (как в приведенном выше примере был дан полный список функций и соответствующих им кнопок) невозможно: подразумевается, что этот список будет меняться с течением времени. Важно сформулировать общий принцип настройки. Исходно в тексте говорится:

Чтобы включить группу интересов в профиль, установите соответствующий флажок.

Мы полагаем, что следует писать более конкретно:

Чтобы включить группу интересов в профиль, установите флажок напротив названия этой группы интересов.

К «артиклям» примыкают, хотя и не относятся в полной мере, слова «подобный», «такой», «аналогичный». Они употребляются в тех случаях, когда необходимо указать на некий объект, не совпадающий с упомянутым выше, но сходный с ним по ряду существенных признаков. Например:

Если массив не содержит минимального набора значений, он не подлежит обработке компонентом **Анализ**. Подобные массивы обрабатываются вручную.

Мы рекомендуем употреблять в этих случаях слово «подобный». Слово «такой» имеет весьма расплывчатое общезыковое значение, поэтому его лучше избегать; слово же «аналогичный» лучше употреблять в тех случаях, когда что-либо поясняется по аналогии. Например:

Учетная запись пользователя аналогична карточке в отделе кадров: она содержит фамилию, имя, отчество и другие данные о пользователе.

4.2. Формулировки

4.2.1. Требования к формулировкам

Мы оперируем здесь понятием *формулировки*, хотя строгого определения этому понятию не даем. Под формулировкой мы понимаем словесное выражение некоторой единой мысли, некоторого суждения относительно тех или иных аспектов функционирования программного продукта. Именно мысль, суждение — то, что автору любого текста, в том числе техническому писателю, приходится формулировать, т. е. подыскивать словесное выражение.

Все формулировки, употребляемые в документации пользователя, должны соответствовать нормам русского литературного языка. Помимо этого, к формулировкам предъявляются следующие требования:

- строгость;
- лаконичность;
- завершенность;
- однозначность.

Формулировки являются плодом творчества технического писателя, и в дальнейшем мы будем обсуждать их именно с этой точки зрения. Между тем, нельзя забывать, что в процессе работы над документацией нами ведется и по мере необходимости пополняется список заготовок для формулировок — *фигур описания*. О фигурах описания подробнее см. п. 3.5.2.

4.2.2. Строгость

Строгость формулировки определяют следующие качества:

- употребление терминов для обозначения специальных понятий, относящихся к предметной области, области информационных технологий или только к документируемому программному продукту;
- последовательное употребление вспомогательной терминологии;

- употребление общезыковой лексики только нейтрального стиля и в соответствии с общепринятыми значениями, закрепленными в толковых словарях;
- точность употребления слов и выражений вообще и терминов в особенности.

Об употреблении терминов и его упорядочении см. п. 4.1.1.

О вспомогательной терминологии и ее упорядоченном употреблении см. п. 4.1.2.

Употребление общезыковой лексики только нейтрального стиля предполагает, в частности, отказ от любых форм экспрессии и экспрессивной оценочности. Не следует ни под каким видом выражать ни положительные, ни отрицательные эмоции по поводу документируемого программного продукта в целом, его отдельных аспектов, а в особенности лиц, взаимодействующих с программным продуктом.

Точность употребления слов и выражений предполагает, что автор не позволяет себе употреблять для обозначения того или иного понятия:

- термины с более широким значением;
- термины с более узким значением;
- термины, значение которых близко к обозначаемому понятию, но не совпадает с ним.

Пример. Если программа позволяет обрабатывать файлы как растровых, так и векторных форматов, допускается писать:

Программа обрабатывает файлы графических форматов.

Если же программа применима только к файлам растровых форматов, такая формулировка будет нестрогой. Следует писать:

Программа обрабатывает файлы растровых графических форматов.

При употреблении слов и выражений общезыкового характера следует отдавать предпочтение словам и выражениям с более узким и, как следствие, более точным значением перед слова-

ми и выражениями близкими по смыслу, но с более широким и общим значением.

Пример. Вместо «большой», в зависимости от контекста, употребляется «значительный», «существенный», «емкостительный» и т.д.; вместо «высокий» — «имеющий значительный размер по вертикали», «расположенный на значительной высоте», «обладающий более высоким тоном» и т.д.

4.2.3. Лаконичность

Лаконичность формулировок сама по себе трудно формулируемое качество. Формулировка тем лаконичнее, чем меньше она включает в себя самостоятельных предложений, частей сложного предложения, причастных, деепричастных и других обособленных оборотов. Очень часто лаконичность формулировок возрастает за счет использования специальных средств подачи информации — перечислений и таблиц. При этом надлежит помнить следующие правила:

- каждое предложение должно по возможности представлять собой либо законченную формулировку какой-либо мысли, либо часть такой формулировки (но не включать в себя сразу несколько формулировок);
- если предложение можно разбить на несколько предложений без ущерба для его смысла, лучше сделать это;
- текст, состоящий из большого количества лаконичных формулировок, легче воспринимается, чем более короткий текст, состоящий из менее лаконичных формулировок.

Пример. В тексте говорится:

Компонент **Анализ** позволяет анализировать вводимые пользователем данные финансового состояния компании, причем благодаря интеграции с компонентом **Контракты** он позволяет также анализировать данные, извлеченные из договорных документов, и передавать результаты анализа в компонент **Отчеты** для подготовки отчетов.

Такое предложение следует разбить на несколько предложений, каждое из которых представляло бы собой формулировку законченной мысли:

Компонент **Анализ** предназначен для анализа данных финансового состояния компании. Он позволяет анализировать данные следующего характера:

- введенные пользователем;
- извлеченные из договорных документов.

Возможность анализа данных, извлеченных из договорных документов, предоставляется благодаря интеграции компонента **Анализ** с компонентом **Контракты**. Результаты анализа из компонента **Анализ** передаются в компонент **Отчеты**. В компоненте **Отчеты** на основе результатов анализа подготавливаются отчеты.

Как видно из примера, текст может стать более пространным, но вместе с тем и более структурированным — за счет обособления формулировок и разделения утверждений. Читать такой текст дольше, но понимать проще.

4.2.4. Завершенность

Завершенность формулировки — качество, которым зачастую пренебрегают, хотя оно играет важную роль в обеспечении легкого и комфортного восприятия читателем документации. Очень часто именно незавершенность формулировок становится причиной отсутствия взаимопонимания между автором текста и его читателем: автор считает, что сообщил все необходимые сведения, тогда как читатель все время ощущает некоторую недосказанность. Это тем более прискорбно, что незавершенность формулировок часто вызвана желанием автора писать лаконично, не перегружая текст излишествами.

Как известно, слова вступают в связи с другими словами, в результате чего и образуется такое явление, как связный текст. При этом каждое слово (разумеется, кроме служебных частей речи, т. е. предлогов, союзов и частиц) обладает обязательными связями (языковеды называют их валентностями, по аналогии с химией). Например, слово «результаты» предполагает в обяза-

тельном порядке поясняющее слово, которые бы указывало первопричину этих результатов: «результаты анализа»; «результаты форматирования»; «результаты моделирования». Употребленное без таких поясняющих слов, слово «результаты» выглядит частью некоторого незавершенного целого.

Слово может иметь несколько валентностей.

Пример. Слово «передается» предполагает три поясняющих слова — что передается, откуда передается и куда передается:

...данные передаются из компонента **Анализ** в компонент **Отчеты**.

Если опустить, скажем, слова «из компонента **Анализ**», то оставшееся словосочетание опять-таки выглядит частью незавершенного целого.

Формулировка считается завершенной, если все валентности всех слов заполнены.

Пример. Завершенная формулировка:

Результаты анализа из компонента **Анализ** передаются в компонент **Отчеты**.

Незавершенная формулировка:

Результаты анализа передаются в компонент **Отчеты**.

Завершенная формулировка:

Анализ данных возможен благодаря интеграции компонента **Анализ** с компонентом **Контракты**.

Незавершенная формулировка:

Анализ возможен благодаря интеграции с компонентом **Контракты**.

Все формулировки в документации пользователя по возможности должны отвечать требованию завершенности. Иногда ав-

тору может показаться, что смысл формулировки очевиден из контекста.

Пример. В тексте говорится:

В компоненте **Анализ** осуществляется анализ данных, извлеченных из договорных документов. Анализ возможен благодаря интеграции с компонентом **Контракты**.

В данном случае читателю, казалось бы, очевидно, что во втором предложении имеется в виду анализ данных и интеграция компонента **Анализ** с компонентом **Контракты**. Однако читатель знакомится с излагаемыми сведениями впервые; он, вполне возможно, и не знает толком, что такое, к примеру, интеграция. Ему не так просто догадаться, интеграция чего и с чем имеется в виду. Предпочтительнее использовать завершенную формулировку:

Анализ данных возможен благодаря интеграции компонента **Анализ** с компонентом **Контракты**.

Читатель, даже не зная, что такое интеграция, мгновенно поймет, что речь идет о какой-то важной возможности совместного использовании двух компонентов, и будет читать дальше. В противном случае он задумается над смыслом конкретной формулировки и может потерять нить изложения.

4.2.5. Однозначность

4.2.5.1. Об однозначных и неоднозначных формулировках

Однозначность формулировок — требование очевидное: если читатель может истолковать одну и ту же формулировку двумя разными способами, это опасно.

Документация пользователя может отвечать всем требованиям — содержать объективно корректные, актуальные, полные и детальные сведения, быть правильно структурированной и терминологически безупречной, однако неоднозначное понимание пользователем той или иной формулировки чревато самыми

серьезными последствиями. Но даже в тех случаях, когда мы рискуем не ввести пользователя в заблуждение, а лишь затруднить ему понимание материала, следует избегать неоднозначных формулировок.

Технический писатель воспитывает в себе что-то вроде профессионального невроза: каждый раз, сформулировав важную мысль, он в фоновом режиме задает себе вопрос — а нельзя ли это понять как-то по-другому, то есть неправильно? Однако в некоторых случаях это общее ощущение тревожности должно заметно усиливаться. При написании текста существуют своего рода зоны риска, о которых сейчас и пойдет речь.

Следует выделить следующие зоны риска возникновения неоднозначных формулировок:

- модальные глаголы и выражения;
- формы множественного и единственного числа;
- союзы «и» и «или»;
- деепричастия;
- возвратные формы глаголов.

4.2.5.2. Модальные глаголы и выражения

К модальным относятся глаголы и выражения, используемые для передачи отношения пользователя к тем или иным действиям или явлениям. Нами различаются следующие модальности и передающие их слова и выражения:

- долженствование: «быть должным», «быть обязанным» и т. п.;
- запрет: «не разрешается», «недопустимо», «запрещается» и т. п.;
- желание: «хотеть», «желать» и т. п.;
- возможность: «мочь», «быть в состоянии» и т. п.;
- невозможность: «невозможно», «нельзя» и т. п.;
- необходимость: «нужно», «необходимо», «надо» и т. п.

Модальные глаголы и выражения, передающие отношения долженствования, к пользователю, как правило, неприложимы (за исключением внешних по отношению к программе предписаний). Пользователь не может быть что-либо кому-либо должен в части ис-

пользования программы. Зачастую словосочетания «пользователь должен», «пользователю надлежит» используются для оформления процедур. Помимо неэтичности этой формулировки, она вызывает недоумение: какого рода обязанности возлагаются на пользователя? Являются ли эти обязанности служебными обязанностями пользователя как сотрудника организации, являются ли они частью обязательств пользователя по лицензионному соглашению или это просто предписания в рамках руководства пользователя? Мы рекомендуем использовать для оформления процедур глаголы в повелительном наклонении. Например:

Для этого нажмите на кнопку **Пуск**.

Либо в изъявительном наклонении (3-е лицо, единственное лицо). Например:

Для этого оператор нажимает на кнопку **Пуск**.

Если же в документации содержится отсылка к должностным обязанностям пользователя (см. об этом в п. 2.3.2), при их описании словосочетания «пользователь должен», «пользователь обязан» как раз вполне уместны. Например:

Пользователь обязан, в соответствии с должностными обязанностями, сохранять в тайне свои идентификационные параметры.

Аналогичным образом, если в документации присутствуют какие-либо указания, основанные на лицензионном соглашении, они оформляются с помощью тех же словосочетаний, с указанием оснований. Например:

В соответствии с лицензионным соглашением, пользователь должен сохранять в тайне номер и код регистрации.

Модальные глаголы и выражения, передающие отношения запрета, к пользователю, как правило, также неприложимы (за исключением внешних по отношению к программе предписаний). Запрет часто используется в тех случаях, когда необходимо

предостеречь пользователя от каких-то неверных действий. Как бы ни были фатальны последствия неверного действия, мы не рекомендуем использовать для предостережения форму запрета. Запрет, как и долженствование, не только неэтичен, но и неоднозначен. Читатель не знает, насколько этот запрет строг, от кого он исходит, что будет, если его нарушить. Мы предлагаем пользоваться фигурой описания «не рекомендуется» или (в особых случаях) «категорически не рекомендуется», причем в предостережении указываются последствия неверного действия.

Пример. Не кажется удачной формулировка:

Запрещается самостоятельно редактировать системные справочники.

Лучше:

Категорически не рекомендуется самостоятельно редактировать системные справочники. Это может привести к утрате работоспособности всей системы.

Опять-таки запрет допустим, если он относится к описанию должностных обязанностей пользователя или лицензионных ограничений (см. выше).

Модальные глаголы и выражения, передающие желание, в документации пользователя неуместны. Как уже говорилось, технический писатель апеллирует не к желаниям пользователя, а к сложившимся намерениям. Зачастую глаголы «хотеть» или «желать» используются именно для формулирования намерений:

Если вы хотите создать новый объект...

Однако употребление этих глаголов также приводит к расплывчатости формулировок: непонятно, речь идет о действиях, непосредственно ведущих к желаемому результату, или о каких-то подготовительных действиях (например, о предварительной настройке программы). Вдобавок глаголы желания часто выступают в паре с выражениями долженствования:

Если вы хотите создать новый объект, вы должны выполнить следующие действия...

Мы рекомендуем вместо этого использовать придаточные цели в сочетании с повелительным наклонением:

Чтобы создать новый объект, выполните следующие действия...

Подобные формулировки и лаконичнее, и однозначнее.

Модальные глаголы и выражения, передающие отношение возможности, следует употреблять в документации пользователя с большой осторожностью. Формулировки с глаголом «мочь» не всегда понятны по смыслу — настолько многозначен этот глагол.

Пример. В тексте говорится:

Число и состав справочников может меняться по мере роста и развития системы.

Что это означает — пользователь имеет возможность менять число и состав справочников, или же пользователь имеет шанс столкнуться с тем, что вчера в системе были такие-то справочники, а сегодня уже другие. И что означает «...может...» — что есть принципиальная возможность изменить число и состав справочников или что такое изменение не скажется отрицательно на работоспособности системы? В таких случаях мы рекомендуем воздерживаться от использования модальных глаголов и выражений и использовать более явное указание на субъект действия, например:

По мере роста и развития системы пользователь по мере необходимости меняет число и состав справочников.

Не рекомендуются к употреблению фигуры описания вроде «вы можете»:

В этом режиме работы вы можете выполнять следующие действия...

Предпочтительнее употребить более специализированное выражение, которое бы в явном виде отражало суть отношений пользователя и программы:

В этом режиме пользователю доступны следующие действия...

Модальные слова, обозначающие невозможность того или иного действия, также не следует употреблять в документации пользователя. Слова «невозможно», «нельзя» и им подобные слишком многозначны, их следует заменять на более специализированные выражения.

Пример. Неудачная формулировка:

В режиме просмотра нельзя редактировать свойства объекта.

Предпочтительнее:

В режиме просмотра возможность редактирования свойств объекта не предоставляется.

Слова, обозначающие необходимость того или иного действия, рекомендуется употреблять только в формулировках общей постановки задачи. Например:

Если необходимо отредактировать графический образ, воспользуйтесь редактором изображений, поставляемым вместе с программой.

Очень часто слова «необходимо», «нужно», «надо» используются при оформлении процедур.

Пример. В тексте говорится:

Чтобы создать новый объект, необходимо...

Или

Если необходимо создать новый объект, выполните следующие действия...

Как уже говорилось, мы рекомендуем в таких случаях использовать придаточные цели в сочетании с повелительным наклонением:

Чтобы создать новый объект, выполните следующие действия...

При использовании довольно выразительных слов «необходимо», «нужно», «надо» неминуемо возникают вопросы: необходимо ли? кому необходимо? зачем необходимо? Кроме того, могут возникнуть коллизии, связанные с тем, что одно и то же слово употребляется и в более общей постановке задачи, и в конкретных инструкциях. Например:

Если необходимо отредактировать графический образ, необходимо воспользоваться редактором изображений, поставляемым вместе с программой.

Все это побуждает нас ограничить употребление модальных слов со значением необходимости.

Разумеется, все сказанное не означает, что в документации пользователя не могут употребляться такие слова, как «мочь», «нельзя», «невозможно». Мы лишь обращаем внимание наших читателей на проблемы, которые при этом могут возникнуть.

4.2.5.3. Формы множественного и единственного числа

Зачастую в документации описываются свойства одиночного объекта, но чтобы подчеркнуть, что эти свойства распространяются на все подобные объекты, употребляется форма множественного числа.

Пример. В тексте говорится:

При необходимости пользователь удаляет объекты из рабочей области главного окна программы.

Такое употребление множественного числа делает возможным неоднозначное понимание формулировки: непонятно,

идет речь обо всех объектах или о произвольном их наборе. Более однозначной была бы формулировка:

При необходимости пользователь удаляет тот или иной объект из рабочей области главного окна программы.

Если же необходимо подчеркнуть возможность действия с несколькими объектами, то:

При необходимости пользователь по своему выбору удаляет один или несколько объектов из рабочей области главного окна программы.

Или:

Для пользователей администратором создаются учетные записи.

В этом случае непонятно, как соотносятся пользователи и учетные записи: может ли, скажем, быть несколько учетных записей у одного пользователя. Предпочтительнее формулировка:

Для каждого пользователя администратором создается одна или несколько учетных записей.

Рекомендуется придерживаться следующих правил при употреблении единственного и множественного числа:

- Если утверждение может быть сделано относительно одного объекта, употребляется форма единственного числа, например:

Чтобы установить шрифт, выполните следующие действия...

- Если утверждение в принципе относится к набору объектов, причем набор произволен, употребляется фигура описания «один или несколько...», например:

Укажите мышью один или несколько объектов.

- Если в утверждении подчеркивается возможность совершить действие в отношении сколь угодно большого количества объектов, употребляется фигура описания «сколь угодно много», например:

Администратор имеет право создать для пользователя сколь угодно много учетных записей.

4.2.5.4. Союзы «и» и «или»

Употребление союзов «и» и «или» для соединения однородных членов предложения также приводит к неоднозначному пониманию формулировок. Это связано с многозначностью союзов «и» и «или».

Союзы «и» и «или» употребляются для соединения двух или нескольких однородных членов предложения.

Если союз «и» соединяет два однородных члена, это соединение может иметь разный смысл.

Пример. Рассмотрим предложение:

По завершении процесса автоматически выполняются расчет проектных значений и сортировка массива.

Оно может интерпретироваться двояким образом: как описание одновременного выполнения расчета и сортировки и как описание их последовательного выполнения.

Чтобы избежать такого смешения смыслов, конструкции с союзом «и» заменяются на конструкции, близкие по значению, но более однозначные (таблица 7).

**Таблица 7. Конструкции с союзом «и»
и их замена на более точные формулировки**

Значение	Замена
Простое перечисление: В программе предусмотрены объекты типа <i>Счет</i>, типа <i>Счет-фактура</i> и типа <i>Выписка</i>	Заменяется на структурированное перечисление из трех пунктов В программе предусмотрены объекты следующих типов: • <i>Счет</i>; • <i>Счет-фактура</i>; • <i>Выписка</i>
Перечисление двух последовательных шагов процедуры: Чтобы удалить объект, нажмите на кнопку Удалить и в открывшемся диалоговом окне нажмите на кнопку ОК	Заменяется на два последовательных нумерованных шага: Чтобы удалить объект, выполните следующие действия: 1. Нажмите на кнопку Удалить. 2. Нажмите на кнопку ОК
Описание двух одновременных событий: При нажатии на кнопку осуществляются замена старых данных на новые в клиентской базе данных и добавление записей в базу данных на сервере	Заменяется на конструкцию, где одновременность выражена явно: При нажатии на кнопку осуществляется замена старых данных на новые в клиентской базе данных одновременно с добавлением записей в базу данных на сервере
Описание двух последовательных событий: По завершении процесса автоматически выполняются расчет проектных значений и сортировка массива	Заменяется на конструкцию с подчеркиванием последовательности действий: По завершении процесса автоматически выполняется сначала расчет проектных значений, а затем сортировка массива
Указание широты охвата того или иного явления: Параметр принимает положительные и отрицательные значения	Заменяется на конструкцию с союзом «как... так и»: Параметр принимает как положительные, так и отрицательные значения
Описание двух сторон одного и того же явления, неотделимых друг от друга: В критической ситуации происходят остановка всех процессов и завершение работы приложения	Заменяется на конструкцию, где это единство выражено явно: В критической ситуации происходит завершение работы приложения, сопровождающееся остановкой всех процессов

Если союз «или» соединяет два однородных члена, это соединение также может иметь разный смысл.

Пример. Рассмотрим предложение:

Ключ защиты информации подключается к LPT-порту или к USB-порту.

Оно может интерпретироваться двояким образом, как описание разных возможностей одного ключа или как описание разных видов ключей.

Чтобы избежать такого смешения смыслов, конструкции с союзом «или» заменяются на конструкции, близкие по значению, но более однозначные (таблица 8).

Таблица 8. Конструкции с союзом «или» и их замена на более точные формулировки

Значение	Замена
Простое перечисление альтернативных возможностей одного и того же объекта: Изображение импортируется в программу в формате GIF или в формате JPEG	Заменяется на структурированное перечисление из двух пунктов: Изображение импортируется в программу в одном из следующих форматов: • GIF; • JPEG
Описание разных способов выполнения одного и того же действия: Нажмите на кнопку Открыть или выберите пункт Открыть в меню Файл	Заменяется на структурированное перечисления способов выполнения действия: Выполните действие одним из двух способов: • Нажмите на кнопку Открыть. • Выберите пункт Открыть в меню Файл
Описание двух разных видов объекта: Ключ защиты информации подключается к PC MCIA-порту или к USB-порту	Заменяется на явное описание двух видов объекта: Существует два вида ключа защиты информации, один из которых подключается к PC MCIA-порту, другой к USB-порту

Значение	Замена
Описание альтернативных сценариев развития ситуации: Поиск завершится появлением результатов в панели результатов поиска или выдачей сообщения о том, что объектов, удовлетворяющих условиям поиска, в базе нет	Заменяется на явное структурированное описание двух альтернативных сценариев: Поиск завершается одним из способов: • результаты отображаются в панели результатов поиска; • выдается сообщение о том, что объектов, удовлетворяющих условиям поиска, в базе нет
Противопоставление двух разных, взаимоисключающих вариантов: Если сигнал отсутствует, это связано с неактивным состоянием датчика или с нарушением связи с сервером	Заменяется на противопоставление с союзом «либо... либо»: Если сигнал отсутствует, это связано либо с неактивным состоянием датчика, либо с нарушением связи с сервером
Обычное перечисление объектов, функций, возможностей и т. п. Для просмотра рисунков воспользуйтесь программой Imaging или программой IrfanView	Заменяется на конструкцию, в которой эта мысль выражена более явно: Для просмотра рисунков воспользуйтесь какой-либо доступной программой просмотра, например Imaging или IrfanView

Если союз «и» или «или» соединяет более чем два однородных члена, конструкция с союзом заменяется на структурированное перечисление. В структурированном перечислении не используются союзы, поэтому соединительное или разделительное значение должна иметь титульная фраза перечисления.

Пример. В тексте говорится:

Главное окно программы включает в себя меню, панель инструментов, рабочую область и строку состояния.

Лучше заменить на:

Главное окно программы включает в себя следующие составные части:

- меню;
- панель инструментов;

- рабочая область;
- строка состояния.

(титulusная фраза имеет соединительное значение).

Пример. В тексте говорится:

Обновление антивирусных баз осуществляется с компакт-диска, по локальной сети или по Интернету.

Лучше заменить на:

Обновление антивирусных баз осуществляется одним из следующих способов:

- с компакт-диска;
- по локальной сети;
- по Интернету.

(титulusная фраза имеет разделительное значение).

4.2.5.5. Деепричастия

В документации пользователя деепричастия рекомендуется использовать по минимуму, поскольку они зачастую порождают неоднозначные формулировки. Деепричастие используется для обозначения действия, побочного по отношению к действию, обозначаемому подлежащим и сказуемым. При этом логические и временные отношения между основным и побочным действием неясны.

Пример. В тексте говорится:

Нажав на кнопку, пользователь сохраняет результаты работы на диске.

Здесь неясно, происходит ли сохранение в результате нажатия кнопки или в результате действий, которые последовали за нажатием кнопки. Поэтому рекомендуется заменять такие формулировки на сходные по смыслу, но не допускающие многозначности:

В результате нажатия на кнопку результаты работы сохраняются на диске.

Или:

После нажатия на кнопку пользователь сохраняет результаты работы на диске.

К этому ограничению не следует относиться как к догме. Во многих случаях деепричастия и деепричастные обороты не порождают никакой двусмысленности. Однако следует относиться с подозрением к этой части речи и по возможности заменять обороты с ее участием на причастные обороты, придаточные и самостоятельные предложения.

4.2.5.6. Возвратные формы глаголов

Возвратные формы глаголов (глаголы, оканчивающиеся на «-ся/сь») допускается употреблять с серьезными ограничениями. Следует помнить, что присоединение к глаголу возвратной частицы совсем не обязательно означает образования формы пассивного залога.

Примеры. Слово «выполняться» далеко не всегда значит выполняться кем-то; в отношении процесса оно может иметь значение «протекать, стремиться к завершению»:

После нажатия на кнопку выполняется сжатие базы данных.

Или слово «изменяться» означает, как правило, вовсе не чье-либо целенаправленное действие по изменению чего-либо, но просто изменение чего-либо:

Значения параметров изменятся на принятые по умолчанию.

Для обозначения конкретных действий пользователя категорически не рекомендуется употреблять возвратные формы глаголов.

Пример. В тексте говорится:

После этого переключатель **Методы** устанавливается в положение **Расширенный**.

Следует заменить на:

После этого установите переключатель **Методы** в положение **Расширенный**.

Или на:

Пользователь устанавливает переключатель **Методы** в положение **Расширенный**.

Или на:

Следует установить переключатель **Методы** в положение **Расширенный**.

Для общего описания действий пользователя также не рекомендуется употреблять возвратные формы глаголов.

Пример. В тексте говорится:

Анализ данных выполняется пользователем с помощью программ сторонних производителей.

Целесообразнее использовать форму активного залога:

Пользователь выполняет анализ данных с помощью программ сторонних производителей.

Вместе с тем, это ограничение имеет более мягкий характер, чем отказ от возвратных форм для обозначения конкретных действий пользователя. Иногда можно даже приветствовать употребление возвратных глаголов.

Пример. В тексте говорится:

В зависимости от настроек, либо резервное копирование выполняется автоматически, либо его при необходимости выполняет пользователь.

Формально говоря, всё изложено корректно и по содержанию, и по форме. Однако для облегчения восприятия хотелось бы написать так, чтобы близкие по природе действия и описывались сходным образом, например:

В зависимости от настроек, резервное копирование выполняется либо автоматически, либо пользователем при необходимости.

Хотя рекомендация употреблять для обозначения действий пользователя глаголы в активном залоге во втором варианте нарушается, этот вариант представляется более удобным для восприятия.

Однако в том случае, если мы употребляем возвратный глагол для общего описания действий пользователя, необходимо в явном виде указать, кто его выполняет.

Пример. В тексте говорится:

Перед обработкой файла создается его резервная копия.

Следует формулировать более точно:

Перед обработкой файла пользователем создается его резервная копия.

При описании действий, которые выполняются без участия пользователя, в обязательном порядке подчеркивается происхождение этих действий:

Перед обработкой файла автоматически создается его резервная копия.

Иногда целесообразно прямо указать программное средство, за счет функционирования которого выполняется действие:

После этого в программе автоматически выполняется обработка данных.

Говоря об ограничениях, налагаемых на возвратные формы глаголов, следует помнить, что существуют глаголы, оканчивающиеся на «-ся/сь», уже давно утратившие связь с родственными словами без «-ся/сь» и представляющие собой слова с самостоятельным значением, например «стремиться», «появляться» и т. д. Разумеется, на их употребление подобные ограничения не распространяются.

Глава 5

Некоторые проблемы изложения материала

5.1. Громоздкие и ветвящиеся процедуры

Одной из самых удобных и легко усваиваемых форм подачи материала в документации пользователя являются процедуры. Перед пользователем четко ставится цель и затем пошагово описываются действия, необходимые для достижения этой цели. Такое изложение сведений наглядно и прозрачно для тех, кто пытается вникнуть в суть выполняемых действий, и в то же время оптимально для тех, кто предпочитает следовать четким указаниям. Наконец, процедура очень эффектно смотрится как на бумажной странице, так и на экране, естественным образом обособляясь от окружающего текста.

К сожалению, всякий, кому приходилось документировать достаточно сложные программные продукты, сталкивался с проблемой, которая в значительной степени сводит на нет наглядность и прозрачность процедур. Дело в том, что предусмотренная разработчиками последовательность действий пользователя очень часто состоит из чересчур большого количества шагов или, хуже того, ветвится на том или ином шаге: в зависимости от действий пользователя и/или заданных им значений дальнейшая последовательность действий может существенным образом различаться.

Пример. Рассмотрим фрагмент процедуры:

1. В раскрывающемся списке **Тип** выберите тип формируемого файла: Текстовый или Графический.
2. В случае если выбран тип Текстовый, выполните следующие действия:
 - a) Укажите следующие сведения о файле:
 - Таблица кодировки символов. Раскрывающийся список **Кодировка**. Доступны варианты **Windows-1251**, **DOS** и **KOI8-R**.
 - Символ-разделитель, используемый при экспорте данных, имеющих табличную структуру. Раскрывающийся список **Разделитель**. Доступны варианты **точка с запятой**, **двоеточие** и **запятая**.
 - b) При необходимости скорректировать содержимое текстового файла нажмите на кнопку **Редактировать**. Откроется окно встроенного текстового редактора, в котором отобразится текст.
 - c) Отредактируйте текст.
3. В случае если выбран тип Графический, выполните следующие действия:
 - a) Укажите следующие сведения о файле:
 - Формат файла. Раскрывающийся список **Формат**. Доступны варианты **JPEG** или **PNG**.
 - Размер файла по горизонтали и по вертикали. Поле **DPI**.
 - b) При необходимости вырезать фрагмент изображения, нажмите на кнопку **Вырезать**. Откроется окно встроенного графического редактора, в котором отобразится изображение.
 - c) Подведите указатель мыши к предполагаемому углу (любому из четырех) вырезаемого фрагмента, нажмите на левую клавишу мыши и, не отпуская ее, переместите указатель мыши так, чтобы пунктирная граница очерчивала вырезаемый фрагмент. Добившись требуемого результата, отпустите клавишу мыши.
4. Введите имя формируемого файла без расширения.
5. Задайте местоположение формируемого файла. Для этого нажмите на кнопку **Обзор** и укажите каталог в стандартном окне Windows.
6. Нажмите на кнопку **Сформировать файл**. Файл будет сформирован в соответствии с заданными настройками и помещен в указанный каталог.

В приведенном примере хорошо видно, как линейная последовательность процедуры нарушается в результате ветвления: в зависимости от выбора типа файла выполняется та или другая последовательность действий. Это означает, что пользователь уже не может рассматривать процедуру как однозначно определенную последовательность шагов: он должен держать в уме определенное условие и, в зависимости от него, выполнять последовательность 2 или 3.

Такую процедуру сложно воспринимать как единое целое: пользователь путается в условных переходах и оказывается не в состоянии запомнить последовательность выполняемых действий. Наглядность процедуры при этом страдает. Если же такое ветвление встречается на протяжении процедуры несколько раз, воспринимать изложение становится совсем тяжело.

В таких случаях мы рекомендуем следовать общему правилу: если какой-то объект представляется слишком крупным, чтобы его можно было охватить одним взглядом и воспринять как целое, следует укрупнить и взгляд на него — разбить объект на составные части и рассмотреть каждую составную часть особо.

Так, если процедура содержит слишком много шагов, ее целесообразно разбить на несколько участков — *этапов*. Каждый этап включает в себя несколько шагов, объединенных некоторой общей промежуточной целью, и рассматривается особо, под отдельным заголовком. Например, если исходно процедура целиком занимала пункт (структурный элемент третьего уровня), не разделенный на подпункты, то после разбиения процедуры на этапы каждый этап рассматривается в отдельном подпункте; первым помещается «обзорный» подпункт, в котором рассматривается последовательность этапов. Хотя в результате этих действий усложняется структура описания, воспринимать описание становится проще.

Если процедура ветвится, ее также целесообразно разбить на несколько этапов — таким образом, чтобы точка ветвления (шаг, на котором выбирается та или другая ветвь процедуры) становилась началом нового этапа. Тогда вместо сплошной последовательности шагов, внешне линейной, но на деле ветвящейся,

пользователь видит перед собой аккуратное и последовательное изложение причин и характера ветвления, а затем обособленное описание каждой из ветвей.

Пример. Приведенный выше фрагмент будет выглядеть следующим образом:

Выбор типа формируемого файла и настройка его параметров

На настоящем этапе осуществляется выбор типа формируемого файла и настройка параметров этого файла, а также вносятся коррективы в предполагаемое содержимое файла.

В приложении предусмотрено формирование файлов следующих типов:

- **Текстовый.**
- **Графический.**

Чтобы задать тип формируемого файла, выберите его в раскрывающемся списке **Тип**. В зависимости от типа файла действия по настройке его параметров будут различными, что связано со спецификой текстового или, напротив, графического представления данных.

Для текстового файла (тип **Текстовый**) укажите следующие сведения:

- Таблица кодировки символов. Раскрывающийся список **Кодировка**. Доступны варианты **Windows-1251**, **DOS** и **KOI8-R**.
- Символ-разделитель, используемый при экспорте данных, имеющих табличную структуру. Раскрывающийся список **Разделитель**. Доступны варианты **точка с запятой**, **двоеточие** и **запятая**.

При необходимости в предполагаемое содержимое текстового файла вносятся исправления.

Для корректировки содержимого текстового файла:

1. Нажмите на кнопку **Редактировать**. Откроется окно встроенного текстового редактора, в котором отобразится текст.

2. Отредактируйте текст.

Для графического файла (тип **Графический**) укажите следующие сведения:

- Формат файла. Раскрывающийся список **Формат**. Доступны варианты **JPEG** или **PNG**.
- Размер файла по горизонтали и по вертикали. Поле **DPI**.

При необходимости предполагаемое содержимое графического файла может быть ограничено фрагментом (вместо целого изображения).

Для того чтобы вырезать фрагмент:

1. Нажмите на кнопку **Вырезать**. Откроется окно встроенного графического редактора, в котором отобразится изображение.
2. Подведите указатель мыши к предполагаемому углу (любому из четырех) вырезаемого фрагмента, нажмите на левую клавишу мыши.
3. Не отпуская ее, переместите указатель мыши так, чтобы пунктирная граница очерчивала вырезаемый фрагмент.
4. Отпустите клавишу мыши.
5. После того как выбраны тип формируемого файла и настроены его параметры, а также внесены коррективы в его предполагаемое содержимое, перейдите к следующему этапу.

Формирование файла

На настоящем этапе указывается имя и местонахождение формируемого файла — полный путь к нему, а также дается команда сформировать файл.

Для того чтобы сформировать файл:

1. Введите имя формируемого файла без расширения.
2. Задайте местоположение формируемого файла. Для этого нажмите на кнопку **Обзор** и укажите каталог в стандартном окне Windows.
3. Нажмите на кнопку **Сформировать файл**. Файл, сформированный в соответствии с заданными настройками, помещается в указанный каталог.

Текст стал более пространным, но и более структурированным, в нем появилось место, чтобы при необходимости вставить дополнительные комментарии к определенным действиям и ситуациям. Он ориентирован не на «идеальную», линейную последовательность шагов, а на рассмотрение различных вариантов действий пользователя в зависимости от действий, предпринятых на предыдущих шагах. Не следует опасаться увеличения объема текста, если речь идет о более подробном рассмотрении действительно сложных вопросов.

Разумеется, здесь предлагается лишь примерный рецепт выхода из сложного положения. В каждом конкретном случае он требует творческого применения. В каких-то случаях могут использоваться дополнительные способы структурирования сведений: маркированные списки, таблицы и т. д. В каких-то случаях речь пойдет об углублении структуры не на один, а к примеру, на два уровня. Однако если процедура ветвится слишком сильно, содержит многочисленные ветвления второго и третьего порядка (то есть ветви, в свою очередь, ветвятся, образуя ветви второго порядка, а те, в свою очередь, — ветви третьего порядка), есть смысл задуматься: а целесообразно ли структурировать сведения как единую процедуру?

Действительно, если пользователь на каждом шагу принимает решения, в зависимости от которых движется в дальнейшем по тому или другому пути, мы, возможно, имеем дело с пользовательской перспективой типа *Среда*: пользователю предоставляется обширный набор элементарных функций, с помощью которых он может тем или иным путем решить свои задачи. В этом случае нецелесообразно описывать действия пользователя в виде громоздкой и неудобочитаемой условной конструкции из множества шагов, переходов между ними, возвратов к предыдущему изложению и т. д. Лучше описать отдельные функции, разъяснить их назначение и по возможности снабдить пользователя методическими рекомендациями, как именно следует эти функции применять. Нелишним будет также разбор наиболее важных (рекомендуемых) частных случаев применения элементарных функций. Например, если настройка программного продукта состоит в том, что для большого количества параметров

задаются их значения, едва ли стоит выстраивать изложение как «процедуру настройки программы». Вместо этого разъясняют смысл каждого параметра и дают рекомендации по поводу того, какие настройки предпочтительнее установить как индивидуальные, какие — без особой нужды лучше не менять (оставить те, что установлены по умолчанию). Иногда рассматривают наиболее важные примеры прикладного применения программного продукта и подробно рассказывают о задаваемых в этом случае параметрах.

5.2. Порядок изложения сведений: сложные случаи

5.2.1. Проблема порядка изложения сведений

Между документацией пользователя, с одной стороны, и реальностью, которая в этой документации описывается, существует неизбывное противоречие. Документация пользователя, как любой текст на естественном языке, представляет собой линейное изложение определенных фактов, а документируемая реальность с составляющими ее объектами, соотносимыми между собой в различных плоскостях, — многомерную конструкцию. При описании даже относительно небольших фрагментов этой реальности часто возникает вопрос: в каком порядке располагать взаимосвязанные факты, чтобы изложение в наибольшей степени соответствовало общей логике существующих между ними взаимосвязей. Скажем, о чем следует писать в первую очередь — о свойствах объекта, о его отношениях с другими объектами или о действиях, которые с ним можно выполнять? Или: есть два взаимосвязанных понятия, одно из которых немыслимо без другого, — в каком порядке следует разъяснять их смысл?

При решении этих проблем невозможно дать какой-либо универсальный рецепт. Мы предлагаем вниманию коллег ряд общих рекомендаций. Разумеется, применять эти рекомендации необходимо творчески, с учетом специфики освещаемой темы и конкретной ситуации, в которой, как предполагается, будет использоваться документация.

5.2.2. Курица или яйцо: взаимосвязанные понятия

Если последовательно рассматриваются два или более тесно взаимосвязанных понятия, так что разъяснить смысл одного, не разъяснив смысла всех остальных, кажется затруднительным, следует сначала дать обзорное описание всех понятий, ввести их формальным образом, дать какие-то общие пояснения, а затем уже рассмотреть каждое понятие в отдельности.

Пример. Во многих программах понятие «учетная запись пользователя» и понятие «группа пользователей» не могут быть рассмотрены отдельно друг от друга: учетная запись пользователя создается только в рамках той или иной группы пользователей, группа же состоит из учетных записей. О чем говорить раньше; фигурально выражаясь, что было раньше — курица или яйцо? Смысл нашей рекомендации в том, чтобы сначала дать обзорное описание, в котором упомянуть о двух взаимосвязанных и не делимых друг от друга понятиях «учетная запись пользователя» и «группа пользователей», а уж затем порознь разъяснить во всех подробностях смысл каждого из них.

5.2.3. Канонический порядок изложения сведений

5.2.3.1. Понятие канонического порядка изложения сведений

В некоторых случаях порядок изложения сведений оказывается в зависимости от взгляда на тот или иной предмет. Так, можно считать, что при описании объекта заданными являются его свойства, а действия, выполняемые с этим объектом, направлены на изменения свойств. И тогда мы сначала будем описывать свойства объекта, а затем действия. А можно исходить из того, что первичны действия, а свойства приобретаются объектом в результате этих действий. И тогда порядок изложения сведений будет иным. Какой же порядок предпочесть?

Чтобы ответить на этот вопрос, мы вводим здесь понятие *канонического порядка изложения сведений*. Смысл этого понятия состоит в том, что существует порядок изложения, который воспри-

нимается читателем, пользователем, как естественный, привычный, соответствующий некоторому шаблону восприятия. Самый простой пример: мы сначала рассказываем о том, как включить прибор, а потом — как его выключить. В принципе ничто бы не мешало нам построить изложение в обратном порядке (в каком-то смысле пользователю было бы даже удобнее, поскольку прибор часто бывает нужно срочно выключить и некогда разыскивать сведения о порядке выключения по всему тексту), но пользователь воспримет этот порядок как противоестественный, аномальный, *неканонический*.

Мы на основе нашего опыта предлагаем свои схемы канонического порядка изложения сведений в следующих важных случаях:

- программный продукт и формы работы с ним;
- процесс и его стадии;
- объект и этапы его жизненного цикла.

Следует принимать во внимание, что представленные ниже схемы имеют общий характер и могут быть скорректированы в соответствии с конкретным контекстом изложения.

5.2.3.2. Программный продукт и формы работы с ним

При работе с программным продуктом предусмотрен ряд основных форм работы, которые рассматриваются в следующем каноническом порядке:

- установка программного продукта на компьютере;
- удаление программного продукта с компьютера;
- настройка программного продукта;
- начало работы с программным продуктом (запуск и/или авторизация);
- завершение работы с программным продуктом;
- целевое применение программного продукта;
- обслуживание программного продукта и обрабатываемых данных;
- дополнительная (опциональная) настройка программного продукта.

Разумеется, материал по одной или нескольким из перечисленных рубрик может отсутствовать.

Канонический порядок, как видно выше, учитывает сразу несколько факторов: последовательность действий как в пределах всего периода эксплуатации программного продукта, так и в пределах одного сеанса работы, частоту применения тех или иных сведений, удобство чтения документации и т. д.

5.2.3.3. Процесс и его стадии

Если обработка данных в программном продукте организована — полностью или в какой-то ее части — как процесс, состоящий из ряда стадий, сведения об этом процессе излагаются в следующем каноническом порядке;

- общая характеристика процесса, перечисление его стадий;
- запуск процесса;
- основные стадии процесса в их основной (штатной) последовательности;
- штатное завершение процесса;
- способы нештатного завершения процесса.

Канонический порядок изложения сведений о процессе предполагает, что сначала дается общая характеристика процесса, перечисляются стадии, потом описывается по порядку штатное протекание процесса, и в заключение, перечисляются способы нештатного завершения процесса.

5.2.3.4. Объект и этапы его жизненного цикла

Если единицей хранения данных в программном продукте является объект той или иной природы (документ, учетная запись пользователя и т. п.), сведения об объекте, его свойствах и совершаемых с ним действиях излагаются в следующем каноническом порядке:

- объект, его назначение и природа;
- свойства объекта и его статусы, обзор жизненного цикла объекта;
- создание объекта;

- редактирование свойств объекта;
- прочие действия с объектом;
- удаление объекта.

Канонический порядок изложения сведений об объекте предполагает, что сначала дается общее описание объекта, потом описываются его статичные характеристики, и в заключение, в соответствии с порядком следования этапов жизненного цикла, — действия, совершаемые с объектом.

Такой порядок изложения, в частности, позволяет решить проблему, с которой нередко сталкивается технический писатель. Создание и редактирование свойств объектов часто осуществляется, по сути, в одном и том же диалоговом окне. В связи с этим возникает проблема логического повтора: следует сначала описать процедуру создания объекта, а потом сослаться на нее при описании процедуры редактирования свойств объекта — или же поступить противоположным образом? Вопрос этот не праздный: создание объекта и редактирование его свойств — действия разные в своей основе. Не слишком логично заставлять пользователя, желающего узнать, как изменить то или иное свойство объекта, обращаться к процедуре создания объекта; кроме того, у пользователя могут попросту отсутствовать права на создание нового объекта и присутствовать — права на изменение свойств существующего объекта.

Предлагаемое нами решение основывается на каноническом порядке изложения, описанном выше. Сначала описываются свойства объекта и диалоговое окно, в котором они задаются. Это диалоговое окно используется впоследствии как при создании нового объекта, так и при редактировании свойств существующего объекта. Процедуры создания нового и редактирования свойств существующего объекта рассматриваются каждая в отдельном разделе, однако само диалоговое окно здесь подробно не описывается; на его описание в обоих разделах помещается перекрестная ссылка. После того как подробно рассмотрены свойства объекта и порядок их задания, можно описывать создание нового объекта и редактирование свойств существующего — в естественном, каноническом порядке:

Создание нового объекта

Чтобы создать новый объект:

1. Выберите в меню пункт **Новый**. Откроется диалоговое окно **Новый объект** (см. рис. 15).
2. Введите имя объекта, его идентификационный номер, а также значения других параметров (о задании свойств объекта см. п. 5.2). Для успешного создания объекта необходимо и достаточно задать обязательные свойства объекта (см. п. 5.2).
3. Нажмите на кнопку **ОК**. После этого, при условии что обязательные свойства объекта были корректно заданы, в списке объектов отобразится новый объект.

Редактирование свойств объекта

Чтобы отредактировать свойства объекта:

1. Выберите объект в списке объектов.
2. Выберите в меню пункт **Редактировать**. Откроется диалоговое окно **Свойства объекта** (см. рис. 15).
3. Отредактируйте свойства объекта (о задании свойств объекта см. п. 5.2; имя объекта и его идентификационный номер для редактирования недоступны).
4. Нажмите на кнопку **ОК**. После этого новые свойства объекта вступят в силу.

5.2.4. Унификация порядка изложения сведений

Порядок изложения сведений во всем документе (желательно — во всей документации) должен быть унифицирован. Это означает, что соблюдаются следующие правила:

- Если изложение одних и тех же сведений повторяется на протяжении документа несколько раз, оно следует одному и тому же порядку.
- Если при изложении сведений, относящихся к ряду объектов (действий, отношений и т. д.), избран определенный порядок, в дальнейшем при изложении любых других сведений, относящихся к этому ряду объектов (действий, отношений и т. д.), соблюдается этот же порядок.

Пример. Пусть перечисляется ряд параметров, задаваемых пользователем. Если ниже следует более подробное описание каждого из параметров, то эти описания следуют в том же порядке.

- Если при изложении сведений по той или иной теме избирается определенный порядок изложения, то в дальнейшем при изложении сведений по сходной теме избирается аналогичный порядок изложения.

Пример. Пусть при описании одного из параметров избран следующий порядок изложения: имя параметра — его смысл — тип значения — особые значения. Тогда и для всех остальных параметров избирается такой же порядок изложения.

5.3. Реальное и виртуальное

5.3.1. Реальное и виртуальное в документации пользователя

Программные продукты могут иметь самое разное назначение, базироваться на различных технологиях и предусматривать различные режимы эксплуатации. Однако существует черта, которая роднит прикладные программные продукты самого широкого спектра: в них так или иначе осуществляется моделирование действительности²³. Виртуальным объектам: электронным документам, массивам данных, которые подвергаются в программном продукте непосредственной обработке, — соответствуют объекты реальности: бумажные документы, показания приборов или замеров. Между первыми и вторыми всегда существует дистанция — объекты реальности, с одной стороны, и соответствующие им виртуальные объекты, с другой, подчиняются разным законам.

²³ Нельзя не заметить, что существует большое количество программных продуктов, которые не моделируют действительность, поскольку целиком и полностью обслуживают мир информационных объектов: файлов, каталогов, программ и т. д. Таковы, например, программы работы с файлами, программы архивирования файлов, программы просмотра файлов определенных форматов.

Пример. Счет как реальный (бумажный) платежный документ и счет как электронный документ программы бухгалтерского учета — два совершенно разных явления. Бумажный счет может не иметь соответствия в бухгалтерской программе, а счет как электронный документ — соответствия в реальной практике бухгалтерского учета. Бумажный счет участвует в реальном движении средств, тогда как его виртуальный образ лишь используется для ввода, хранения и обработки информации об этом движении. При этом в разных программах виртуальный образ одного и того же бухгалтерского документа может подчиняться разным требованиям и по-разному создаваться и обрабатываться — скажем, в одних случаях он в произвольной форме создается пользователем, в других порождается системой как артефакт какого-то более общего процесса — продажи товара или предоставления услуги и т. д.

Эта дистанция между реальным и виртуальным и одновременно неразрывная связь между ними ставят технического писателя перед важной проблемой: недопустимо, чтобы читатель (пользователь) смешивал одно и другое. Он должен в любой момент времени понимать, о чем идет речь, — о реальном объекте, явлении, процессе и т. п. или о виртуальном образе реальности. В противном случае он будет систематически упускать из виду важнейший аспект работы с программой — перевод реальных задач на язык виртуальных моделей и обратно.

Пример. Движение товара на складе является реальным процессом, который моделируется в программе складского учета. При поступлении на склад партии товара для нее заводится уникальная запись (создание виртуального образа для реального объекта и виртуальной модели для реального процесса движения товаров); при каждом перемещении товара в программу вводится информация об этом перемещении (изменение состояния виртуальной модели в соответствии с реальным прототипом); по итогам отдельных стадий и процесса в целом формируются выходные документы (преобразование данных компьютерной обработки в бумажные документы, используемые в реальном процессе движения товаров). Технические писатели в подобной ситуации часто увлекаются описанием внутренних механиз-

мов передачи и обработки данных в программном продукте; между тем, очень важно отразить в документации именно зазор между реальностью и моделью, рассмотреть случаи расхождения между ними, объяснить пользователю, какие реальные задачи позволит ему решить программный продукт — получить распечатки документов, оперативно получать сведения о движении товаров за любой период и т. д.

Именно поэтому особенно важно постоянно отслеживать границу между тем, что происходит в реальности, и тем, как происходящее отражается в программе.

5.3.2. Объекты реальности, их образы и идентификаторы

Многие программные продукты оперируют объектами, которые являются виртуальными образами объектов реальности. При этом между реальным и виртуальными объектами возникают не столь уж тривиальные взаимосвязи. Обычно в основе этих взаимосвязей лежит представление о соответствии виртуального объекта реальному.

Пример. В тексте вводится новое понятие:

Чтобы ввести в программу данные о сделке, пользователем создается объект типа *Сделка*.

Обратим внимание, что в приведенной фразе проводится граница между реальной сделкой и ее виртуальным образом. Иногда этим пренебрегают, что приводит к недоразумению в тех случаях, когда ставится под сомнение принцип взаимно-однозначного соответствия между реальными и виртуальными объектами. Например, для одной сделки невнимательный пользователь, вообще говоря, может создать два или более объекта типа *Сделка*. С точки зрения задач, которые ставятся перед программой, такое дублирование едва ли оправданно, тогда как технически создание объектов-«двойников» не всегда может быть заблокировано. В этом случае контроль возлагается на пользователя, о чем так или иначе придется говорить в документации, например:

Категорически не рекомендуется создавать два или более объекта типа *Сделка*, соответствующих одной реальной сделке.

Если же на уровне понятийной системы, принятой в документации, не проводится границы между реальными и виртуальными объектами, даже сформулировать эту рекомендацию будет затруднительно.

Разумеется, это не означает, что виртуальный объект необходимо постоянно обозначать громоздким выражением вроде «объект типа *Сделка*». В той части документа, где нет риска смешения реального и виртуального, виртуальный объект допускается называть попросту — сделкой. Однако необходимо в явном виде, четко и ясно оговорить такое словопотребление. Например:

В дальнейшем всюду, где это не приводит к недоразумениям, объект типа *Сделка* называется просто сделкой.

Как реальные, так и виртуальные объекты имеют свои идентификаторы. При этом далеко не всегда реальные объекты имеют уникальные идентификаторы. Между тем, виртуальные объекты одного типа, как правило, снабжаются по крайней мере одним уникальным идентификатором — номером, идентификационным кодом или названием.

Пример. Реальным объектом является помещаемая в электронный архив статья, виртуальным — та же статья в электронной форме, помещенная в архив. Реальный объект идентифицируется заголовком. Этот идентификатор несет богатую содержательную информацию, но не является уникальным. Виртуальный объект имеет уникальный номер, который не несет никакой содержательной информации, но идеально подходит для идентификации объекта в архиве. Если пользователь занят поиском материалов по какой-либо теме, ему важнее содержательный идентификатор реального объекта. Если же ему необходимо точно идентифицировать единицу хранения архива, на передний план выступает уникальный идентификатор виртуального объекта.

Рекомендуется разделять реальные и виртуальные идентификаторы. Например, для обозначения виртуального объекта вместо формулировки: «статья *Создание электронных архивов*» — пишут «статья 123, заглавие *Создание электронных архивов*», поскольку статей с таким заглавием может быть несколько, а идентификационный номер в архиве уникален.

5.3.3. Люди и организации, учетные записи, имена

Дистанция между реальным и виртуальным приобретает особое значение, когда в программном продукте используются данные о физических и юридических лицах. Очень важно провести границу между человеком или организацией, с одной стороны, и учетной записью о физическом или юридическом лице — с другой. В устной повседневной речи специалисты по информационным технологиям, а вслед за ними и пользователи говорят: «создать пользователя», «удалить организацию», — однако в документации таких выражений рекомендуется избегать, поскольку они порождают не слишком удачные или попросту невнятные формулировки.

Пример. Пользователь столкнулся с тем, что для него еще не создана учетная запись. Если использовать выражение «создать пользователя», то возникнет неудачная формулировка вроде: «для этого пользователя еще не создан пользователь». Или например, в программе предусмотрена штатная возможность создания нескольких учетных записей для одного пользователя (такие ситуации встречаются не так уж редко). Если использовать выражение «создать пользователя», то возникнет формулировка вроде: «Для одного пользователя можно создать более одного пользователя» — тоже не слишком удачная. Поэтому мы настоятельно рекомендуем использовать выражения «создать учетную запись», «удалить учетную запись» и т. д. — как для людей, так и для организаций.

Пользователь (физическое лицо) естественным порядком имеет идентификатор — фамилию, имя и отчество; строго говоря, он не является уникальным. Пользовательская

учетная запись обычно имеет уникальный идентификатор — учетное имя и/или идентификационный номер. Мы рекомендуем использовать «природные» идентификаторы только для обозначения пользователей — физических лиц вне контекста определенной пользовательской учетной записи. Если же упоминается пользовательская учетная запись или реальный пользователь как обладатель учетной записи, рекомендуется использовать учетное имя или идентификационный номер. Например, вместо «пользовательская учетная запись *Иванов Петр Петрович*» пишут: «пользовательская учетная запись *123*». Или например, можно написать:

На рис. 25 в списке пользователей выбран пользователь *Петров Иван Иванович*:

Но лучше:

На рис. 25 в списке пользователей выбран пользователь *petrov*.

Этим подчеркивается, что выбрана именно пользовательская учетная запись, а не реальное лицо, для которого, повторяем, может создаваться и несколько учетных записей.

Пример. Аналогичным образом упоминаются в документации учетные записи юридических лиц. Например, можно написать:

Из списка удалена организация ООО «Спецмонтаж».

Но лучше:

Из списка удалена учетная запись 3, наименование организации ООО «Спецмонтаж».

5.4. Образно или строго?

5.4.1. Язык образов и его место в строгом изложении

В документации пользователя следует избегать любых образных выражений, употребления слов в переносном значении, иронических и шутливых высказываний и т.д. Следует помнить: образ, метафора, ирония могут быть по-разному поняты разными людьми. Может оказаться, что то, что казалось очевидным автору, совсем не так очевидно читателю.

Иногда ссылаются на чрезвычайную, будто бы наглядную убедительность образов и метафор при разъяснении сложных понятий. Подобные средства действительно эффективны, когда речь идет о сколько-нибудь продолжительном процессе обучения: учащийся постепенно вникает в суть сообщаемых ему знаний, осмысляет предлагаемые ему сравнения и аналогии и от них переходит к усвоению строгого смысла понятий. Документация пользователя не учебное пособие; она ориентирована на оперативное освоение пользователем программного продукта и потому содержит предельно точное и лаконичное изложение необходимых пользователю сведений.

5.4.2. Антропоморфизм и техногенность

Очень часто возникает соблазн описывать функционирование программного продукта так, как будто это деятельность разумного существа. Так в документацию пользователя попадают выражения вроде «программа понимает...», «система изучает...», «приложение ожидает, чтобы...» и т.д. Мы рекомендуем использовать такие антропоморфные выражения в отношении технического средства с большой осторожностью.

В частности, не следует приписывать программному продукту сложные психические процессы, доступные лишь человеку. Программа не может ничего «понимать», система не может ничего «изучать», а приложение чего-то «ожидать». Все эти действия предполагают какие-то осознанные усилия со стороны программных средств. Между тем, за образным выражением скрываются

некие важные для пользователя сведения, которые остаются за пределами изложения. Говоря о системе, что она «изучает» данные, автор имел в виду набор действий, которые действительно выполняются системой. Что имелось в виду? Автоматический поиск со сложным условием? Фильтрация данных по заранее заданному фильтру? Применение к данным каких-то аналитических методик? Пойдя на поводу у своего воображения, технический писатель скрыл от читателя то, что последнему как раз и хотелось бы выяснить.

С другой стороны, следует избегать и слишком конкретных глаголов, которыми обычно обозначаются действия, совершаемые человеком. Например, нежелательно использовать выражения «программа рисует...», «программа печатает...» и т. д. Дело в том, что эти действия совершаются человеком не где-нибудь, а в интерфейсе. По сути дела, все эти действия суть способы ввода данных. Программа действительно может что-то «нарисовать» там, где это обычно делает человек, но смысл этого действия нуждается в объяснении с точки зрения внутренних механизмов программы: на основе каких данных сформировано изображение, с какой целью оно отображается в интерфейсе, какие действия предполагает со стороны пользователя? Пользователю было бы гораздо полезнее получить конкретную формулировку. Например:

Автоматически сформированная диаграмма отобразится в отдельном окне.

На основе данных геоинформационной системы и заданных пользователем параметров в рабочей области автоматически сформируется карта.

Проблематичность подобных формулировок хорошо видна на следующем примере:

Пример. В тексте говорится:

Программа автоматически заполнит следующие поля...

Что значит — «автоматически заполнить»? Имеет ли автор в виду, что программа подставит определенные значения по умолчанию, оставив пользователю возможность ввести другие значения вручную, если он сочтет это необходимым? Или автоматически заполненные поля сделаются недоступными для ввода/редактирования? Или в полях отобразятся действующие значения параметров, которые не подлежат изменению, а сообщаются пользователю просто для сведения? На все эти вопросы нет ответа. Было бы гораздо лучше дать более конкретную формулировку, например:

В следующих полях отобразятся значения по умолчанию, автоматически рассчитанные программой... В каждом из перечисленных полей пользователь по своему усмотрению вручную вводит другое значение или оставляет нетронутым значение по умолчанию.

В то же время программному продукту допускается приписывать отвлеченные действия общего характера: «программа выполняет...», «программа обрабатывает...» и т. д. При этом не возникает двусмысленности: программе приписываются ровно те действия, которые она и в самом деле осуществляет. Тем не менее, мы рекомендуем по возможности использовать безличную форму: «в программе выполняется...», «в программе обрабатывается...» и т. д. Таким образом подчеркивается, что программа не действует самостоятельно, а выступает лишь как средство осуществления тех или иных действий.

5.4.3. Наглядные средства подачи информации

В документации пользователя часто употребляются различные средства, позволяющие сделать изложение, особенно в наиболее тяжелых для восприятия местах, более наглядным. К таким средствам относятся разного рода блок-схемы, организационные диаграммы и тому подобные материалы.

Подобные средства и в самом деле могут сделать изложение более наглядным для пользователя. Однако следует помнить, что каждый материал такого рода в обязательном порядке должен

сопровождаться подробным текстовым объяснением. Далеко не все люди хорошо воспринимают информацию, представленную в графическом виде, и даже те, кому этот способ представления информации кажется естественным и доступным, могут неверно интерпретировать отдельные элементы схемы или диаграммы и сделать для себя неверные выводы. Графическое представление информации всегда в какой-то степени рассчитано на образное восприятие, на воображение читателя, а поэтому чревато двусмысленностями и недоразумениями.

Нельзя не сказать и еще об одном средстве, которое используется, чтобы сделать изложение более наглядным, — о примерах. Пользователь предпочитает учиться применению программного продукта не на предписаниях общего характера, а на конкретных примерах. Спору нет, при описании сложных, многоступенчатых действий пользователя бывает весьма полезно проиллюстрировать общую процедуру конкретным примером. Однако подменять примерами общие описания категорически не рекомендуется. Каждый пример — это всего лишь пример: при его рассмотрении какие-то возможности программы будут упущены из виду, какие-то ограничения и требования не будут упомянуты. Пользователь, знакомясь с примером, непременно упустит из виду что-то важное для себя, а некоторые особенности данного примера, напротив, примет за общее правило. Чтобы избежать этого, необходимо следовать одному и тому же порядку изложения — сначала общее описание, потом конкретный пример. При этом, приступая к рассмотрению примера, необходимо в явном виде указывать: рассматривается конкретный пример применения процедуры, использования функции или ряда функций. Пользователь должен уяснить: пример иллюстрирует общее описание, но не заменяет его.

Об авторе

Tech writers of Ukraine

Юрий Валентинович Кагарлицкий родился в 1967 г. в Москве. Закончил филологический факультет МГУ (1995). Кандидат филологических наук (2000). Научный сотрудник Института русского языка им. В. В. Виноградова Российской академии наук.

С 1999 г. работает в компании «Философт». Занимается документированием программных продуктов и автоматизированных систем. Преподает на курсах повышения квалификации для технических писателей, консультирует разработчиков технической документации.

О проекте

Tech writers of Ukraine

Предлагаемая книга — часть большого проекта, который осуществляет компания «Философт». Цель проекта — систематизировать и донести до российского читателя знания о технической коммуникации, обширной области, включающей в себя практики и технологии эффективного обмена технической информацией. Техническая коммуникация охватывает различные направления работы, в том числе, сбор технической информации, ее изложение, перевод, публикацию, каталогизацию, распространение. Сюда же относится разработка технической документации.

В современном мире техническая коммуникация оформилась в самостоятельную производственную отрасль. У нас в стране эта область, к сожалению, заметно отстает по сравнению с передовыми странами Запада. На развитие технической коммуникации в России и направлен наш проект. Он вызван к жизни следующими причинами.

Во-первых, большинство книг и статей по технической коммуникации написано и издано на Западе. Эти публикации ориентированы на западного читателя, на знакомую ему производственную и деловую среду, традиции документирования, издательские и стилистические стандарты. Думается, российский специалист остро нуждается в освещении тех же тем применительно к отечественной практике.

Во-вторых, насколько нам известно, в отечественных вузах до сих пор нет ни кафедр, ни даже специализированных курсов, где средства и методы технической коммуникации преподавались бы систематически. Поэтому специалист, который намерен освоить эту область, вынужден самостоятельно собирать сведения из разрозненных источников, сопоставлять их и складывать в единую картину.

Наш способ ведения этого проекта — постепенное накопление знаний по частным вопросам технической коммуникации, их систематизация и изложение в книгах. Выпуску книг предшествует публикация статей, которые вы найдете у нас на сайте: www.philosoft.ru.

Мы приглашаем заинтересованных читателей к участию в проекте. Вы можете предложить нам готовый материал для публикации или включиться в работу по нашему плану. Любое конструктивное участие будет уместным и ценным.

Пишите нам по адресу: books@philosoft.ru.

Ю. В. Кагарлицкий

Разработка документации пользователя программного продукта

{методика и стиль изложения}

Задача этой книги – помочь разработчикам технической документации создавать информативные и удобные руководства для пользователей программных продуктов. Подробно рассматриваются различные этапы создания документации: изучение программного продукта и выявление особенностей его применения; определение состава комплекта документации; отбор сведений, включаемых в документацию в целом и в каждый документ в отдельности; составление структуры отдельного документа и ее последующая корректировка; работа над текстом документации и соблюдение базовых требований к этому тексту.

Для технических писателей, программистов, инженеров служб технической поддержки.

